UsdOptixViewer

A Literate Program

Pablo Figueroa

Written in the style of Donald E. Knuth's *Literate Programming*.
The source files of the program are tangled from this document; `lp.py check`
verifies that the tangled output is identical to the files in the repository.

Contents

Introduction **1**
3. How to read the code . . . 1
4. The parts of the program . . . 1
5. A word on correctness . . . 1

The build system **3**
13. Device code . . . 4
14. The executable . . . 5
16. Embedding a file . . . 5

Shared parameters **7**
25. The launch parameters . . . 9

The scene **11**
31. Entities and components . . . 11
38. The implementation file . . . 13
40. Opening a stage . . . 14
45. The flattened geometry . . . 16
49. From prims to triangles . . . 17
62. Materials . . . 21
86. Textures . . . 28
90. Materials per face . . . 30
92. World space . . . 31

The renderer **33**
99. The interface . . . 33
109. The implementation . . . 35
115. Mipmaps . . . 37
119. Device buffers . . . 39
122. Life and death of the renderer . . . 39
127. The pipeline . . . 41
135. The shader binding table . . . 43
140. Loading a scene . . . 44
149. Textures . . . 46
155. The image and the camera . . . 48
158. Rendering . . . 49

The device programs **51**
166. Random numbers . . . 52
169. The payload . . . 52
172. Texture sampling with ray differentials . . . 53
179. Normal mapping . . . 55
182. The GGX microfacet model . . . 56
186. Ray generation . . . 57
191. Alpha cutouts . . . 59
192. Shading . . . 59
203. The directional light . . . 62
207. Glossy reflections . . . 63

The application **66**
215. The main program . . . 66
216. The application class . . . 66

225. The implementation 68
228. Creation and destruction 70
235. Opening files 72
238. The main loop 73
240. The layout 74
244. Menus 75
248. The hierarchy 77
255. The inspector 79
266. The camera 83
269. The viewport 85
280. The status bar and the popups 88
282. Decoding images 89

Installation for all users **91**

Names of the sections **94**

Introduction

1. UsdOptixViewer is a small, complete viewer for scenes described in Pixar's Universal Scene Description (USD). It opens a stage, shows its prims as a tree, lets the reader inspect every attribute, and draws the geometry with a ray tracer that runs on NVIDIA's OptiX engine. The program was written to exercise, in one place, a set of tools installed on a workstation: CUDA and OptiX for ray tracing, SDL3 for the window and input, Dear ImGui for the user interface, EnTT for the scene graph, and OpenUSD for reading scenes.

This document *is* the program. Following Donald Knuth's *Literate Programming* (1984), the code is presented in the order in which it is best understood by a human reader, not in the order a compiler wants it. Each piece of code lives in a numbered *section*, together with the prose that explains it. A tool, `lp.py`, *tangles* the sections back into the thirteen source files of the program, and *weaves* them into the book you are reading.

2. The rendering is progressive. While the reader does not touch anything, the viewer keeps adding one jittered sample per pixel and averages them, up to 256 samples. Materials follow the UsdPreviewSurface specification: diffuse colors and textures, roughness and metallic values or maps, tangent-space normal maps, alpha cutouts and per-face materials. Textures are filtered with mipmaps whose level is chosen by ray differentials, and glossy reflections are sampled from the GGX distribution of visible normals. All of these terms are explained where they appear.

3. How to read the code. A *chunk* of code has a name in angle brackets, followed by the number of the section where it is first defined, as in ⟨Install the viewer 15⟩; the ≡ sign introduces its definition, and +≡ marks a continuation of a chunk defined earlier. When a chunk name appears inside another chunk, the code it names is to be inserted at that point, indented like the reference. After each definition, a small note tells where the chunk is used. A chunk whose name is a file name, such as `UsdOptixViewer/src/main.cpp`, is a *root*: its expansion is written to that file.

Code is typeset in the manner of Knuth's CWEB: reserved words in **bold**, identifiers in *italic*, strings in `typewriter` type and comments in roman. Some operators are shown with their mathematical symbols: ≠ for `!=`, ≤ and ≥ for `<=` and `>=`, ≡ for `==`, ∧ and ∨ for `&&` and `||`, ¬ for `!`, and → for `->`. Long lines are broken and continued with a deeper indentation.

4. The parts of the program. The book follows the path of the data, from the build system to the pixels on the screen:

- *The build system* compiles the C++ code, turns the OptiX programs into PTX and embeds them in the executable.
- *Shared parameters* define the one structure that the CPU fills in and the GPU reads, `LaunchParams.h`.
- *The scene* reads a USD stage, mirrors its prims into an EnTT registry, resolves materials and textures, and flattens all geometry into arrays of world-space triangles.
- *The renderer* creates the OptiX pipeline, builds the acceleration structure, uploads the textures with their mipmaps, and launches the ray tracing.
- *The device programs* run on the GPU: ray generation, shading, shadows, reflections and alpha cutouts.
- *The application* ties everything together with SDL3 and Dear ImGui: the hierarchy, the inspector, the viewport and the camera.
- *Installation for all users* shows the script that moved the tools to a system-wide location.

The third-party image decoder `stb_image.h`, about eight thousand lines by Sean Barrett, is used as it comes and is not part of this book.

5. A word on correctness. Literate programs are easy to let drift away from the code that actually runs. To prevent that, the command `python3 lp.py check` tangles this book in memory and compares every root chunk, byte for byte, with the corresponding file in the repository; it also fails if any source file of the program

has no root chunk. The book and the program therefore always agree, and every line of the program appears somewhere in these pages.

The build system

6. The program is built with CMake. The project file has five parts, which we shall meet in turn.

⟨`UsdOptixViewer/CMakeLists.txt` 6⟩ ≡
⟨Declare the project and the language standard 7⟩
⟨Locate the visualization tools 8⟩
⟨Compile the device programs to PTX and embed them 13⟩
⟨Build Dear ImGui and the viewer executable 14⟩
⟨Install the viewer 15⟩
This code is written to file `UsdOptixViewer/CMakeLists.txt`.

7. We need CMake 3.24 or newer and C++20, the latter because EnTT 4 requires it. A build without an explicit type is an optimized *Release* build: ray tracing a scene in a debug build is no fun.

⟨Declare the project and the language standard 7⟩ ≡
```
cmake_minimum_required(VERSION 3.24)
project(UsdOptixViewer VERSION 0.1.0 LANGUAGES C CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
if(NOT CMAKE_BUILD_TYPE)
  set(CMAKE_BUILD_TYPE Release)
endif()
```
This code is used in section 6.

8. Finding the tools. The tools were first installed for one user under `~/.local/opt` and later for everybody under `/opt/viztools`, with CUDA in `/usr/local/cuda-13.2`. The project prefers the system-wide copy and falls back to the personal one; every location can also be overridden from the command line or the environment.

⟨Locate the visualization tools 8⟩ ≡
```
# ----------------------------------------------------------------------------
# Dependencies. Locations can be given through CMAKE_PREFIX_PATH or the
# variables / environment variables below (see /etc/profile.d/viztools.sh).
# Defaults to the all-users install in /opt/viztools, falling back to the
# per-user install in ~/.local/opt.
# ----------------------------------------------------------------------------
```
⟨Choose the root of the installed tools and of CUDA 9⟩
⟨Find the OptiX headers 10⟩
⟨Find the Dear ImGui sources 11⟩
⟨Find SDL3, EnTT, OpenUSD and CUDA 12⟩
This code is used in section 6.

9. The variable *VIZ_PREFIX* records the root of the tools. CUDA has its own rule: an explicit *CUDAToolkit_ROOT* wins, then the *CUDA_HOME* variable set by the login scripts, then the standard system directory.

⟨Choose the root of the installed tools and of CUDA 9⟩ ≡
```
if(EXISTS "/opt/viztools")
  set(_vizDefault "/opt/viztools")
else()
  set(_vizDefault "$ENV{HOME}/.local/opt")
endif()
set(VIZ_PREFIX "${_vizDefault}" CACHE PATH "Root of the installed visualization tools")
if(NOT DEFINED CUDAToolkit_ROOT)
  if(DEFINED ENV{CUDA_HOME})
    set(CUDAToolkit_ROOT "$ENV{CUDA_HOME}")
  elseif(EXISTS "/usr/local/cuda-13.2")
    set(CUDAToolkit_ROOT "/usr/local/cuda-13.2")
  else()
```

```
    set(CUDAToolkit_ROOT "${VIZ_PREFIX}/cuda-13.2")
  endif()
endif()
```

This code is used in section 8.

10. OptiX is a header-only SDK: the implementation ships inside the NVIDIA driver, and the headers only declare a table of function pointers that is filled in at run time. All we need, then, is the directory that contains `optix.h`.

⟨Find the OptiX headers 10⟩ ≡

```
set(OptiX_INSTALL_DIR "$ENV{OptiX_INSTALL_DIR}" CACHE PATH "OptiX SDK root (contains include/
      optix.h)")
if(NOT OptiX_INSTALL_DIR)
  set(OptiX_INSTALL_DIR "${VIZ_PREFIX}/optix-9.1.0" CACHE PATH "" FORCE)
endif()
find_path(OptiX_INCLUDE_DIR optix.h PATHS "${OptiX_INSTALL_DIR}/include" REQUIRED NO_DEFAULT_PATH)
```

This code is used in section 8.

11. Dear ImGui is distributed as source files to be compiled into each application, so we look for its directory rather than for a package.

⟨Find the Dear ImGui sources 11⟩ ≡

```
set(IMGUI_DIR "$ENV{IMGUI_DIR}" CACHE PATH "Dear ImGui source directory")
if(NOT IMGUI_DIR)
  set(IMGUI_DIR "${VIZ_PREFIX}/imgui-1.93.0" CACHE PATH "" FORCE)
endif()
if(NOT EXISTS "${IMGUI_DIR}/imgui.h")
  message(FATAL_ERROR "imgui.h not found in IMGUI_DIR=${IMGUI_DIR}")
endif()
```

This code is used in section 8.

12. The remaining dependencies provide CMake package files. OpenUSD's package is called `pxr`, after Pixar.

⟨Find SDL3, EnTT, OpenUSD and CUDA 12⟩ ≡

```
list(APPEND CMAKE_PREFIX_PATH "${VIZ_PREFIX}/sdl3" "${VIZ_PREFIX}/entt-4.0.0" "${VIZ_PREFIX}/
      usd-26.11")
find_package(CUDAToolkit 12 REQUIRED)
find_package(SDL3 REQUIRED CONFIG)
find_package(EnTT REQUIRED CONFIG)
find_package(pxr REQUIRED CONFIG)
```

This code is used in section 8.

13. Device code. The programs that run on the GPU (described in the part on device programs) are written in CUDA C++, but OptiX does not take machine code: it takes PTX, NVIDIA's portable assembly language, and compiles it for the actual GPU when the pipeline is created. We ask `nvcc` for PTX targeting the virtual architecture `compute_75`, the oldest one supported by CUDA 13, so that the same executable runs on any recent GPU. The warning 20044 that we silence is `nvcc`'s remark that an **extern** declaration of the launch parameters is treated as a definition, which is exactly what OptiX expects.

The PTX text is then turned into a C++ source file by a small CMake script, ⟨`UsdOptixViewer/cmake/EmbedFile.cmake` 16⟩, and compiled into the executable. The viewer therefore needs no data files at run time.

⟨Compile the device programs to PTX and embed them 13⟩ ≡

```
# ------------------------------------------------------------------------
# OptiX device code: compiled to PTX with nvcc and embedded into the binary.
# ------------------------------------------------------------------------
set(OPTIX_ARCH "compute_75" CACHE STRING "Virtual architecture used for the OptiX PTX")
set(PTX_FILE "${CMAKE_CURRENT_BINARY_DIR}/devicePrograms.ptx")
set(PTX_CPP  "${CMAKE_CURRENT_BINARY_DIR}/devicePrograms_ptx.cpp")
add_custom_command(
  OUTPUT "${PTX_FILE}"
  COMMAND "${CUDAToolkit_NVCC_EXECUTABLE}" -ptx -arch=${OPTIX_ARCH} -std=c++17 --use_fast_math
```

```
        -diag-suppress 20044
        -ccbin "${CMAKE_CXX_COMPILER}"
        -I "${OptiX_INCLUDE_DIR}" -I "${CMAKE_CURRENT_SOURCE_DIR}/src"
        "${CMAKE_CURRENT_SOURCE_DIR}/src/devicePrograms.cu" -o "${PTX_FILE}"
    DEPENDS src/devicePrograms.cu src/LaunchParams.h
    COMMENT "Compiling OptiX programs to PTX")
  add_custom_command(
    OUTPUT "${PTX_CPP}"
    COMMAND "${CMAKE_COMMAND}" -DINPUT=${PTX_FILE} -DOUTPUT=${PTX_CPP} -DSYMBOL=g_devicePrograms
        -P "${CMAKE_CURRENT_SOURCE_DIR}/cmake/EmbedFile.cmake"
    DEPENDS "${PTX_FILE}" cmake/EmbedFile.cmake
    COMMENT "Embedding PTX")
```

This code is used in section 6.

14. The executable. Dear ImGui and its two back ends (SDL3 for input, SDL_Renderer for drawing) become a static library. The viewer itself consists of five C++ files plus the generated PTX source. It links the parts of OpenUSD it uses: `usd` for stages, `usdGeom` for geometry, `usdShade` for materials, `ar` for resolving and opening assets, and the base libraries `sdf`, `tf`, `gf` and `vt`.

⟨Build Dear ImGui and the viewer executable 14⟩ ≡

```
# ------------------------------------------------------------------------
# Application
# ------------------------------------------------------------------------
add_library(imgui STATIC
  ${IMGUI_DIR}/imgui.cpp ${IMGUI_DIR}/imgui_draw.cpp ${IMGUI_DIR}/imgui_tables.cpp
  ${IMGUI_DIR}/imgui_widgets.cpp ${IMGUI_DIR}/imgui_demo.cpp
  ${IMGUI_DIR}/backends/imgui_impl_sdl3.cpp ${IMGUI_DIR}/backends/imgui_impl_sdlrenderer3.cpp)
target_include_directories(imgui PUBLIC ${IMGUI_DIR} ${IMGUI_DIR}/backends)
target_link_libraries(imgui PUBLIC SDL3::SDL3)

add_executable(UsdOptixViewer
  src/main.cpp src/App.cpp src/Scene.cpp src/OptixRenderer.cpp src/StbImage.cpp "${PTX_CPP}")
target_include_directories(UsdOptixViewer PRIVATE src ${OptiX_INCLUDE_DIR})
target_link_libraries(UsdOptixViewer PRIVATE
  imgui SDL3::SDL3 EnTT::EnTT usd usdGeom usdShade ar sdf tf gf vt
  CUDA::cudart CUDA::cuda_driver ${CMAKE_DL_LIBS})
# USD headers trigger a few deprecation warnings under C++20
target_compile_options(UsdOptixViewer PRIVATE -Wno-deprecated -Wno-deprecated-declarations)
```

This code is used in section 6.

15. Installation copies the executable and the test scenes. The libraries it depends on stay where they are: the run-time search path (RPATH) of the installed binary points to them, so the program starts without any `LD_LIBRARY_PATH` settings.

⟨Install the viewer 15⟩ ≡

```
# ------------------------------------------------------------------------
# Installation: bin/UsdOptixViewer + sample scenes. RPATH points to the
# dependency libraries so the installed binary runs without LD_LIBRARY_PATH.
# ------------------------------------------------------------------------
include(GNUInstallDirs)
set_target_properties(UsdOptixViewer PROPERTIES
  INSTALL_RPATH_USE_LINK_PATH ON
  INSTALL_RPATH "\$ORIGIN/../${CMAKE_INSTALL_LIBDIR}")
install(TARGETS UsdOptixViewer RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR})
install(DIRECTORY assets/ DESTINATION ${CMAKE_INSTALL_DATADIR}/UsdOptixViewer/assets)
```

This code is used in section 6.

16. Embedding a file. CMake has no command to turn a file into a C++ array, but its scripting language is enough. We read the PTX as a hexadecimal string, turn every pair of hex digits into a byte literal, and write a source file that defines two symbols: the bytes followed by a terminating zero, and the size without it. The renderer declares both as **extern** and hands them to OptiX.

⟨`UsdOptixViewer/cmake/EmbedFile.cmake` 16⟩ ≡

```
# Usage: cmake -DINPUT=file -DOUTPUT=file.cpp -DSYMBOL=name -P EmbedFile.cmake
# Writes a C++ source defining `extern const char SYMBOL[]` (NUL terminated)
# and `extern const size_t SYMBOL_size` (without the terminator).
file(READ "${INPUT}" content HEX)
string(LENGTH "${content}" hexLength)
math(EXPR size "${hexLength} / 2")
string(REGEX REPLACE "([0-9a-f][0-9a-f])" "0x\\1," bytes "${content}")
file(WRITE "${OUTPUT}"
  "#include <cstddef>\n"
  "extern const char ${SYMBOL}[] = {${bytes}0x00};\n"
  "extern const size_t ${SYMBOL}_size = ${size};\n")
```

This code is written to file `UsdOptixViewer/cmake/EmbedFile.cmake`.

Shared parameters

17. The CPU and the GPU must agree on one thing above all: the layout of the data that the host hands to every OptiX launch. That data is described once, in `LaunchParams.h`, and the header is compiled twice: by the host C++ compiler for `OptixRenderer.cpp`, which fills the structures in, and by `nvcc` for `devicePrograms.cu`, which reads them. Because both sides see the very same declarations, a field added here cannot be forgotten on one side; it can only be left unset.

⟨`UsdOptixViewer/src/LaunchParams.h` 17⟩ ≡
⟨Opening of the shared parameters header 18⟩
⟨The background mesh id and the ray types 19⟩
⟨The camera frame 20⟩
⟨Scalar material inputs 21⟩
⟨Material data on the device 22⟩
⟨The launch parameters 25⟩
This code is written to file `UsdOptixViewer/src/LaunchParams.h`.

18. The header includes only CUDA and OptiX type definitions, never their function declarations. From OptiX we take *OptixTraversableHandle*, from CUDA's texture types *cudaTextureObject_t*, and from CUDA's vector types *float3*, *float4*, *uchar4* and friends. These headers declare the same sizes and the same alignments for the host compiler and for `nvcc`, which is what makes a byte-for-byte copy from one side to the other legitimate.

⟨Opening of the shared parameters header 18⟩ ≡
```
// Data shared between the host (OptixRenderer.cpp) and the OptiX programs
// (devicePrograms.cu).
#pragma once

#include <optix_types.h>
#include <texture_types.h>
#include <vector_types.h>
```
This code is used in section 17.

19. When a camera ray hits nothing, the pixel belongs to no mesh. We mark such pixels with the largest unsigned integer, *kNoMesh*, which can never be a valid mesh index. The application reads these ids back to implement picking: a click on the background clears the selection.

The renderer traces two kinds of rays. *Radiance* rays ask "what colour arrives along this ray?"; *shadow* rays only ask "is the light visible from here?". OptiX selects the program that handles a miss, and the record that handles a hit, by an index into the shader binding table; the enumeration gives these indices names, and *RAY_TYPE_COUNT* is the stride between the records of consecutive geometries.

⟨The background mesh id and the ray types 19⟩ ≡
```
constexpr unsigned int kNoMesh = 0xffffffffu;

enum RayType { RAY_TYPE_RADIANCE = 0, RAY_TYPE_SHADOW = 1, RAY_TYPE_COUNT };
```
This code is used in section 17.

20. The camera is a pinhole. Rather than storing a field of view and an aspect ratio, the host precomputes three vectors: w points from the eye to the centre of the image plane, while u and v point from that centre to the middle of the right and top edges of the image. A point of the image with normalized device coordinates $(x, y) \in [-1, 1]^2$ then lies in the direction $w + x\,u + y\,v$, and the ray generation program needs no trigonometry at all. If ϕ is the vertical field of view and a the aspect ratio, the host sets $|w| = 1$, $|v| = \tan(\phi/2)$ and $|u| = a\tan(\phi/2)$.

⟨The camera frame 20⟩ ≡
```
struct Camera
{
    float3 eye;
    float3 u, v, w;   // right, up and forward scaled to the image plane
};
```

This code is used in section 17.

21. Many inputs of a UsdPreviewSurface material, such as roughness or metallic, are single numbers that may either be constant or come from one channel of a texture. A *ScalarParam* holds both possibilities. When *texture* is negative the value is *value*; otherwise it is channel *channel* (0 to 3 for red, green, blue and alpha) of the texture, mapped through the *scale* and *bias* that the UsdUVTexture node specifies for that channel. Keeping the channel number lets one texture carry several inputs, as in the common "packed" maps where green is roughness and blue is metallic.

⟨Scalar material inputs 21⟩ ≡
```
// A float input: `value`, or texture[channel] * scale + bias when texture >= 0.
struct ScalarParam
{
    float value;
    int   texture;
    int   channel;
    float scale, bias;
};
```
This code is used in section 17.

22. Materials. The scene part of the program reduces every UsdPreviewSurface it meets to one *MaterialData*, and each triangle refers to one of them by index. The comment summarizes how the closest-hit program interprets the fields; we shall see each formula again when it is used.

The order of the fields is not accidental. A *float4* must be aligned on a 16-byte boundary, while a *float3* needs only 4 bytes. Putting the six *float4* members first, and pairing each *float3* with a following **int**, fills every 16-byte line exactly, so the compiler inserts no padding and the structure has the same size on both sides of the bus. Texture references are indices into the array of CUDA texture objects of ⟨The scene as seen by the device 28⟩, with -1 meaning "no texture".

⟨Material data on the device 22⟩ ≡
```
// UsdPreviewSurface subset.
// diffuse = texture(st) * texScale + texBias, or `color` when untextured
// cutout = hits where opacityTexture(st)[opacityChannel] < opacityThreshold are ignored
// normal = normalTexture(st) * normalScale + normalBias (tangent space)
// specular = GGX; F0 from metallic + ior, or specColor in the specular workflow
struct MaterialData
{
    ⟨Texture remapping coefficients of a material 23⟩
    ⟨Colors, textures and inputs of a material 24⟩
};
```
This code is used in section 17.

23. The six vectors are the *scale* and *bias* inputs of the UsdUVTexture nodes that feed the diffuse color, the normal and the specular color. They are applied componentwise to every texel, $c = s \cdot t + b$. For a normal map the specification asks for $s = (2, 2, 2)$ and $b = (-1, -1, -1)$, which maps the stored range $[0, 1]$ to the vector range $[-1, 1]$; files that use the DirectX convention flip the green channel through these same coefficients.

⟨Texture remapping coefficients of a material 23⟩ ≡
```
float4 texScale;
float4 texBias;
float4 normalScale;
float4 normalBias;
float4 specScale;
float4 specBias;
```
This code is used in section 22.

24. Then come the remaining inputs. *color* is the diffuse color of an untextured material. *specColor* and *specTexture* are used only in the *specular workflow*, where *specularWorkflow* is nonzero and the reflectance at normal incidence is given directly; otherwise the *metallic workflow* derives it from *metallic* and the index of refraction *ior*. The opacity fields describe an alpha cutout: the *opacityChannel* of *opacityTexture* is compared with *opacityThreshold*.

⟨Colors, textures and inputs of a material 24⟩ ≡
```
float3 color;
int    texture;  // index into LaunchParams::textures, -1 for none
float3 specColor;
int    specTexture;
ScalarParam roughness;
ScalarParam metallic;
int    opacityTexture;
int    opacityChannel;
float  opacityThreshold;
int    normalTexture;
int    specularWorkflow;
float  ior;
```
This code is used in section 22.

25. The launch parameters. Everything a launch needs is gathered in a single structure. The host copies it to a small device buffer before each launch, and OptiX makes it visible to all programs as the *__constant__* variable *params*. Constant memory is cached and broadcast to all threads of a warp, which suits data that every ray reads and no ray writes; the output buffers it points to, of course, live in ordinary global memory.

⟨The launch parameters 25⟩ ≡
```
struct LaunchParams
{
    ⟨Output buffers of a launch 26⟩
    ⟨Camera, light and rendering switches 27⟩
    ⟨The scene as seen by the device 28⟩
};
```
This code is used in section 17.

26. The output of a launch. Rendering is progressive: each launch traces one new sample per pixel. *accumBuffer* keeps, for every pixel, the running sum of the samples in its first three components and their number in the fourth, and *colorBuffer* receives their average, encoded for display. *frameIndex* counts the samples already taken; the host sets it back to zero whenever the camera, the scene or any option changes, and the next launch then overwrites the sums instead of adding to them. *meshIdBuffer* records which mesh each pixel shows; it is written only on frame 0, whose rays go through the pixel centres.

⟨Output buffers of a launch 26⟩ ≡
```
// output: colorBuffer = accumBuffer / (frameIndex + 1); meshIdBuffer is written on frame 0
uchar4*       colorBuffer;
float4*       accumBuffer;
unsigned int* meshIdBuffer;  // mesh under each pixel (kNoMesh for background)
unsigned int  width, height;
unsigned int  frameIndex;    // progressive sample index, reset to 0 after any change
```
This code is used in section 25.

27. Next come the camera, the single directional light, and the switches of the *View* menu, each an integer used as a boolean. Turning a switch off does not change the scene, only how it is drawn, which is handy for comparing the effect of a feature. *selectedMesh* is the mesh highlighted in orange, or *kNoMesh*.

⟨Camera, light and rendering switches 27⟩ ≡
```
Camera camera;
float3 lightDir;  // direction towards the light, normalized
unsigned int shadows;
unsigned int mipmaps;       // 0: always sample the finest texture level
unsigned int normalMaps;    // 0: ignore normal maps
unsigned int reflections;   // 0: reflect the sky only, without tracing rays
unsigned int selectedMesh;
```
This code is used in section 25.

28. The scene. All geometry is flattened into one list of world-space triangles and stored in a single geometry acceleration structure (GAS), whose handle is what *optixTrace* needs. Alongside it, plain arrays indexed by

the primitive number that OptiX reports for a hit give everything the shading code wants to know about a triangle: its vertices, its mesh, its material, the texture coordinates and shading normals of its three corners. Storing UVs and normals per corner rather than per vertex wastes some memory, but it handles face-varying data, where one vertex has different values on different faces, without any special case. A zero normal at the first corner means "no shading normals, use the flat geometric one".

⟨The scene as seen by the device 28⟩ ≡

```
// scene: one GAS with all triangles in world space
OptixTraversableHandle handle;
const float3*        vertices;
const uint3*         indices;
const unsigned int*  triMesh;        // mesh id of each triangle
const unsigned int*  triMaterial;    // material of each triangle
const float2*        triUVs;         // st of the 3 corners of each triangle
const float3*        triNormals;     // shading normal of the 3 corners (zero: flat)
const MaterialData*  materials;
const cudaTextureObject_t* textures;
```

This code is used in section 25.

The scene

29. Everything the viewer knows about a USD file passes through one object of class *Scene*. It has three jobs. First, it opens the stage and mirrors its prims into an EnTT *registry*, so that the user interface can walk the hierarchy and attach information to any prim. Second, it resolves the materials of the scene: it follows UsdShade connections to the inputs of the UsdPreviewSurface shader, and loads the textures they name. Third, it *flattens* every visible piece of geometry into a few large arrays of world-space triangles, which is the only form of geometry the renderer understands.

The declarations live in `Scene.h` and the code in `Scene.cpp`. The header is short enough to be read as a table of contents for this part.

⟨`UsdOptixViewer/src/Scene.h` 29⟩ ≡
⟨Headers needed by the scene declarations 30⟩
⟨The PrimInfo component 31⟩
⟨The Hierarchy component 32⟩
⟨The MeshGeometry component 33⟩
⟨Texture wrap modes and decoded textures 62⟩
⟨The ScalarInput material parameter 63⟩
⟨The Material record 64⟩
⟨The flattened world-space geometry 45⟩
⟨The Scene class 34⟩
This code is written to file `UsdOptixViewer/src/Scene.h`.

30. The header needs the standard containers, EnTT, and two USD headers: the stage itself and the UsdShade material class, which appears in the signature of a private member function.

⟨Headers needed by the scene declarations 30⟩ ≡
```
// A USD stage mirrored into an EnTT registry: one entity per prim, with
// components describing its place in the hierarchy and its geometry.
#pragma once

#include <array>
#include <map>
#include <string>
#include <tuple>
#include <vector>

#include <entt/entt.hpp>
#include <pxr/usd/usd/stage.h>
#include <pxr/usd/usdShade/material.h>
```
This code is used in section 29.

31. Entities and components. EnTT is an *entity–component–system* library. An entity is nothing but an integer handle; data are attached to it as *components*, plain structures stored in dense arrays, one array per component type. We create one entity for every prim of the stage, including the invisible pseudo-root "/". This costs little and gives every prim an identity that the user interface can store (the current selection is an entity) and that other parts of the program can decorate: only the prims that produce triangles receive a *MeshGeometry* component, so "is this prim drawn?" is simply "does this entity have that component?".

Every entity carries a *PrimInfo*, which names the prim it stands for. The path is the key back into the USD stage; the name and the type name are copied because the hierarchy panel shows them for every visible row, every frame.

⟨The PrimInfo component 31⟩ ≡
```
// ---- components ----------------------------------------------------------
struct PrimInfo
{
    pxr::SdfPath path;
    std::string  name;
```

```
    std::string   typeName;
};
```

This code is used in section 29.

32. The tree structure is stored explicitly. USD can of course enumerate the children of a prim, but doing so through the stage on every frame of the user interface would be wasteful, and the list would have to be filtered the same way every time. A *Hierarchy* keeps the parent and the children in USD's order. The root's parent is *entt::null*.

⟨The Hierarchy component 32⟩ ≡

```
struct Hierarchy
{
    entt::entity parent = entt::null;
    std::vector<entt::entity> children;
};
```

This code is used in section 29.

33. A prim that contributed triangles remembers where they went. Its triangles occupy a contiguous range of the flattened arrays, from *firstTriangle* on; *meshId* is the small integer that the renderer writes into its picking buffer, so that a click on the image can be mapped back to the entity. The remaining fields are for the inspector: the index of the material bound to the prim, how many GeomSubsets override it, and whether texture coordinates were found.

⟨The MeshGeometry component 33⟩ ≡

```
// Present on prims that contributed triangles to the rendered geometry.
struct MeshGeometry
{
    unsigned int meshId;
    unsigned int firstTriangle;
    unsigned int triangleCount;
    unsigned int material;          // material bound to the prim itself
    unsigned int subsetMaterials;   // number of GeomSubsets with their own material
    bool hasUVs;
};
```

This code is used in section 29.

34. The *Scene* class is small from the outside. A scene is either empty or holds one open stage; *load* replaces whatever was there. The accessors give the user interface the stage, the registry, the root entity, the flattened geometry and the list of warnings collected while reading the file. The up axis matters to the camera, which must know whether "up" is $+y$ or $+z$.

⟨The Scene class 34⟩ ≡

```
class Scene
{
public:
    ⟨Public interface of the scene 35⟩

private:
    ⟨Private helpers of the scene 37⟩

    ⟨Data members of the scene 36⟩
};
```

This code is used in section 29.

35.

⟨Public interface of the scene 35⟩ ≡

```
bool load(const std::string& path, std::string& error);
void clear();

bool loaded() const { return static_cast<bool>(m_stage); }
const std::string& filePath() const { return m_filePath; }
const pxr::UsdStageRefPtr& stage() const { return m_stage; }
entt::registry& registry() { return m_registry; }
entt::entity root() const { return m_root; }
```

```
const FlatGeometry& geometry() const { return m_geometry; }
const std::vector<std::string>& warnings() const { return m_warnings; }
bool zUp() const { return m_zUp; }

pxr::UsdPrim prim(entt::entity e) const;
```

This code is used in section 34.

36. Besides the registry and the geometry, the scene keeps the time at which the stage is sampled, and two caches that we shall explain with the materials: a material is converted only once however many prims bind it, and a texture is decoded only once however many materials read it.

⟨Data members of the scene 36⟩ ≡

```
std::string             m_filePath;
pxr::UsdStageRefPtr     m_stage;
pxr::UsdTimeCode        m_time;
entt::registry          m_registry;
entt::entity            m_root = entt::null;
FlatGeometry            m_geometry;
std::vector<std::string> m_warnings;
std::map<pxr::SdfPath, unsigned int> m_materialCache;
std::map<std::tuple<std::string, bool, int, int>, int> m_textureCache;
bool                    m_zUp = false;
```

This code is used in section 34.

37. The private member functions do the real work, and each has its own sections below.

⟨Private helpers of the scene 37⟩ ≡

```
entt::entity createEntities(const pxr::UsdPrim& prim, entt::entity parent);
void extractGeometry();
unsigned int materialFor(const pxr::UsdShadeMaterial& material);
unsigned int displayColorMaterial(const pxr::UsdPrim& prim);
// srgbByDefault: color space when sourceColorSpace is neither "raw" nor "sRGB"
int loadTexture(const pxr::UsdShadeShader& shader, bool srgbByDefault = true);
ScalarInput scalarInput(const pxr::UsdShadeShader& surface, const char* name, float fallback);
void warn(const std::string& message);
```

This code is used in section 34.

38. The implementation file. `Scene.cpp` begins with a number of helpers that know nothing about the *Scene* class: they turn USD geometry into triangles and read values out of shading networks. They live in an anonymous namespace. The member functions follow.

⟨`UsdOptixViewer/src/Scene.cpp` 38⟩ ≡

```
⟨Headers needed by the scene implementation 39⟩

namespace {

⟨Local triangle soup of one prim 50⟩

⟨Convert a UsdGeomMesh into a local triangle soup 51⟩

⟨Close the bookkeeping of a generated primitive 59⟩

⟨Generate the triangles of a cube 60⟩

⟨Generate the triangles of a sphere 61⟩

⟨Read the value of a shader input 68⟩

⟨Find the shader that drives an input 69⟩

⟨Identify a shader and its wrap modes 70⟩

}  // namespace

⟨Open a stage and build the scene 40⟩

⟨Forget the current stage 41⟩

⟨Collect a warning once 42⟩

⟨Find the prim of an entity 43⟩

⟨Mirror the prim hierarchy into the registry 44⟩
```

⟨Material for a prim without a bound material 71⟩

⟨Convert a UsdShade material into a Material record 72⟩

⟨Read a float input that may come from a texture 85⟩

⟨Load a texture once 86⟩

⟨Flatten the stage into world-space triangles 46⟩

This code is written to file `UsdOptixViewer/src/Scene.cpp`.

39. Besides the USD schemas for geometry and shading, we need the asset resolver (to read textures, even from inside `.usdz` packages) and the image decoder. *PXR_NAMESPACE_USING_DIRECTIVE* lets us write *UsdPrim* instead of *pxr*::*UsdPrim* in this file.

⟨Headers needed by the scene implementation 39⟩ ≡

```
#include "Scene.h"

#include <algorithm>
#include <cmath>

#include <pxr/base/gf/matrix4d.h>
#include <pxr/usd/ar/asset.h>
#include <pxr/usd/ar/resolver.h>
#include <pxr/usd/usdGeom/cube.h>
#include <pxr/usd/usdGeom/mesh.h>
#include <pxr/usd/usdGeom/metrics.h>
#include <pxr/usd/usdGeom/primvarsAPI.h>
#include <pxr/usd/usdGeom/sphere.h>
#include <pxr/usd/usdGeom/subset.h>
#include <pxr/usd/usdGeom/tokens.h>
#include <pxr/usd/usdGeom/xformCache.h>
#include <pxr/usd/usdShade/materialBindingAPI.h>
#include <pxr/usd/usdShade/utils.h>

#include "third_party/stb_image.h"

PXR_NAMESPACE_USING_DIRECTIVE
```

This code is used in section 38.

40. Opening a stage. Loading starts from a clean slate. *UsdStage*::*Open* composes the file with everything it references; if that fails, the error message is returned to the caller, who shows it in a dialog.

Two global properties of the stage are recorded. The up axis is part of the stage metadata and defaults to y. The time is the start of the authored time range if there is one, and USD's *default* time otherwise: the viewer shows a single moment, and for an animated file the first frame is the natural choice. Then we build the entities and flatten the geometry.

⟨Open a stage and build the scene 40⟩ ≡

```
bool Scene::load(const std::string& path, std::string& error)
{
    clear();
    UsdStageRefPtr stage = UsdStage::Open(path);
    if (¬stage)
    {
        error = "Could not open USD stage: " + path;
        return false;
    }
    m_stage = stage;
    m_filePath = path;
    m_zUp = UsdGeomGetStageUpAxis(stage) ≡ UsdGeomTokens→z;
    m_time = stage→HasAuthoredTimeCodeRange() ? UsdTimeCode(stage→GetStartTimeCode()) :
            UsdTimeCode::Default();
    m_root = createEntities(stage→GetPseudoRoot(), entt::null);
    extractGeometry();
    return true;
}
```

This code is used in section 38.

41. Clearing destroys every entity and empties both caches. Releasing the stage pointer closes the stage, unless somebody else still holds it.

⟨Forget the current stage 41⟩ ≡
```
void Scene::clear()
{
    m_registry.clear();
    m_root = entt::null;
    m_geometry = FlatGeometry{};
    m_warnings.clear();
    m_materialCache.clear();
    m_textureCache.clear();
    m_stage = nullptr;
    m_filePath.clear();
}
```
This code is used in section 38.

42. Problems found while reading a file are not errors: an unsupported shader or a missing texture should not prevent us from showing the rest of the scene. They are collected as warnings, which the inspector lists. A large scene can produce the same complaint for hundreds of prims, so each distinct message is kept only once.

⟨Collect a warning once 42⟩ ≡
```
void Scene::warn(const std::string& message)
{
    if (std::find(m_warnings.begin(), m_warnings.end(), message) ≡ m_warnings.end())
        m_warnings.push_back(message);
}
```
This code is used in section 38.

43. Going from an entity back to its prim is a path lookup on the stage. An invalid entity (for instance *entt*::*null*, meaning "nothing selected") yields an invalid prim, which callers test with **if** (*prim*).

⟨Find the prim of an entity 43⟩ ≡
```
UsdPrim Scene::prim(entt::entity e) const
{
    if (¬m_stage ∨ ¬m_registry.valid(e))
        return {};
    return m_stage→GetPrimAtPath(m_registry.get<PrimInfo>(e).path);
}
```
This code is used in section 38.

44. The hierarchy is copied by a recursive walk. Each entity gets its *PrimInfo* before its children are visited, so the entities are created in depth-first preorder, and its *Hierarchy* after, when the children's handles are known.

The predicate *UsdTraverseInstanceProxies* deserves a word. USD shares the contents of *instanceable* prims: all instances of a model point to a single hidden *prototype*, and by default the children of an instance are not visible at all. With this predicate, the walk descends into every instance and yields *instance proxies*, read-only stand-ins for the prototype's prims at the instance's location. Each instance therefore gets its own entities, its own row in the tree and, later, its own copy of the triangles. That is wasteful for scenes with thousands of instances, but it keeps the renderer simple: there is just one bag of triangles.

⟨Mirror the prim hierarchy into the registry 44⟩ ≡
```
entt::entity Scene::createEntities(const UsdPrim& prim, entt::entity parent)
{
    const entt::entity e = m_registry.create();
    m_registry.emplace<PrimInfo>(e, prim.GetPath(), prim.IsPseudoRoot() ? std::string("/") :
            prim.GetName().GetString(),
                                    prim.GetTypeName().GetString());
    Hierarchy hierarchy{parent, {}};
    for (const UsdPrim& child : prim.GetFilteredChildren(UsdTraverseInstanceProxies()))
        hierarchy.children.push_back(createEntities(child, e));
    m_registry.emplace<Hierarchy>(e, std::move(hierarchy));
```

```
    return e;
}
```

This code is used in section 38.

45. The flattened geometry. The renderer builds a single acceleration structure over all the triangles of the scene, already placed in world space. *FlatGeometry* is that soup, together with everything a hit point needs in order to be shaded. Positions are shared between the triangles of one prim and addressed by *indices*; all the other arrays are *per triangle*, so that the GPU, which learns only the index of the triangle that was hit, can find its mesh, its material, the texture coordinates of its three corners and the shading normals of its three corners. A zero normal means "no shading normal; use the flat normal of the triangle". Materials and textures are kept in arrays and referred to by index. Finally, *meshEntity* maps a *meshId* back to its entity, and the bounds let the camera frame the whole scene.

⟨The flattened world-space geometry 45⟩ ≡

```
// Geometry flattened to world space, ready to upload to the renderer.
struct FlatGeometry
{
    std::vector<float>          vertices;     // xyz
    std::vector<unsigned int>   indices;      // 3 per triangle
    std::vector<unsigned int>   triMesh;      // mesh id per triangle
    std::vector<unsigned int>   triMaterial;  // material per triangle
    std::vector<float>          triUVs;       // 6 per triangle (st of each corner)
    std::vector<float>          triNormals;   // 9 per triangle (shading normal of each corner, 0 = flat)
    std::vector<Material>       materials;
    std::vector<Texture>        textures;
    std::vector<entt::entity>   meshEntity;   // entity of each mesh id
    float boundsMin[3] = {0, 0, 0};
    float boundsMax[3] = {0, 0, 0};

    bool empty() const { return indices.empty(); }
};
```

This code is used in section 29.

46. Flattening is one pass over the entities. For each prim that should be drawn we produce its triangles in local space, decide the material of each face, read its texture coordinates, transform it to world space and append it to the arrays. Each of these steps gets a section of its own.

⟨Flatten the stage into world-space triangles 46⟩ ≡

```
// ---- geometry ------------------------------------------------------------
void Scene::extractGeometry()
{
    ⟨Prepare the flattening of the stage 47⟩
    for (auto [e, info] : m_registry.view<PrimInfo>().each())
    {
        ⟨Skip prims that are not visible renderable geometry 48⟩
        ⟨Triangulate a prim in its local space 49⟩
        ⟨Assign a material to every face of a prim 90⟩
        ⟨Read the texture coordinates of a prim 91⟩
        ⟨Transform the points of a prim to world space 92⟩
        ⟨Append the triangles of a prim to the flat geometry 93⟩
        ⟨Warn about textured prims without texture coordinates 96⟩
    }
    ⟨Record the bounds of the scene 97⟩
}
```

This code is used in section 38.

47. A *UsdGeomXformCache* computes local-to-world matrices and remembers the matrices of ancestors, so that siblings deep in the tree do not recompute the same chain of transforms. The bounds start empty, at $+\infty$ and $-\infty$.

⟨Prepare the flattening of the stage 47⟩ ≡
```
UsdGeomXformCache xformCache(m_time);
FlatGeometry& g = m_geometry;
float lo[3] = {INFINITY, INFINITY, INFINITY}, hi[3] = {−INFINITY, −INFINITY, −INFINITY};
```
This code is used in section 46.

48. Only *gprims*, the geometric primitives of UsdGeom, can be drawn. Visibility is inherited, so we ask USD to *compute* it rather than read the prim's own attribute: a mesh under an invisible transform is invisible. The *purpose* works the same way; we draw what a final render would draw (purpose **default** or *render*) and skip the *proxy* and *guide* geometry that applications use for previews and manipulators.

⟨Skip prims that are not visible renderable geometry 48⟩ ≡
```
const UsdPrim p = m_stage→GetPrimAtPath(info.path);
if (¬p ∨ ¬p.IsA<UsdGeomGprim>())
    continue;
const UsdGeomImageable imageable(p);
const TfToken purpose = imageable.ComputePurpose();
if (imageable.ComputeVisibility(m_time) ≡ UsdGeomTokens→invisible ∨
    (purpose ≠ UsdGeomTokens→default_ ∧ purpose ≠ UsdGeomTokens→render))
    continue;
```
This code is used in section 46.

49. From prims to triangles. Three kinds of gprims are supported: polygonal meshes, and the implicit cube and sphere. Each is converted into a *LocalMesh*, a triangle soup in the prim's own coordinate system. The other gprims (cylinders, cones, capsules, curves, points) produce no triangles and are skipped. The default sizes are USD's: a cube of edge 2 and a sphere of radius 1.

⟨Triangulate a prim in its local space 49⟩ ≡
```
LocalMesh local;
if (p.IsA<UsdGeomMesh>())
    meshFromUsdMesh(UsdGeomMesh(p), m_time, local);
else if (p.IsA<UsdGeomCube>())
{
    double size = 2.0;
    UsdGeomCube(p).GetSizeAttr().Get(&size, m_time);
    meshFromCube(size, local);
}
else if (p.IsA<UsdGeomSphere>())
{
    double radius = 1.0;
    UsdGeomSphere(p).GetRadiusAttr().Get(&radius, m_time);
    meshFromSphere(radius, local);
}
if (local.indices.empty())
    continue;
```
This code is used in section 46.

50. A triangle soup must remember more than three point indices per triangle. USD attaches data to meshes at different rates, called *interpolations*: once per point (*vertex* or *varying*), once per face (*uniform*), once per corner of each face (*faceVarying*), or once for the whole mesh (*constant*). To fetch the right value for each corner of each triangle later, we keep, besides the point index, the *face-vertex* index of the corner (its position in USD's *faceVertexIndices* array) and the index of the face the triangle came from. The face index also serves GeomSubsets, which assign materials to sets of faces. The shading normals, when there are any, are resolved per corner right away.

⟨Local triangle soup of one prim 50⟩ ≡
```
// Local-space triangle soup produced for one prim.
struct LocalMesh
{
    std::vector<GfVec3f> points;
    std::vector<unsigned int> indices;      // 3 point indices per triangle
    std::vector<unsigned int> cornerFv;     // 3 face-vertex indices per triangle
```

```
    std::vector<unsigned int> faceOfTri;    // source face of each triangle
    std::vector<GfVec3f> cornerNormals;     // 3 shading normals per triangle, empty = flat
    size_t faceCount = 1;
};
```

This code is used in section 38.

51. A *UsdGeomMesh* describes polygons by three arrays: the points, the number of vertices of each face, and the concatenated vertex indices of all faces. Converting it takes four steps.

```
⟨Convert a UsdGeomMesh into a local triangle soup 51⟩ ≡
bool meshFromUsdMesh(const UsdGeomMesh& mesh, UsdTimeCode time, LocalMesh& out)
{
    ⟨Read the points and the face topology of a mesh 52⟩
    ⟨Fan-triangulate the faces of a mesh 53⟩
    ⟨Rewind the triangles of a left-handed mesh 54⟩
    ⟨Compute the shading normals of each triangle corner 55⟩
    return true;
}
```

This code is used in section 38.

52. A mesh lacking any of the three arrays has nothing to draw.

```
⟨Read the points and the face topology of a mesh 52⟩ ≡
VtArray<GfVec3f> points;
VtArray<int> counts, faceIndices;
if (¬mesh.GetPointsAttr().Get(&points, time) ∨ ¬mesh.GetFaceVertexCountsAttr().Get(&counts, time) ∨
    ¬mesh.GetFaceVertexIndicesAttr().Get(&faceIndices, time))
    return false;
```

This code is used in section 51.

53. A face with n vertices $v_0, \ldots, v_{n-1}$ becomes the $n-2$ triangles (v_0, v_k, v_{k+1}) for $1 \le k \le n-2$, a *fan* around the first vertex. This is exact for triangles and for convex polygons, which is what real assets contain almost exclusively; a concave polygon may come out wrong, and we accept that. The variable *offset* is the face-vertex index of the face's first corner, so *c0*, *c1* and *c2* are the face-vertex indices of the triangle's corners.

Files are not always well formed, and a bad index would make the GPU read outside its buffers. A negative or excessive vertex count ends the conversion; a triangle that names a nonexistent point is dropped.

```
⟨Fan-triangulate the faces of a mesh 53⟩ ≡
out.points.assign(points.begin(), points.end());
out.faceCount = counts.size();
size_t offset = 0;
for (size_t face = 0; face < counts.size(); ++face)
{
    const int count = counts[face];
    if (count < 0 ∨ offset + count > faceIndices.size())
        break;
    for (int k = 1; k + 1 < count; ++k)   // fan triangulation
    {
        const size_t c0 = offset, c1 = offset + k, c2 = offset + k + 1;
        const int i0 = faceIndices[c0], i1 = faceIndices[c1], i2 = faceIndices[c2];
        if (i0 < 0 ∨ i1 < 0 ∨ i2 < 0 ∨ size_t(std::max({i0, i1, i2})) ≥ points.size())
            continue;
        out.indices.insert(out.indices.end(), {unsigned(i0), unsigned(i1), unsigned(i2)});
        out.cornerFv.insert(out.cornerFv.end(), {unsigned(c0), unsigned(c1), unsigned(c2)});
        out.faceOfTri.push_back(unsigned(face));
    }
    offset += count;
}
```

This code is used in section 51.

54. USD meshes are right-handed by default: seen from the outside, the vertices of a face go counterclockwise, and the cross product of two edges, $(p_1 - p_0) \times (p_2 - p_0)$, points outward. A mesh may declare

itself *leftHanded*, with the opposite winding. The renderer relies on the cross product to know which side of a triangle faces out (it needs this for the normal-map frame and to orient shading normals), so we normalize the winding here, once: swapping two corners of each triangle reverses it. The face-vertex indices are swapped along, so that each corner keeps its own texture coordinates and normals.

⟨Rewind the triangles of a left-handed mesh 54⟩ ≡

```
// Make every triangle counter-clockwise when seen from outside, so the
// renderer can derive the outward normal (and the normal-map frame) from it.
TfToken orientation;
if (mesh.GetOrientationAttr().Get(&orientation) ∧ orientation ≡ UsdGeomTokens→leftHanded)
    for (size_t t = 0; t < out.indices.size(); t += 3)
    {
        std::swap(out.indices[t + 1], out.indices[t + 2]);
        std::swap(out.cornerFv[t + 1], out.cornerFv[t + 2]);
    }
```

This code is used in section 51.

55. Shading normals. A glossy highlight on a sphere made of flat facets shows every facet, so we interpolate *shading normals* across triangles when the mesh is meant to look smooth. The rules follow what Pixar's Hydra renderers do. Normals authored in the file win. Otherwise the subdivision scheme decides: USD's default scheme is Catmull–Clark, meaning "this cage describes a smooth surface", and we approximate that surface by smoothing the normals of the unrefined cage; a mesh with *subdivisionScheme* = `"none"` is a true polygonal mesh and is shaded flat. (The test scenes of this repository mark their boxes and walls *none* for exactly this reason: with the default scheme a cube shades like a rounded blob, which is correct USD behavior.)

⟨Compute the shading normals of each triangle corner 55⟩ ≡

```
// Shading normals: authored primvars:normals or normals, otherwise smooth
// vertex normals for subdivision surfaces (as Hydra does) and flat for
// polygonal meshes.
⟨Find the authored normals of a mesh 56⟩
⟨Compute smooth normals for a subdivision surface 57⟩
⟨Pick the shading normal of each triangle corner 58⟩
```

This code is used in section 51.

56. Normals may be authored in two places. The primvar `primvars:normals` takes precedence over the older *normals* attribute, as the UsdGeom documentation prescribes. *ComputeFlattened* expands an *indexed* primvar (values plus an index array) into one value per element, so we never deal with the indices ourselves.

⟨Find the authored normals of a mesh 56⟩ ≡

```
VtArray<GfVec3f> normals;
TfToken interpolation;
if (const UsdGeomPrimvar pv = UsdGeomPrimvarsAPI(mesh).GetPrimvar(UsdGeomTokens→normals);
    pv ∧ pv.ComputeFlattened(&normals, time))
    interpolation = pv.GetInterpolation();
else if (mesh.GetNormalsAttr().Get(&normals, time))
    interpolation = mesh.GetNormalsInterpolation();
```

This code is used in section 55.

57. Smooth normals are accumulated per point. The cross product of two edges of a triangle has length twice its area, so summing the unnormalized cross products weights each triangle by its area: large faces dominate, and a sliver triangle does not tilt the normal. Since the triangles were rewound in the previous step, all the cross products point outward. The sums are normalized later, after the transformation to world space.

⟨Compute smooth normals for a subdivision surface 57⟩ ≡

```
TfToken scheme;
mesh.GetSubdivisionSchemeAttr().Get(&scheme);
if (normals.empty() ∧ scheme ≠ UsdGeomTokens→none)
{
    normals.assign(points.size(), GfVec3f(0.0f));
    for (size_t t = 0; t < out.indices.size(); t += 3)
    {
```

```
            const unsigned i0 = out.indices[t], i1 = out.indices[t + 1], i2 = out.indices[t + 2];
            const GfVec3f n = GfCross(points[i1] − points[i0], points[i2] − points[i0]);  // area weighted
            normals[i0] += n;
            normals[i1] += n;
            normals[i2] += n;
        }
        interpolation = UsdGeomTokens→vertex;
    }
```

This code is used in section 55.

58. Finally each corner looks up its normal according to the interpolation: by face-vertex index, by point index, or by face. A *constant* normal falls through all the tests and uses element 0. An index beyond the array, which only a malformed file produces, gives the zero vector, which the renderer treats as "flat". When there are no normals at all, *cornerNormals* stays empty and the whole mesh is shaded flat.

```
⟨Pick the shading normal of each triangle corner 58⟩ ≡
    if (¬normals.empty())
        for (size_t t = 0; t < out.indices.size(); t += 3)
            for (size_t c = 0; c < 3; ++c)
            {
                size_t i = 0;
                if (interpolation ≡ UsdGeomTokens→faceVarying)
                    i = out.cornerFv[t + c];
                else if (interpolation ≡ UsdGeomTokens→vertex ∨ interpolation ≡ UsdGeomTokens→varying)
                    i = out.indices[t + c];
                else if (interpolation ≡ UsdGeomTokens→uniform)
                    i = out.faceOfTri[t / 3];
                out.cornerNormals.push_back(i < normals.size() ? normals[i] : GfVec3f(0.0f));
            }
```

This code is used in section 55.

59. Implicit primitives. The cube and the sphere have no faces in USD's sense, so after generating their triangles we fill in the bookkeeping: each corner's "face-vertex" index is simply its point index, and the whole primitive counts as a single face, number 0. (These primitives carry no texture coordinates, so the choice only has to be consistent.)

```
⟨Close the bookkeeping of a generated primitive 59⟩ ≡
    void finishPrimitive(LocalMesh& out)
    {
        out.cornerFv = out.indices;
        out.faceOfTri.assign(out.indices.size() / 3, 0u);
    }
```

This code is used in section 38.

60. The eight corners of a cube centered at the origin are numbered by three bits: bit 0 selects $\pm x$, bit 1 $\pm y$ and bit 2 $\pm z$. The twelve triangles, two per side, are listed so that each is counter-clockwise when seen from outside; for instance $(0, 2, 1)$ lies on the $-z$ side, and $(p_2 - p_0) \times (p_1 - p_0)$ is a positive multiple of $(0, 1, 0) \times (1, 0, 0) = (0, 0, -1)$, the outward direction. A cube has no shading normals: it is meant to be flat.

```
⟨Generate the triangles of a cube 60⟩ ≡
    void meshFromCube(double size, LocalMesh& out)
    {
        const float h = float(size) * 0.5f;
        for (int i = 0; i < 8; ++i)
            out.points.emplace_back(i & 1 ? h : −h, i & 2 ? h : −h, i & 4 ? h : −h);
        out.indices = {0, 2, 1, 1, 2, 3, 4, 5, 6, 5, 7, 6, 0, 1, 4, 1, 5, 4,
                       2, 6, 3, 3, 6, 7, 0, 4, 2, 2, 4, 6, 1, 3, 5, 3, 7, 5};
        finishPrimitive(out);
    }
```

This code is used in section 38.

61. The sphere is a latitude–longitude grid: 24 stacks from the north pole ($\phi = 0$, on the $+y$ axis) to the

south pole ($\phi = \pi$), and 48 slices around. A point is

$$r\,(\sin\phi\cos\theta,\ \cos\phi,\ \sin\phi\sin\theta).$$

Each grid cell, with corners a and $a+1$ on one stack and b and $b+1$ on the next, becomes two triangles. The order $(a, a+1, b)$ is the outward one: near the equator at $\theta = 0$, the edge from a to $a+1$ runs toward $+z$ and the edge from a to b runs toward $-y$, and $(0,0,1) \times (0,-1,0) = (1,0,0)$ points away from the center. (An earlier version had the opposite order, which went unnoticed while the shading was two-sided and flat; it became visible as soon as shading normals had to agree with the winding.)

On a sphere centered at the origin the normal at a point is the point itself, scaled; we store the points as normals and let the world-space step normalize them. Every corner thus gets an exact, smooth normal.

⟨Generate the triangles of a sphere 61⟩ ≡

```
void meshFromSphere(double radius, LocalMesh& out)
{
    const int slices = 48, stacks = 24;
    for (int j = 0; j ≤ stacks; ++j)
    {
        const double phi = M_PI * j / stacks;
        for (int i = 0; i ≤ slices; ++i)
        {
            const double theta = 2.0 * M_PI * i / slices;
            out.points.emplace_back(float(radius * std::sin(phi) * std::cos(theta)), float(radius * std::cos(phi)),
                                    float(radius * std::sin(phi) * std::sin(theta)));
        }
    }
    for (int j = 0; j < stacks; ++j)
        for (int i = 0; i < slices; ++i)
        {
            const unsigned a = j * (slices + 1) + i, b = a + slices + 1;
            out.indices.insert(out.indices.end(), {a, a + 1, b, a + 1, b + 1, b});   // counter-clockwise from outside
        }
    finishPrimitive(out);
    for (unsigned idx : out.indices)
        out.cornerNormals.push_back(out.points[idx]);   // normalized later
}
```

This code is used in section 38.

62. Materials. The viewer understands a subset of *UsdPreviewSurface*, the simple physically based material that every USD tool can read. A material in USD is a small graph: a *Material* prim whose surface output is connected to a *UsdPreviewSurface* shader, whose inputs are either values or connections to other shaders, typically *UsdUVTexture* nodes, which in turn read their texture coordinates from *UsdPrimvarReader_float2* nodes. We condense such a graph into a flat *Material* record that the GPU can read without following any pointers.

The first kind of record field is used for decoded images. A *Texture* holds eight-bit RGBA pixels, whether they should be converted from sRGB to linear values when sampled, and what happens outside the unit square of texture coordinates in each direction. USD's *UsdUVTexture* offers repeat, mirror, clamp and black, which map directly to CUDA's address modes.

⟨Texture wrap modes and decoded textures 62⟩ ≡

```
// ---- render data ------------------------------------------------------
enum class TextureWrap { Repeat, Mirror, Clamp, Black };

struct Texture
{
    std::string path;   // resolved path (may point inside a .usdz)
    int width = 0, height = 0;
    std::vector<unsigned char> rgba;
    bool srgb = true;
    TextureWrap wrapS = TextureWrap::Repeat, wrapT = TextureWrap::Repeat;
};
```

This code is used in section 29.

63. Several inputs of UsdPreviewSurface, such as roughness and metallic, are single numbers that may instead come from one channel of a texture. Packing several such maps into the channels of one image is common practice, so we record which channel to read, and that channel's scale and bias from the *UsdUVTexture* node, which maps the texel t to $t \cdot scale + bias$.

⟨The ScalarInput material parameter 63⟩ ≡
```
// A float material input: a constant, or one channel of a texture
// (texture[channel] * scale + bias).
struct ScalarInput
{
    float value = 0.0f;
    int texture = -1;
    int channel = 0;
    float scale = 1.0f, bias = 0.0f;
};
```
This code is used in section 29.

64. The *Material* record. Texture fields are indices into *FlatGeometry*::*textures*, with -1 for "none".

⟨The Material record 64⟩ ≡
```
// Simplified UsdPreviewSurface: diffuse color, optionally from a texture
// (texture * scale + bias), tangent-space normal maps, alpha cutouts
// (opacity + opacityThreshold) and GGX specular in the metallic or the
// specular workflow.
struct Material
{
    ⟨Diffuse, cutout and normal-map fields of a material 65⟩
    ⟨Specular fields of a material 66⟩
    ⟨Test whether a material reads any texture 67⟩
};
```
This code is used in section 29.

65. The name is for the inspector. The diffuse color defaults to a light gray, which is what a prim without any material or display color looks like. The texture coordinates are read from the primvar named by *uvSet*, almost always *st*. A cutout discards hits where one channel of a texture, usually alpha, falls below a threshold. The defaults of the normal-map remapping, $2t - 1$, turn the usual eight-bit encoding of a unit vector, with components in $[0, 1]$, back into components in $[-1, 1]$.

⟨Diffuse, cutout and normal-map fields of a material 65⟩ ≡
```
std::string name;   // material prim path, or "displayColor"
std::array<float, 3> color = {0.7f, 0.7f, 0.7f};
int texture = -1;
std::array<float, 4> scale = {1, 1, 1, 1};
std::array<float, 4> bias = {0, 0, 0, 0};
std::string uvSet = "st";   // primvar read by the texture
// alpha cutout: hits where opacityTexture[opacityChannel] < opacityThreshold are ignored
int opacityTexture = -1;
int opacityChannel = 3;
float opacityThreshold = 0.0f;
// tangent-space normal = normalTexture * normalScale + normalBias
int normalTexture = -1;
std::array<float, 4> normalScale = {2, 2, 2, 1};
std::array<float, 4> normalBias = {-1, -1, -1, 0};
```
This code is used in section 64.

66. UsdPreviewSurface has two ways of describing the specular reflection. In the default *metallic workflow*, a metallic factor blends between a dielectric, whose reflectance at normal incidence follows from the index of refraction, and a metal, which reflects with its diffuse color. In the *specular workflow* the reflectance at normal incidence is given directly as a color. The defaults are those of the specification: roughness 0.5, metallic 0, index of refraction 1.5.

⟨Specular fields of a material 66⟩ ≡
```
ScalarInput roughness{0.5f};
```

```
ScalarInput metallic{0.0f};
float ior = 1.5f;
// useSpecularWorkflow: specularColor replaces metallic as the reflectance at normal incidence
bool specularWorkflow = false;
std::array<float, 3> specularColor = {0, 0, 0};
int specularTexture = -1;
std::array<float, 4> specularScale = {1, 1, 1, 1};
std::array<float, 4> specularBias = {0, 0, 0, 0};
```

This code is used in section 64.

67. A material that reads any texture needs texture coordinates. This question is asked when choosing which primvar to read and when deciding whether to warn about missing coordinates.

⟨Test whether a material reads any texture 67⟩ ≡

```
bool usesTextures() const
{
    return texture ≥ 0 ∨ opacityTexture ≥ 0 ∨ normalTexture ≥ 0 ∨ roughness.texture ≥ 0 ∨
        metallic.texture ≥ 0 ∨ specularTexture ≥ 0;
}
```

This code is used in section 64.

68. Reading shading networks. UsdShade connections can be indirect: a shader input may be connected to an input of the enclosing *Material* (its *interface*), which may in turn be connected to something else, possibly through node graphs. *GetValueProducingAttributes* follows such chains to their ends and returns the attributes that actually produce the value: either outputs of shaders, or inputs that hold a value. We want a plain value, so we accept only the latter. This is how a texture whose file name is published on the material's interface, as in our test scene, still finds its file.

The function is a template because inputs come in many types: colors, floats, integers, tokens, asset paths. When the input exists but holds a value of another type, *Get* fails and so do we, and the caller keeps its default.

⟨Read the value of a shader input 68⟩ ≡

```
// Value of a shader input, following connections to material interface
// inputs. Returns false when the input is unset or driven by another shader.
template <typename T>
bool inputValue(const UsdShadeShader& shader, const char* name, T* value)
{
    const UsdShadeInput input = shader.GetInput(TfToken(name));
    if (¬input)
        return false;
    for (const UsdAttribute& attr : input.GetValueProducingAttributes())
        if (UsdShadeUtils::GetType(attr.GetName()) ≡ UsdShadeAttributeType::Input ∧ attr.Get(value))
            return true;
    return false;
}
```

This code is used in section 38.

69. The other question to ask of an input is "which shader drives it?". With *shaderOutputsOnly* set, the same function returns only shader outputs; we take the first. Callers sometimes need the output itself, because its name (*outputs:rgb*, *outputs:r*, …) says which channels are used, and sometimes only the shader.

⟨Find the shader that drives an input 69⟩ ≡

```
// Shader output that drives the given input (through node graphs), if any.
UsdAttribute sourceOutput(const UsdShadeInput& input)
{
    if (input)
        for (const UsdAttribute& attr : input.GetValueProducingAttributes(/*shaderOutputsOnly=*/true))
            return attr;
    return UsdAttribute();
}

UsdShadeShader sourceShader(const UsdShadeInput& input)
{
    const UsdAttribute output = sourceOutput(input);
```

```
    return output ? UsdShadeShader(output.GetPrim()) : UsdShadeShader();
}
```

This code is used in section 38.

70. A shader's kind is its *info:id*, such as *UsdUVTexture*. The wrap mode *useMetadata*, USD's default, asks to use whatever the image file says; the formats we decode carry no such metadata, so it is treated as *repeat*, like anything unrecognized.

⟨Identify a shader and its wrap modes 70⟩ ≡

```
TfToken shaderId(const UsdShadeShader& shader)
{
    TfToken id;
    shader.GetShaderId(&id);
    return id;
}

TextureWrap wrapMode(const UsdShadeShader& shader, const char* name)
{
    TfToken mode;
    inputValue(shader, name, &mode);
    if (mode ≡ "mirror")
        return TextureWrap::Mirror;
    if (mode ≡ "clamp")
        return TextureWrap::Clamp;
    if (mode ≡ "black")
        return TextureWrap::Black;
    return TextureWrap::Repeat;   // "repeat" and "useMetadata"
}
```

This code is used in section 38.

71. A prim without a bound material is drawn with its *display color*, a primvar that every gprim may carry, and with all other properties at their defaults. Display colors differ from prim to prim, so these materials are not cached; each such prim gets its own record. A display color may vary over the surface; we use its first value only.

⟨Material for a prim without a bound material 71⟩ ≡

```
// ---- materials and textures ------------------------------------------------
unsigned int Scene::displayColorMaterial(const UsdPrim& prim)
{
    Material m;
    m.name = "displayColor";
    VtArray<GfVec3f> colors;
    if (UsdGeomGprim(prim).GetDisplayColorPrimvar().ComputeFlattened(&colors, m_time) ∧ ¬colors.empty())
        m.color = {colors[0][0], colors[0][1], colors[0][2]};
    m_geometry.materials.push_back(m);
    return unsigned(m_geometry.materials.size() − 1);
}
```

This code is used in section 38.

72. Converting a UsdShade material proceeds input by input. *ComputeSurfaceSource* finds the shader connected to the material's surface output. A material without one keeps the defaults; one with a surface shader has its inputs read one group at a time.

⟨Convert a UsdShade material into a Material record 72⟩ ≡

```
unsigned int Scene::materialFor(const UsdShadeMaterial& material)
{
    ⟨Return the cached Material of a UsdShade material 73⟩
    ⟨Start a Material with the UsdPreviewSurface defaults 74⟩
    if (const UsdShadeShader surface = material.ComputeSurfaceSource())
    {
        ⟨Read the diffuse color of a surface 76⟩
        ⟨Read the diffuse texture of a surface 77⟩
        ⟨Read an alpha cutout of a surface 80⟩
```

⟨Read the specular parameters of a surface 81⟩

⟨Read the normal map of a surface 82⟩

}

⟨Store a new Material in the cache 75⟩

}

This code is used in section 38.

73. Many prims usually share a material, so the conversion is cached by the path of the material prim.

⟨Return the cached Material of a UsdShade material 73⟩ ≡

```
const SdfPath path = material.GetPath();
if (auto it = m_materialCache.find(path); it ≠ m_materialCache.end())
    return it→second;
```

This code is used in section 72.

74. A material that does bind but specifies nothing is a UsdPreviewSurface with all inputs at their defaults, and the default diffuse color of the specification is a dark gray, 0.18.

⟨Start a Material with the UsdPreviewSurface defaults 74⟩ ≡

```
Material m;
m.name = path.GetString();
m.color = {0.18f, 0.18f, 0.18f};   // UsdPreviewSurface default
```

This code is used in section 72.

75.

⟨Store a new Material in the cache 75⟩ ≡

```
m_geometry.materials.push_back(m);
const unsigned index = unsigned(m_geometry.materials.size() − 1);
m_materialCache[path] = index;
return index;
```

This code is used in section 72.

76. The diffuse color. A plain value, possibly published on the material's interface, is the easy case. But an input can carry both a value and a connection: many exporters write the color that a texture would average to next to the connection to the texture. *inputValue* does not return that value, because the connection wins, so we read the attribute's own value separately. It becomes the color used if the texture cannot be loaded. This matters in practice: before textures were supported, it was the reason a textured model looked plausible but plain.

⟨Read the diffuse color of a surface 76⟩ ≡

```
GfVec3f c;
if (inputValue(surface, "diffuseColor", &c))
    m.color = {c[0], c[1], c[2]};
else if (const UsdShadeInput in = surface.GetInput(TfToken("diffuseColor")); in ∧ in.GetAttr().Get(&c))
    m.color = {c[0], c[1], c[2]};   // authored default beside a connection
```

This code is used in section 72.

77. If *diffuseColor* is driven by a *UsdUVTexture*, the record points to the decoded image, and the texture's coordinates tell us which primvar to read. Any other kind of shader (a procedural, or a MaterialX node) is beyond us, and we say so.

⟨Read the diffuse texture of a surface 77⟩ ≡

```
const UsdShadeShader texture = sourceShader(surface.GetInput(TfToken("diffuseColor")));
if (texture ∧ shaderId(texture) ≡ "UsdUVTexture")
{
    ⟨Read the scale, bias and fallback of the diffuse texture 78⟩

    ⟨Find the primvar that holds the diffuse texture coordinates 79⟩
}
else if (texture)
    warn("Unsupported shader driving diffuseColor: " + shaderId(texture).GetString() + " (" + m.name +
        ")");
```

This code is used in section 72.

78. The texture node's own *fallback* value, meant for when the image is missing, takes precedence over the color found above. Diffuse textures are decoded as sRGB unless the node says otherwise, which is what nearly every color image is.

```
⟨Read the scale, bias and fallback of the diffuse texture 78⟩ ≡
    GfVec4f v;
    if (inputValue(texture, "scale", &v))
        m.scale = {v[0], v[1], v[2], v[3]};
    if (inputValue(texture, "bias", &v))
        m.bias = {v[0], v[1], v[2], v[3]};
    if (inputValue(texture, "fallback", &v))
        m.color = {v[0], v[1], v[2]};
    m.texture = loadTexture(texture);
```

This code is used in section 77.

79. The *st* input of the texture is normally connected to a primvar reader, whose *varname* names the primvar. The specification says *varname* is a string, but older files, and some current exporters, write a token; we accept both. A *UsdTransform2d* node between the reader and the texture would scale, rotate or offset the coordinates; we do not implement it, and the coordinates are then read, untransformed, from the default primvar *st*.

```
⟨Find the primvar that holds the diffuse texture coordinates 79⟩ ≡
    // texture coordinates come from a UsdPrimvarReader_float2
    const UsdShadeShader reader = sourceShader(texture.GetInput(TfToken("st")));
    VtValue varname;
    if (reader ∧ shaderId(reader) ≡ "UsdTransform2d")
        warn("UsdTransform2d texture transforms are ignored (" + m.name + ")");
    else if (reader ∧ inputValue(reader, "varname", &varname))
    {
        if (varname.IsHolding<TfToken>())
            m.uvSet = varname.UncheckedGet<TfToken>().GetString();
        else if (varname.IsHolding<std::string>())
            m.uvSet = varname.UncheckedGet<std::string>();
    }
```

This code is used in section 77.

80. Alpha cutouts. UsdPreviewSurface treats opacity in two ways. With an *opacityThreshold* greater than zero, a point is either fully present (opacity at or above the threshold) or fully absent: this is how leaves, fences and grass blades are cut out of simple quads. With a zero threshold, opacity means partial transparency, which a ray tracer could do but this one does not; such materials are drawn opaque, with a warning. Only an opacity that comes from a texture can cut anything out, so a constant opacity is ignored. The channel is named by the texture output the input is connected to: *outputs:a* for alpha, or one of the color channels.

```
⟨Read an alpha cutout of a surface 80⟩ ≡
    // alpha cutout from a texture channel (translucency is not supported)
    float threshold = 0.0f;
    inputValue(surface, "opacityThreshold", &threshold);
    const UsdAttribute opacity = sourceOutput(surface.GetInput(TfToken("opacity")));
    if (opacity ∧ shaderId(UsdShadeShader(opacity.GetPrim())) ≡ "UsdUVTexture")
    {
        if (threshold > 0.0f)
        {
            const std::string channel = UsdShadeUtils::GetBaseNameAndType(opacity.GetName()).first.GetString();
            m.opacityTexture = loadTexture(UsdShadeShader(opacity.GetPrim()));
            m.opacityChannel = channel ≡ "r" ? 0 : channel ≡ "g" ? 1 : channel ≡ "b" ? 2 : 3;
            m.opacityThreshold = threshold;
        }
        else
            warn("Translucent materials are rendered opaque (" + m.name + ")");
    }
```

This code is used in section 72.

81. The specular parameters. Roughness and metallic are read by ⟨Read a float input that may come from

a texture 85⟩, the index of refraction is a plain number, and *useSpecularWorkflow* is an integer flag. In the specular workflow the *specularColor* may be a constant or a color texture, with the texture's scale and bias.

```
⟨Read the specular parameters of a surface 81⟩ ≡
    // specular: metallic or specular workflow
    m.roughness = scalarInput(surface, "roughness", 0.5f);
    m.metallic = scalarInput(surface, "metallic", 0.0f);
    inputValue(surface, "ior", &m.ior);
    int useSpecularWorkflow = 0;
    inputValue(surface, "useSpecularWorkflow", &useSpecularWorkflow);
    m.specularWorkflow = useSpecularWorkflow ≠ 0;
    if (m.specularWorkflow)
    {
        GfVec3f c;
        if (inputValue(surface, "specularColor", &c))
            m.specularColor = {c[0], c[1], c[2]};
        const UsdShadeShader specular = sourceShader(surface.GetInput(TfToken("specularColor")));
        if (specular ∧ shaderId(specular) ≡ "UsdUVTexture")
        {
            m.specularTexture = loadTexture(specular);
            GfVec4f v;
            if (inputValue(specular, "scale", &v))
                m.specularScale = {v[0], v[1], v[2], v[3]};
            if (inputValue(specular, "bias", &v))
                m.specularBias = {v[0], v[1], v[2], v[3]};
        }
    }
```

This code is used in section 72.

82. Normal maps. A tangent-space normal map stores, in each texel, a direction relative to the surface: x along the direction of increasing s, y along increasing t, z out of the surface. The texel values are data, not colors, so the image is decoded *raw* unless the file insists on sRGB.

```
⟨Read the normal map of a surface 82⟩ ≡
    // tangent-space normal map
    const UsdShadeShader normal = sourceShader(surface.GetInput(TfToken("normal")));
    if (normal ∧ shaderId(normal) ≡ "UsdUVTexture")
    {
        m.normalTexture = loadTexture(normal, /*srgbByDefault=*/false);
        ⟨Decide how to decode the normal map 83⟩
        ⟨Find the primvar that holds the normal-map coordinates 84⟩
    }
    else if (normal)
        warn("Unsupported shader driving normal: " + shaderId(normal).GetString() + " (" + m.name + ")");
```

This code is used in section 72.

83. The specification asks files to say, through the texture's scale and bias, how texels become vectors: for eight-bit images, scale $(2, 2, 2, 1)$ and bias $(-1, -1, -1, 0)$. We follow what is authored exactly. This is not pedantry: normal maps made for DirectX have the y axis flipped, and well-made USD files compensate with scale $(2, -2, 2)$ and bias $(-1, 1, -1)$, as the McUsd sample asset does. If only one of the two is authored, the other keeps the *UsdUVTexture* default. If neither is authored, the file is not compliant (`usdchecker` complains about it), but such files are common, so rather than read the vectors as nonsense we assume the usual remapping and warn.

```
⟨Decide how to decode the normal map 83⟩ ≡
    GfVec4f scale, bias;
    const bool hasScale = inputValue(normal, "scale", &scale), hasBias = inputValue(normal, "bias", &bias);
    if (hasScale ∨ hasBias)
    {
        // as authored; UsdUVTexture defaults are scale 1, bias 0
        m.normalScale = hasScale ? std::array<float, 4>{scale[0], scale[1], scale[2], scale[3]}
                                 : std::array<float, 4>{1, 1, 1, 1};
        m.normalBias = hasBias ? std::array<float, 4>{bias[0], bias[1], bias[2], bias[3]}
                               : std::array<float, 4>{0, 0, 0, 0};
```

```
}
else
    warn("Normal map without scale/bias, assuming the usual 2x-1 remap (" + m.name + ")");
```

This code is used in section 82.

84. A material has one *uvSet*. When it has a diffuse texture, that texture has already chosen it; otherwise the normal map's primvar reader decides.

⟨Find the primvar that holds the normal-map coordinates 84⟩ ≡

```
// 'uvSet' is shared with the diffuse texture when there is one
if (m.texture < 0)
    if (const UsdShadeShader reader = sourceShader(normal.GetInput(TfToken("st"))))
    {
        VtValue varname;
        if (inputValue(reader, "varname", &varname))
        {
            if (varname.IsHolding<TfToken>())
                m.uvSet = varname.UncheckedGet<TfToken>().GetString();
            else if (varname.IsHolding<std::string>())
                m.uvSet = varname.UncheckedGet<std::string>();
        }
    }
```

This code is used in section 82.

85. A float input such as roughness starts at its default, then takes the authored value, if any. If it is driven by a texture, we record which channel: the output *outputs:g* means green, and so on; *outputs:r* and anything else, such as *outputs:rgb*, mean the first channel. Like normal maps, roughness and metallic maps are data and are decoded raw by default. The texture's scale and bias are reduced to those of the chosen channel, and if the image cannot be loaded, the texture's *fallback* for that channel replaces the constant.

⟨Read a float input that may come from a texture 85⟩ ≡

```
ScalarInput Scene::scalarInput(const UsdShadeShader& surface, const char* name, float fallback)
{
    ScalarInput in;
    in.value = fallback;
    inputValue(surface, name, &in.value);
    const UsdAttribute output = sourceOutput(surface.GetInput(TfToken(name)));
    if (¬output)
        return in;
    const UsdShadeShader texture(output.GetPrim());
    if (shaderId(texture) ≠ "UsdUVTexture")
    {
        warn("Unsupported shader driving " + std::string(name) + ": " + shaderId(texture).GetString());
        return in;
    }
    const std::string channel = UsdShadeUtils::GetBaseNameAndType(output.GetName()).first.GetString();
    in.channel = channel ≡ "g" ? 1 : channel ≡ "b" ? 2 : channel ≡ "a" ? 3 : 0;
    in.texture = loadTexture(texture, /*srgbByDefault=*/false);
    GfVec4f v;
    if (inputValue(texture, "scale", &v))
        in.scale = v[in.channel];
    if (inputValue(texture, "bias", &v))
        in.bias = v[in.channel];
    if (in.texture < 0 ∧ inputValue(texture, "fallback", &v))
        in.value = v[in.channel];
    return in;
}
```

This code is used in section 38.

86. Textures. Loading a texture turns a *UsdUVTexture* node into an index into *FlatGeometry::textures*, or −1 with a warning if anything goes wrong.

⟨Load a texture once 86⟩ ≡
```
int Scene::loadTexture(const UsdShadeShader& shader, bool srgbByDefault)
{
    ⟨Resolve the file of a texture 87⟩
    ⟨Decide the color space and wrap modes, and consult the texture cache 88⟩
    ⟨Decode a texture through the asset resolver 89⟩
}
```
This code is used in section 38.

87. The file is an *asset path*. USD resolves asset paths when it composes the stage, relative to the layer that authored them, so *GetResolvedPath* already gives an absolute location. For a file inside a `.usdz` package, that location has the form `package.usdz[textures/image.png]`, which no ordinary file function can open; we shall see in a moment how it is read. UDIM textures, whose name contains the pattern `<UDIM>` and stands for a whole set of images, one per tile of texture space, are not supported.

⟨Resolve the file of a texture 87⟩ ≡
```
SdfAssetPath file;
if (¬inputValue(shader, "file", &file) ∨ file.GetAssetPath().empty())
{
    warn("Texture without a file: " + shader.GetPath().GetString());
    return -1;
}
if (file.GetAssetPath().find("<UDIM>") ≠ std::string::npos)
{
    warn("UDIM textures are not supported: " + file.GetAssetPath());
    return -1;
}
const std::string resolved = file.GetResolvedPath();
if (resolved.empty())
{
    warn("Texture not found: " + file.GetAssetPath());
    return -1;
}
```
This code is used in section 86.

88. The color space. *sourceColorSpace* may be *sRGB*, *raw* or **auto** (the default). An explicit choice is obeyed. For **auto** the decision should depend on the image; we let it depend on the use instead, which is the same thing in practice: colors are sRGB, data (normals, roughness, metallic) are raw. The caller says which.

The renderer turns each *Texture* into a CUDA texture object, in which both the sRGB conversion and the wrap modes are fixed. The same image read in two different ways is therefore two textures, and the cache key contains all four properties: resolved path, color space and both wrap modes.

⟨Decide the color space and wrap modes, and consult the texture cache 88⟩ ≡
```
TfToken colorSpace;
inputValue(shader, "sourceColorSpace", &colorSpace);
const bool srgb = colorSpace ≡ "sRGB" ∨ (colorSpace ≠ "raw" ∧ srgbByDefault);
const TextureWrap wrapS = wrapMode(shader, "wrapS"), wrapT = wrapMode(shader, "wrapT");
const auto key = std::make_tuple(resolved, srgb, int(wrapS), int(wrapT));
if (auto it = m_textureCache.find(key); it ≠ m_textureCache.end())
    return it→second;
```
This code is used in section 86.

89. The asset resolver, *Ar*, is USD's layer between asset paths and bytes. Asking it to open the resolved path works for plain files and for files inside packages alike, and gives us the whole file in memory, which is exactly what the decoder `stb_image` wants. We ask the decoder for four channels of eight bits whatever the file contains; grayscale and RGB images gain the missing channels, with an opaque alpha. Failures are remembered in the cache as well, so a missing or broken image is reported once, not once per material that reads it.

⟨Decode a texture through the asset resolver 89⟩ ≡
```
// ArResolver also reads files packaged inside .usdz archives
int index = -1;
```

```
std::shared_ptr<ArAsset> asset = ArGetResolver().OpenAsset(ArResolvedPath(resolved));
std::shared_ptr<const char> buffer = asset ? asset→GetBuffer() : nullptr;
int w = 0, h = 0, channels = 0;
unsigned char* pixels =
    buffer ? stbi_load_from_memory(reinterpret_cast<const stbi_uc*>(buffer.get()), int(asset→GetSize()), &w,
            &h,
                                    &channels, 4)
           : nullptr;
if (pixels)
{
    Texture t;
    t.path = resolved;
    t.width = w;
    t.height = h;
    t.rgba.assign(pixels, pixels + size_t(w) * h * 4);
    t.srgb = srgb;
    t.wrapS = wrapS;
    t.wrapT = wrapT;
    stbi_image_free(pixels);
    m_geometry.textures.push_back(std::move(t));
    index = int(m_geometry.textures.size() − 1);
}
else
    warn("Could not load texture " + resolved + (buffer ? std::string(": ") + stbi_failure_reason() : ""));
m_textureCache[key] = index;
return index;
```

This code is used in section 86.

90. Materials per face. Back in the flattening loop, each prim needs a material for each of its faces. The material bound to the prim is found by *ComputeBoundMaterial*, which applies USD's binding rules for us: bindings are inherited from ancestors, and a binding marked "stronger than descendants" overrides those below it. A prim with no material falls back to its display color.

A mesh may also be divided into *GeomSubsets*, named sets of faces, and the subsets of the family *materialBind* can carry their own material binding. Those faces get the subset's material instead; indices outside the mesh are ignored. The count of such subsets is shown in the inspector.

⟨Assign a material to every face of a prim 90⟩ ≡

```
// materials: the prim's own binding, overridden per face by GeomSubsets
UsdShadeMaterialBindingAPI binding(p);
const UsdShadeMaterial primMaterial = binding.ComputeBoundMaterial();
const unsigned baseMaterial = primMaterial ? materialFor(primMaterial) : displayColorMaterial(p);
std::vector<unsigned> faceMaterial(local.faceCount, baseMaterial);
unsigned subsetMaterials = 0;
for (const UsdGeomSubset& subset : binding.GetMaterialBindSubsets())
{
    const UsdShadeMaterial m = UsdShadeMaterialBindingAPI(subset.GetPrim()).ComputeBoundMaterial();
    VtIntArray faces;
    if (¬m ∨ ¬subset.GetIndicesAttr().Get(&faces, m_time))
        continue;
    const unsigned index = materialFor(m);
    for (int f : faces)
        if (f ≥ 0 ∧ size_t(f) < faceMaterial.size())
            faceMaterial[f] = index;
    ++subsetMaterials;
}
```

This code is used in section 46.

91. Texture coordinates. Each prim gets one set of texture coordinates per triangle corner, read from one primvar. Which primvar? The one the material asks for: the prim's own material, unless some face uses a textured material, in which case the first such material decides. Prims whose subsets read different primvars are not handled, but they are rare.

Primvars can be inherited: a primvar authored with *constant* interpolation on an ancestor applies to

the whole subtree, which is why we use *FindPrimvarWithInheritance*. As with normals, *ComputeFlattened* resolves indexed primvars, and the lambda *cornerUV* picks the value of each corner by the same rules as ⟨Pick the shading normal of each triangle corner 58⟩. A missing value gives the coordinates $(0, 0)$.

⟨Read the texture coordinates of a prim 91⟩ ≡

```
// texture coordinates, read from the primvar the material asks for
std::string uvSet = g.materials[baseMaterial].uvSet;
for (unsigned m : faceMaterial)
    if (g.materials[m].usesTextures())
    {
        uvSet = g.materials[m].uvSet;
        break;
    }
VtArray<GfVec2f> uvs;
TfToken interpolation;
if (const UsdGeomPrimvar pv = UsdGeomPrimvarsAPI(p).FindPrimvarWithInheritance(TfToken(uvSet));
    pv ∧ pv.ComputeFlattened(&uvs, m_time))
    interpolation = pv.GetInterpolation();
auto cornerUV = [&](unsigned tri, int corner) → GfVec2f {
    size_t i = 0;
    if (interpolation ≡ UsdGeomTokens→faceVarying)
        i = local.cornerFv[3 * tri + corner];
    else if (interpolation ≡ UsdGeomTokens→vertex ∨ interpolation ≡ UsdGeomTokens→varying)
        i = local.indices[3 * tri + corner];
    else if (interpolation ≡ UsdGeomTokens→uniform)
        i = local.faceOfTri[tri];
    return i < uvs.size() ? uvs[i] : GfVec2f(0.0f);
};
```

This code is used in section 46.

92. World space. The local-to-world matrix M of the prim moves its points. USD's matrices act on row vectors, $p' = pM$, and *Transform* includes the translation. The points are transformed in double precision and stored as floats, and the bounds of the scene grow to include them. *baseVertex* is the index that the prim's first point receives in the shared array.

Normals cannot be transformed with M itself: under a non-uniform scale, a vector perpendicular to the surface would stop being perpendicular. The correct matrix is the inverse transpose, $(M^{-1})^{\mathsf{T}}$, applied without translation (*TransformDir*). It keeps the transformed normal perpendicular to every transformed tangent, because for a tangent u and a normal n with $u \cdot n = 0$ we have $(uM) \cdot \big(n(M^{-1})^{\mathsf{T}}\big) = u\,M M^{-1}\,n^{\mathsf{T}} = u \cdot n = 0$.

⟨Transform the points of a prim to world space 92⟩ ≡

```
const GfMatrix4d toWorld = xformCache.GetLocalToWorldTransform(p);
const GfMatrix4d normalToWorld = toWorld.GetInverse().GetTranspose();
const unsigned baseVertex = unsigned(g.vertices.size() / 3);
for (const GfVec3f& pt : local.points)
{
    const GfVec3d w = toWorld.Transform(GfVec3d(pt));
    for (int k = 0; k < 3; ++k)
    {
        g.vertices.push_back(float(w[k]));
        lo[k] = std::min(lo[k], float(w[k]));
        hi[k] = std::max(hi[k], float(w[k]));
    }
}
```

This code is used in section 46.

93. Appending the triangles. The prim's mesh id is the next free one; its triangles start where the previous prim's ended. The point indices are offset by *baseVertex*. Every triangle is tagged with the mesh id.

⟨Append the triangles of a prim to the flat geometry 93⟩ ≡

```
const unsigned meshId = unsigned(g.meshEntity.size());
const unsigned firstTri = unsigned(g.indices.size() / 3);
const unsigned triCount = unsigned(local.indices.size() / 3);
for (unsigned idx : local.indices)
```

```
        g.indices.push_back(baseVertex + idx);
    g.triMesh.insert(g.triMesh.end(), triCount, meshId);
    ⟨Append the material, texture coordinates and normals of each triangle 94⟩
    ⟨Record the mesh in the registry 95⟩
```

This code is used in section 46.

94. Then come the per-triangle attributes: the material of the triangle's face, and for each corner its texture coordinates and its shading normal in world space. Normals are normalized here, after the transformation, which also takes care of the unnormalized sums of the smooth normals and of the sphere's points. A degenerate normal is left at zero, and a flat mesh stores zeros throughout: both mean "use the triangle's own normal".

```
⟨Append the material, texture coordinates and normals of each triangle 94⟩ ≡
    for (unsigned t = 0; t < triCount; ++t)
    {
        g.triMaterial.push_back(faceMaterial[local.faceOfTri[t]]);
        for (int c = 0; c < 3; ++c)
        {
            const GfVec2f uv = cornerUV(t, c);
            g.triUVs.insert(g.triUVs.end(), {uv[0], uv[1]});
            GfVec3d n(0.0);
            if (¬local.cornerNormals.empty())
            {
                n = normalToWorld.TransformDir(GfVec3d(local.cornerNormals[3 * t + c]));
                if (n.GetLength() > 1e-12)
                    n.Normalize();
            }
            g.triNormals.insert(g.triNormals.end(), {float(n[0]), float(n[1]), float(n[2])});
        }
    }
```

This code is used in section 93.

95. The entity learns where its triangles are. Adding a *MeshGeometry* component while we iterate over the *PrimInfo* components is safe in EnTT, because each component type has its own storage and the view walks only the *PrimInfo* storage.

```
⟨Record the mesh in the registry 95⟩ ≡
    g.meshEntity.push_back(e);
    m_registry.emplace<MeshGeometry>(e, meshId, firstTri, triCount, baseMaterial, subsetMaterials,
                                     ¬uvs.empty());
```

This code is used in section 93.

96. A prim whose material reads textures but which has no texture coordinates would show one texel stretched over the whole surface. Better to say so.

```
⟨Warn about textured prims without texture coordinates 96⟩ ≡
    bool textured = false;
    for (unsigned m : faceMaterial)
        textured = textured ∨ g.materials[m].usesTextures();
    if (textured ∧ uvs.empty())
        warn("Textured prim without '" + uvSet + "' texture coordinates: " + info.path.GetString());
```

This code is used in section 46.

97. When the loop is over, the bounds are stored, but only if some geometry was found; an empty scene keeps the zero box rather than an inverted infinite one.

```
⟨Record the bounds of the scene 97⟩ ≡
    if (¬g.empty())
        for (int k = 0; k < 3; ++k)
        {
            g.boundsMin[k] = lo[k];
            g.boundsMax[k] = hi[k];
        }
```

This code is used in section 46.

The renderer

98. The renderer is the part of the program that talks to the GPU. It owns the CUDA stream, the OptiX device context and pipeline, the acceleration structure built over the scene's triangles, the textures, and the buffers into which the device programs write their pixels. The rest of the program sees a small interface: hand it a *FlatGeometry*, a camera and a few switches, call *render*, and read back an RGBA image together with the identity of the mesh under every pixel.

Everything the GPU needs for one launch travels in a single structure, *LaunchParams*; its definition is in `LaunchParams.h`. The renderer keeps a host copy of it, *m_params*, updates its fields as the user changes things, and copies it to the device right before every launch. Most setters below therefore do no more than write one field.

99. The interface. The header declares the class *OptixRenderer*. It cannot be copied: it owns GPU resources that must be released exactly once.

⟨`UsdOptixViewer/src/OptixRenderer.h` 99⟩ ≡

```
// Ray traces a FlatGeometry with OptiX into a host-side RGBA image.
#pragma once

⟨Headers and declarations of the renderer interface 100⟩

class OptixRenderer
{
public:
    ⟨Renderer construction and initialization 101⟩

    ⟨Renderer scene and view settings 102⟩

    ⟨Renderer frame production and results 103⟩

private:
    ⟨The device buffer helper 104⟩

    ⟨Private steps of the renderer 105⟩

    ⟨CUDA and OptiX handles of the renderer 106⟩

    ⟨Scene and output buffers of the renderer 107⟩

    ⟨Host-side results of the renderer 108⟩
};
```

This code is written to file `UsdOptixViewer/src/OptixRenderer.h`.

100. The header needs the CUDA types (*CUdeviceptr*, *CUstream*, *cudaMipmappedArray_t*), the OptiX handle types, and the shared parameter structure. The scene's *FlatGeometry* is only passed by reference, so a forward declaration is enough and the header does not drag in OpenUSD.

⟨Headers and declarations of the renderer interface 100⟩ ≡

```
#include <string>
#include <vector>

#include <cuda.h>
#include <cuda_runtime.h>
#include <optix_types.h>

#include "LaunchParams.h"

struct FlatGeometry;
```

This code is used in section 99.

101. Construction does nothing; the real work happens in *init*, which may fail (no GPU, an old driver, a PTX that does not compile) and reports the reason as text instead of throwing, so that the application can show it in a dialog and keep running without a viewport.

⟨Renderer construction and initialization 101⟩ ≡

```
OptixRenderer() = default;
~OptixRenderer();
OptixRenderer(const OptixRenderer&) = delete;
```

```
OptixRenderer& operator=(const OptixRenderer&) = delete;

// Creates the CUDA/OptiX context and pipeline. Returns false and sets
// `error` on failure.
bool init(std::string& error);
const std::string& deviceName() const { return m_deviceName; }
```

This code is used in section 99.

102. The scene is replaced as a whole with *setGeometry*. The view is described by the image size, the camera and the light. The remaining switches map one-to-one onto fields of the launch parameters and are read by the device programs: shadows, mipmapped texture filtering, normal maps, traced reflections, and the mesh to highlight as selected.

```
⟨Renderer scene and view settings 102⟩ ≡
    void setGeometry(const FlatGeometry& geometry);
    void resize(unsigned int width, unsigned int height);
    void setCamera(const float eye[3], const float target[3], const float up[3], float fovYDeg);
    void setLightDir(const float dir[3]);
    void setShadows(bool on) { m_params.shadows = on ? 1u : 0u; }
    void setMipmaps(bool on) { m_params.mipmaps = on ? 1u : 0u; }
    void setNormalMaps(bool on) { m_params.normalMaps = on ? 1u : 0u; }
    void setReflections(bool on) { m_params.reflections = on ? 1u : 0u; }
    void setSelectedMesh(unsigned int meshId) { m_params.selectedMesh = meshId; }
```

This code is used in section 99.

103. Rendering is progressive. Each call of *render* traces one new sample per pixel and adds it to an accumulation buffer on the GPU; the image handed back is the running average. The counter *frameIndex* in the launch parameters is the number of samples accumulated so far, so resetting the accumulation is just setting it to zero. The caller must do that after any change that invalidates the picture, such as a camera move.

The results are an array of RGBA pixels, ready to be uploaded to an SDL texture, and a lookup *meshIdAt* used for picking with the mouse.

```
⟨Renderer frame production and results 103⟩ ≡
    // Adds one progressive sample per pixel and copies the average to the
    // host buffers. Call resetAccumulation() after changing anything.
    void render();
    void resetAccumulation() { m_params.frameIndex = 0; }
    unsigned int samples() const { return m_params.frameIndex; }
    unsigned int width() const { return m_params.width; }
    unsigned int height() const { return m_params.height; }
    const std::vector<uchar4>& pixels() const { return m_pixels; }
    unsigned int meshIdAt(unsigned int x, unsigned int y) const;
    float lastRenderMs() const { return m_lastRenderMs; }
```

This code is used in section 99.

104. A *DeviceBuffer* is a block of GPU memory together with its size. OptiX wants device addresses as *CUdeviceptr*, a plain integer, so that is how we store them. The helper is deliberately small: it can be resized, filled from host memory, and freed. It has no destructor, because the order in which GPU memory is released matters less than the fact that it is released before the CUDA context goes away; the renderer's destructor frees every buffer explicitly.

```
⟨The device buffer helper 104⟩ ≡
    struct DeviceBuffer
    {
        CUdeviceptr ptr = 0;
        size_t size = 0;
        void resize(size_t bytes);
        void upload(const void* data, size_t bytes);
        void free();
    };
```

This code is used in section 99.

105.

⟨Private steps of the renderer 105⟩ ≡
```
void createPipeline();
void buildSbt();
void uploadTextures(const FlatGeometry& geometry);
void freeTextures();
```
This code is used in section 99.

106. The handles fall into three groups. First come CUDA and OptiX objects that live as long as the renderer: the CUDA context and stream, the OptiX context, the module compiled from the PTX, the program groups and the pipeline linked from them, and the shader binding table with its three record buffers.

⟨CUDA and OptiX handles of the renderer 106⟩ ≡
```
std::string m_deviceName;
CUcontext m_cuContext = nullptr;
CUstream m_stream = nullptr;
OptixDeviceContext m_context = nullptr;
OptixModule m_module = nullptr;
OptixPipeline m_pipeline = nullptr;
OptixProgramGroup m_raygenPG = nullptr, m_missPG[RAY_TYPE_COUNT] = {}, m_hitPG = nullptr;
OptixShaderBindingTable m_sbt = {};
DeviceBuffer m_raygenRecord, m_missRecords, m_hitRecords;
```
This code is used in section 99.

107. Then come the per-scene and per-image buffers. The scene arrays mirror those of *FlatGeometry*: vertex positions, triangle indices, and for every triangle its mesh, material, corner texture coordinates and corner normals. *m_gas* holds the acceleration structure. The image needs three buffers: the 8-bit colors shown on screen, the floating-point sums of the progressive samples, and the mesh ids for picking. Finally, every texture is a mipmapped CUDA array plus a texture object, and the objects are gathered in a device table that the shaders index by number.

⟨Scene and output buffers of the renderer 107⟩ ≡
```
DeviceBuffer m_vertices, m_indices, m_triMesh, m_triMaterial, m_triUVs, m_triNormals, m_materials, m_gas;
DeviceBuffer m_colorBuffer, m_accumBuffer, m_meshIdBuffer, m_paramsBuffer;
std::vector<cudaMipmappedArray_t> m_textureArrays;
std::vector<cudaTextureObject_t> m_textureObjects;
DeviceBuffer m_textureTable;
LaunchParams m_params = {};
```
This code is used in section 99.

108.

⟨Host-side results of the renderer 108⟩ ≡
```
std::vector<uchar4> m_pixels;
std::vector<unsigned int> m_meshIds;
float m_lastRenderMs = 0.0f;
```
This code is used in section 99.

109. The implementation. The source file is organized as follows; the helpers in the anonymous namespace are private to it.

⟨`UsdOptixViewer/src/OptixRenderer.cpp` 109⟩ ≡
```
⟨Headers of the renderer implementation 110⟩
⟨Declare the embedded PTX 111⟩
namespace {
⟨Error checking of CUDA and OptiX calls 112⟩
⟨The shader binding table record type 113⟩
⟨Small vector helpers of the host 114⟩
⟨Compute the next mip level of a texture 115⟩
} // namespace
```

⟨Resize a device buffer 119⟩

⟨Upload host data to a device buffer 120⟩

⟨Free a device buffer 121⟩

⟨Destroy the renderer 122⟩

⟨Initialize CUDA and OptiX 123⟩

⟨Create the OptiX pipeline 127⟩

⟨Build the shader binding table 135⟩

⟨Set the scene geometry 140⟩

⟨Upload the textures 149⟩

⟨Free the textures 154⟩

⟨Resize the output image 155⟩

⟨Set the camera 156⟩

⟨Set the light direction 157⟩

⟨Render one progressive sample 158⟩

⟨Look up the mesh under a pixel 162⟩

This code is written to file `UsdOptixViewer/src/OptixRenderer.cpp`.

110. OptiX's host functions are not exported by any library. Instead, one header defines a global table of function pointers,

`optix_function_table_definition.h`,

and another, `optix_stubs.h`, defines inline functions with the documented names that call through that table; *optixInit* fills the table from the driver. The definition header must be included in exactly one translation unit of the program, and this is it.

⟨Headers of the renderer implementation 110⟩ ≡

```
#include "OptixRenderer.h"

#include <algorithm>
#include <array>
#include <chrono>
#include <cmath>
#include <cstring>
#include <stdexcept>

#include <cuda_runtime.h>
#include <optix.h>
#include <optix_function_table_definition.h>
#include <optix_stubs.h>

#include "Scene.h"
```

This code is used in section 109.

111. The device programs reach us as PTX text, compiled into the executable by the build system (see ⟨Compile the device programs to PTX and embed them 13⟩). The array is terminated by a zero byte, and the size excludes it.

⟨Declare the embedded PTX 111⟩ ≡

```
extern const char g_deviceProgramsPTX[];
extern const size_t g_deviceProgramsPTX_size;
```

This code is used in section 109.

112. Errors. Every CUDA and OptiX call returns a status code. Rather than test each one by hand, we pass it to an overloaded *check* that throws an exception carrying the name of the operation and the library's description of the error. The exceptions are caught in *init* and in the application's render loop, which turn them into a message for the user.

⟨Error checking of CUDA and OptiX calls 112⟩ ≡

```
void check(cudaError_t r, const char* what)
{
```

```
    if (r ≠ cudaSuccess)
        throw std::runtime_error(std::string(what) + ": " + cudaGetErrorString(r));
}
void check(OptixResult r, const char* what)
{
    if (r ≠ OPTIX_SUCCESS)
        throw std::runtime_error(std::string(what) + ": " + optixGetErrorName(r));
}
```

This code is used in section 109.

113. Shader binding table records. OptiX finds the program to run for a ray generation, a miss or a hit through the *shader binding table* (SBT), an array of records in device memory. Each record starts with an opaque header of *OPTIX_SBT_RECORD_HEADER_SIZE* bytes, which identifies a program group, and may continue with data of our own; records must be aligned to *OPTIX_SBT_RECORD_ALIGNMENT* bytes. Our programs find all their data in the launch parameters, so the payload type is the empty *Empty*, and every record is just an aligned header.

⟨The shader binding table record type 113⟩ ≡

```
template <typename T>
struct alignas(OPTIX_SBT_RECORD_ALIGNMENT) SbtRecord
{
    alignas(OPTIX_SBT_RECORD_ALIGNMENT) char header[OPTIX_SBT_RECORD_HEADER_SIZE];
    T data;
};
struct Empty {};
```

This code is used in section 109.

114. CUDA's *float3* is a plain structure without operators on the host side, and the camera code needs a handful of vector operations. These five functions are all it takes; *normalize* leaves a zero vector unchanged instead of dividing by zero.

⟨Small vector helpers of the host 114⟩ ≡

```
float3 make3(const float v[3]) { return make_float3(v[0], v[1], v[2]); }
float3 sub(float3 a, float3 b) { return make_float3(a.x − b.x, a.y − b.y, a.z − b.z); }
float3 scale(float3 a, float s) { return make_float3(a.x * s, a.y * s, a.z * s); }
float3 cross(float3 a, float3 b) { return make_float3(a.y * b.z − a.z * b.y, a.z * b.x − a.x * b.z, a.x * b.y − a.y *
        b.x); }
float3 normalize(float3 a)
{
    const float len = std::sqrt(a.x * a.x + a.y * a.y + a.z * a.z);
    return len > 0.0f ? scale(a, 1.0f / len) : a;
}
```

This code is used in section 109.

115. Mipmaps. A texture seen from far away covers many texels per pixel. If we sampled only the full-resolution image, neighboring pixels would pick unrelated texels and the picture would shimmer with moiré patterns. The classical cure is a *mip chain*: a sequence of images, each half the size of the previous one, down to 1×1. The texture unit then reads from the level whose texels match the footprint of a pixel; the device programs estimate that footprint with ray differentials.

CUDA can store a mip chain but does not compute one, so we build the levels on the CPU. Each texel of level $k+1$ is the average of a 2×2 block of level k (a box filter). When a dimension is odd, the last row or column simply has no partner, and we repeat the edge texel; when it is already 1, it stays 1.

⟨Compute the next mip level of a texture 115⟩ ≡

```
// Next mip level with a 2x2 box filter (edge texels are repeated for odd
// sizes). sRGB color channels are averaged in linear space.
std::vector<unsigned char> downsample(const std::vector<unsigned char>& src, int w, int h, bool srgb)
{
    ⟨The sRGB to linear lookup table 116⟩
    const int nw = std::max(1, w / 2), nh = std::max(1, h / 2);
    std::vector<unsigned char> dst(size_t(nw) * nh * 4);
    for (int y = 0; y < nh; ++y)
```

```
        for (int x = 0; x < nw; ++x)
        {
            ⟨Gather the 2x2 block of source texels 117⟩
            for (int c = 0; c < 4; ++c)
            {
                ⟨Average one channel of the block 118⟩
            }
        }
    return dst;
}
```

This code is used in section 109.

116. Color textures are almost always stored in sRGB, a nonlinear encoding that spends more of the 256 levels on dark tones. Averaging sRGB values directly makes the smaller levels too dark: the average of black and white should be light that is half as bright, whose sRGB code is 188, not 128. We therefore convert to linear light, average, and convert back. The conversion from an 8-bit code $c/255$ to linear light is

$$\ell = \begin{cases} c/12.92, & \text{if } c \le 0.04045; \\ \big((c+0.055)/1.055\big)^{2.4}, & \text{otherwise,} \end{cases}$$

and since it has only 256 possible inputs we tabulate it once, the first time the function is called.

```
⟨The sRGB to linear lookup table 116⟩ ≡
    static const auto toLinear = [] {
        std::array<float, 256> lut{};
        for (int i = 0; i < 256; ++i)
        {
            const float c = i / 255.0f;
            lut[i] = c ≤ 0.04045f ? c / 12.92f : std::pow((c + 0.055f) / 1.055f, 2.4f);
        }
        return lut;
    }();
```

This code is used in section 115.

117. The four source texels of output texel (x, y) are those at $(2x, 2y)$ to $(2x+1, 2y+1)$, clamped to the image; pixels are four bytes, RGBA.

```
⟨Gather the 2x2 block of source texels 117⟩ ≡
    const int x0 = std::min(2 * x, w − 1), x1 = std::min(2 * x + 1, w − 1);
    const int y0 = std::min(2 * y, h − 1), y1 = std::min(2 * y + 1, h − 1);
    const unsigned char* texels[4] = {&src[(size_t(y0) * w + x0) * 4], &src[(size_t(y0) * w + x1) * 4],
                                      &src[(size_t(y1) * w + x0) * 4], &src[(size_t(y1) * w + x1) * 4]};
    unsigned char* out = &dst[(size_t(y) * nw + x) * 4];
```

This code is used in section 115.

118. The color channels of an sRGB texture go through linear space, and the result is encoded again with the inverse of the curve above,

$$c = \begin{cases} 12.92\,\ell, & \text{if } \ell \le 0.0031308; \\ 1.055\,\ell^{1/2.4} - 0.055, & \text{otherwise.} \end{cases}$$

The alpha channel, and every channel of a raw texture (normal maps, roughness, metallic), is linear already and is averaged as integers, with +2 for rounding to nearest.

```
⟨Average one channel of the block 118⟩ ≡
    if (srgb ∧ c < 3)
    {
        float sum = 0.0f;
        for (const unsigned char* t : texels)
            sum += toLinear[t[c]];
        const float l = sum * 0.25f;
        const float e = l ≤ 0.0031308f ? l * 12.92f : 1.055f * std::pow(l, 1.0f / 2.4f) − 0.055f;
        out[c] = (unsigned char)std::lround(std::clamp(e, 0.0f, 1.0f) * 255.0f);
```

```
    }
    else
        out[c] = (unsigned char)((texels[0][c] + texels[1][c] + texels[2][c] + texels[3][c] + 2) / 4);
```

This code is used in section 115.

119. Device buffers. Resizing to the current size is free; otherwise the old block is released and a new one allocated. A size of zero leaves the buffer empty, with a null pointer, which happens for a scene without textures or without triangles.

⟨Resize a device buffer 119⟩ ≡

```
// ---- DeviceBuffer ------------------------------------------------------
void OptixRenderer::DeviceBuffer::resize(size_t bytes)
{
    if (bytes ≡ size)
        return;
    free();
    if (bytes)
        check(cudaMalloc(reinterpret_cast<void**>(&ptr), bytes), "cudaMalloc");
    size = bytes;
}
```

This code is used in section 109.

120. Uploads are synchronous: they happen when a scene is loaded, not in the render loop, so simplicity wins over overlap.

⟨Upload host data to a device buffer 120⟩ ≡

```
void OptixRenderer::DeviceBuffer::upload(const void* data, size_t bytes)
{
    resize(bytes);
    if (bytes)
        check(cudaMemcpy(reinterpret_cast<void*>(ptr), data, bytes, cudaMemcpyHostToDevice),
              "cudaMemcpy");
}
```

This code is used in section 109.

121.

⟨Free a device buffer 121⟩ ≡

```
void OptixRenderer::DeviceBuffer::free()
{
    if (ptr)
        cudaFree(reinterpret_cast<void*>(ptr));
    ptr = 0;
    size = 0;
}
```

This code is used in section 109.

122. Life and death of the renderer. If *init* never got as far as creating an OptiX context, there is nothing to release. Otherwise we wait for the GPU to finish any work in flight and release the resources in the reverse order of their creation: textures and buffers first, then the pipeline, the program groups and the module it was linked from, then the OptiX context, and last the CUDA stream. The CUDA primary context itself is reference-counted by the runtime and is released when the process exits.

⟨Destroy the renderer 122⟩ ≡

```
// ---- OptixRenderer -----------------------------------------------------
OptixRenderer::~OptixRenderer()
{
    if (¬m_context)
        return;
    cudaDeviceSynchronize();
    freeTextures();
    for (DeviceBuffer* b : {&m_raygenRecord, &m_missRecords, &m_hitRecords, &m_vertices, &m_indices, &m_
```

```
            triMesh,
                        &m_triMaterial, &m_triUVs, &m_triNormals, &m_materials, &m_gas, &m_
                            colorBuffer,
                        &m_accumBuffer, &m_meshIdBuffer,
                        &m_paramsBuffer})
        b→free();
    optixPipelineDestroy(m_pipeline);
    optixProgramGroupDestroy(m_raygenPG);
    for (OptixProgramGroup pg : m_missPG)
        optixProgramGroupDestroy(pg);
    optixProgramGroupDestroy(m_hitPG);
    optixModuleDestroy(m_module);
    optixDeviceContextDestroy(m_context);
    cudaStreamDestroy(m_stream);
}
```

This code is used in section 109.

123. Initialization proceeds in three steps, any of which may throw; the exception's message becomes the error text.

```
⟨Initialize CUDA and OptiX 123⟩ ≡
bool OptixRenderer::init(std::string& error)
{
    try
    {
        ⟨Select the GPU and its primary context 124⟩
        ⟨Create the OptiX device context 125⟩
        ⟨Build the pipeline and set the default parameters 126⟩
    }
    catch (const std::exception& e)
    {
        error = e.what();
        return false;
    }
    return true;
}
```

This code is used in section 109.

124. The workstation has two GPUs; one is plenty for an interactive viewer, so we use device 0. The CUDA runtime creates a device's *primary context* lazily, on the first call that needs one; *cudaFree*(**nullptr**) is the customary way to force that creation without doing anything else. OptiX, however, is built on the lower-level driver API and wants a *CUcontext*, so we ask the driver for the context that is now current, which is that primary context. Runtime and driver calls then share the same context, and memory allocated with *cudaMalloc* is valid for OptiX. The stream orders all our GPU work: acceleration builds, launches and copies.

```
⟨Select the GPU and its primary context 124⟩ ≡
int deviceCount = 0;
check(cudaGetDeviceCount(&deviceCount), "cudaGetDeviceCount");
if (deviceCount ≡ 0)
    throw std::runtime_error("no CUDA capable device found");
check(cudaSetDevice(0), "cudaSetDevice");
check(cudaFree(nullptr), "cudaFree");  // creates the primary context
cudaDeviceProp props{};
check(cudaGetDeviceProperties(&props, 0), "cudaGetDeviceProperties");
m_deviceName = props.name;
check(cudaStreamCreate(&m_stream), "cudaStreamCreate");
if (cuCtxGetCurrent(&m_cuContext) ≠ CUDA_SUCCESS)
    throw std::runtime_error("cuCtxGetCurrent failed");
```

This code is used in section 123.

125. *optixInit* locates the OptiX implementation inside the installed driver and fills the function table; it fails if the driver is too old for the headers we compiled against (OptiX 9.1 needs driver R590 or newer). The

device context is then created on top of the CUDA context, with default options.

⟨Create the OptiX device context 125⟩ ≡
```
check(optixInit(), "optixInit");
OptixDeviceContextOptions options = {};
check(optixDeviceContextCreate(m_cuContext, &options, &m_context), "optixDeviceContextCreate");
```
This code is used in section 123.

126. With a context in hand we compile the pipeline and fill in the shader binding table, then give the launch parameters their initial values: nothing selected, every feature on, and a light coming from above, to the right and in front of the default camera. The device copy of the parameters needs space for one *LaunchParams*.

⟨Build the pipeline and set the default parameters 126⟩ ≡
```
createPipeline();
buildSbt();
m_params.selectedMesh = kNoMesh;
m_params.shadows = 1u;
m_params.mipmaps = 1u;
m_params.normalMaps = 1u;
m_params.reflections = 1u;
const float light[3] = {0.4f, 0.8f, 0.45f};
setLightDir(light);
m_paramsBuffer.resize(sizeof(LaunchParams));
```
This code is used in section 123.

127. The pipeline. An OptiX *pipeline* is the complete program run by a launch. It is assembled from *program groups*, which in turn refer to entry functions inside *modules*, the compiled form of PTX. Our single module holds all the device programs.

⟨Create the OptiX pipeline 127⟩ ≡
```
void OptixRenderer::createPipeline()
{
    ⟨Module compile options 128⟩
    ⟨Pipeline compile options 129⟩
    ⟨Compile the embedded PTX into a module 130⟩
    ⟨Create the ray generation program group 131⟩
    ⟨Create the miss program groups 132⟩
    ⟨Create the hit group 133⟩
    ⟨Link the pipeline and set its stack size 134⟩
}
```
This code is used in section 109.

128. The module is compiled with the default optimization level and the minimal debug level, which keeps line information for profilers without slowing the code down.

⟨Module compile options 128⟩ ≡
```
OptixModuleCompileOptions moduleOptions = {};
moduleOptions.maxRegisterCount = OPTIX_COMPILE_DEFAULT_MAX_REGISTER_COUNT;
moduleOptions.optLevel = OPTIX_COMPILE_OPTIMIZATION_DEFAULT;
moduleOptions.debugLevel = OPTIX_COMPILE_DEBUG_LEVEL_MINIMAL;
```
This code is used in section 127.

129. The pipeline compile options must be identical for every module of a pipeline, because they shape the calling convention between programs.

- The whole scene is one geometry acceleration structure (GAS), with no instancing, so we promise OptiX a traversable graph of a single GAS, its fastest case.
- The *payload* is the set of 32-bit registers a ray carries between the program that traces it and the programs it triggers. Radiance rays use six: the three color components as float bits, the id of the mesh that was hit, the recursion depth, and the state of the random number generator. Shadow rays use

only the first.

- Triangles report two *attributes*, their barycentric coordinates.
- The launch parameters live in a constant-memory variable that the device code calls *params*; OptiX copies our parameter buffer into it.
- Only built-in triangles are used, which lets OptiX drop support for other primitive types.

⟨Pipeline compile options 129⟩ ≡
```
OptixPipelineCompileOptions pipelineOptions = {};
pipelineOptions.traversableGraphFlags = OPTIX_TRAVERSABLE_GRAPH_FLAG_ALLOW_SINGLE_GAS;
pipelineOptions.numPayloadValues = 6;
pipelineOptions.numAttributeValues = 2;
pipelineOptions.exceptionFlags = OPTIX_EXCEPTION_FLAG_NONE;
pipelineOptions.pipelineLaunchParamsVariableName = "params";
pipelineOptions.usesPrimitiveTypeFlags = OPTIX_PRIMITIVE_TYPE_FLAGS_TRIANGLE;
```
This code is used in section 127.

130. The PTX is compiled to machine code for the installed GPU here, at run time. OptiX writes warnings and errors to a log buffer; *logSize* goes in as the capacity and comes back as the length used, so it is reset before every call. We do not print the log: on failure the error code already says what went wrong, and the PTX was compiled and checked at build time.

⟨Compile the embedded PTX into a module 130⟩ ≡
```
char log[2048];
size_t logSize = sizeof(log);
check(optixModuleCreate(m_context, &moduleOptions, &pipelineOptions, g_devicePrograms, g_devicePrograms_size,
                        log, &logSize, &m_module),
      "optixModuleCreate");
```
This code is used in section 127.

131. Program groups name their entry functions by string. OptiX requires the names to start with a prefix that tells the kind of program: `__raygen__`, `__miss__`, `__closesthit__` or `__anyhit__`. The ray generation program is launched once per pixel.

⟨Create the ray generation program group 131⟩ ≡
```
OptixProgramGroupOptions pgOptions = {};
OptixProgramGroupDesc desc = {};
desc.kind = OPTIX_PROGRAM_GROUP_KIND_RAYGEN;
desc.raygen.module = m_module;
desc.raygen.entryFunctionName = "__raygen__renderFrame";
logSize = sizeof(log);
check(optixProgramGroupCreate(m_context, &desc, 1, &pgOptions, log, &logSize, &m_raygenPG), "raygen group");
```
This code is used in section 127.

132. There are two ray types, radiance and shadow, and each has its own miss program: a radiance ray that escapes returns the color of the sky, while a shadow ray that escapes reports that the light is visible.

⟨Create the miss program groups 132⟩ ≡
```
const char* missNames[RAY_TYPE_COUNT] = {"__miss__radiance", "__miss__shadow"};
for (int i = 0; i < RAY_TYPE_COUNT; ++i)
{
    desc = {};
    desc.kind = OPTIX_PROGRAM_GROUP_KIND_MISS;
    desc.miss.module = m_module;
    desc.miss.entryFunctionName = missNames[i];
    logSize = sizeof(log);
    check(optixProgramGroupCreate(m_context, &desc, 1, &pgOptions, log, &logSize, &m_missPG[i]), "miss
        group");
}
```
This code is used in section 127.

133. A *hit group* bundles the programs run when a ray meets a triangle. The closest-hit program shades the nearest intersection; the any-hit program runs for every candidate intersection along the ray and is where

alpha cutouts discard the transparent parts of leaves and fences. A single hit group serves both ray types: shadow rays are traced with closest-hit disabled, so for them only the any-hit program matters, and that is exactly what a shadow test needs.

⟨Create the hit group 133⟩ ≡
```
desc = {};
desc.kind = OPTIX_PROGRAM_GROUP_KIND_HITGROUP;
desc.hitgroup.moduleCH = m_module;
desc.hitgroup.entryFunctionNameCH = "__closesthit__radiance";
desc.hitgroup.moduleAH = m_module;
desc.hitgroup.entryFunctionNameAH = "__anyhit__cutout";
logSize = sizeof(log);
check(optixProgramGroupCreate(m_context, &desc, 1, &pgOptions, log, &logSize, &m_hitPG), "hit group");
```
This code is used in section 127.

134. Linking needs the maximum depth of nested *optixTrace* calls. A camera ray (depth 1) may hit a glossy surface whose closest-hit program traces a reflection ray (depth 2); where that one lands, the closest-hit program traces a shadow ray (depth 3). Nothing goes deeper: the reflected hit does not reflect again, and shadow rays do not shade.

Recursion lives on a stack whose size OptiX must know. In general one adds up the stack needs of all programs; OptiX 9 can do this for us from the call depths: three levels of tracing, no continuation or direct callables, and a traversable graph one level deep.

⟨Link the pipeline and set its stack size 134⟩ ≡
```
OptixProgramGroup groups[] = {m_raygenPG, m_missPG[0], m_missPG[1], m_hitPG};
OptixPipelineLinkOptions linkOptions = {};
linkOptions.maxTraceDepth = 3;   // camera -> reflection -> shadow
logSize = sizeof(log);
check(optixPipelineCreate(m_context, &pipelineOptions, &linkOptions, groups, 4, log, &logSize, &m_pipeline),
      "optixPipelineCreate");
check(optixPipelineSetStackSizeFromCallDepths(m_pipeline, linkOptions.maxTraceDepth, 0, 0, 0, 1),
      "optixPipelineSetStackSizeFromCallDepths");
```
This code is used in section 127.

135. The shader binding table. When a ray of type r hits geometry, OptiX picks the hit record at index offset + stride × geometry index, where offset and stride are arguments of *optixTrace*; misses use the miss record named by another argument. The device programs pass the ray type as the miss index, an offset of 0 and a stride of *RAY_TYPE_COUNT*. With a single geometry (index 0), every ray therefore lands on hit record 0, and the second record exists only so that the table has one record per ray type, the layout that a stride of *RAY_TYPE_COUNT* presumes.

⟨Build the shader binding table 135⟩ ≡
```
void OptixRenderer::buildSbt()
{
    ⟨Pack the ray generation record 136⟩
    ⟨Pack the miss records 137⟩
    ⟨Pack the hit records 138⟩
    ⟨Point the shader binding table at the records 139⟩
}
```
This code is used in section 109.

136. Packing a record writes the header that identifies the program group; then the record is copied to the GPU.

⟨Pack the ray generation record 136⟩ ≡
```
SbtRecord<Empty> raygen;
check(optixSbtRecordPackHeader(m_raygenPG, &raygen), "pack raygen");
m_raygenRecord.upload(&raygen, sizeof(raygen));
```
This code is used in section 135.

137.

⟨Pack the miss records 137⟩ ≡
```
SbtRecord<Empty> miss[RAY_TYPE_COUNT];
for (int i = 0; i < RAY_TYPE_COUNT; ++i)
    check(optixSbtRecordPackHeader(m_missPG[i], &miss[i]), "pack miss");
m_missRecords.upload(miss, sizeof(miss));
```
This code is used in section 135.

138.

⟨Pack the hit records 138⟩ ≡
```
// Both ray types use the same record; the shadow ray disables closest-hit.
SbtRecord<Empty> hit[RAY_TYPE_COUNT];
for (int i = 0; i < RAY_TYPE_COUNT; ++i)
    check(optixSbtRecordPackHeader(m_hitPG, &hit[i]), "pack hit");
m_hitRecords.upload(hit, sizeof(hit));
```
This code is used in section 135.

139.

⟨Point the shader binding table at the records 139⟩ ≡
```
m_sbt.raygenRecord = m_raygenRecord.ptr;
m_sbt.missRecordBase = m_missRecords.ptr;
m_sbt.missRecordStrideInBytes = sizeof(SbtRecord<Empty>);
m_sbt.missRecordCount = RAY_TYPE_COUNT;
m_sbt.hitgroupRecordBase = m_hitRecords.ptr;
m_sbt.hitgroupRecordStrideInBytes = sizeof(SbtRecord<Empty>);
m_sbt.hitgroupRecordCount = RAY_TYPE_COUNT;
```
This code is used in section 135.

140. Loading a scene. *setGeometry* replaces everything that depends on the scene: the arrays, the materials, the textures and the acceleration structure. The old acceleration structure is invalid as soon as the arrays change, so its handle is cleared first, and the accumulated image restarts.

⟨Set the scene geometry 140⟩ ≡
```
void OptixRenderer::setGeometry(const FlatGeometry& g)
{
    ⟨Upload the flattened scene arrays 141⟩
    ⟨Pack the materials for the device 142⟩
    ⟨Upload the textures and publish the device pointers 145⟩
    ⟨Describe the triangles to OptiX 146⟩
    ⟨Choose the acceleration structure build options 147⟩
    ⟨Build the geometry acceleration structure 148⟩
}
```
This code is used in section 109.

141. The arrays of *FlatGeometry* are copied as they are. Their element types on the device (*float3*, *uint3*, *float2*) have the same layout as the consecutive **float**s and **unsigned int**s of the host vectors.

⟨Upload the flattened scene arrays 141⟩ ≡
```
m_params.handle = 0;
m_params.frameIndex = 0;
m_gas.free();
m_vertices.upload(g.vertices.data(), g.vertices.size() * sizeof(float));
m_indices.upload(g.indices.data(), g.indices.size() * sizeof(unsigned int));
m_triMesh.upload(g.triMesh.data(), g.triMesh.size() * sizeof(unsigned int));
m_triMaterial.upload(g.triMaterial.data(), g.triMaterial.size() * sizeof(unsigned int));
m_triUVs.upload(g.triUVs.data(), g.triUVs.size() * sizeof(float));
m_triNormals.upload(g.triNormals.data(), g.triNormals.size() * sizeof(float));
```
This code is used in section 140.

142. Materials are another matter. The scene describes them with C++ types that are convenient on the host (*std*::*array*, **bool**, *ScalarInput*); the device wants the flat, CUDA-aligned *MaterialData* of `LaunchParams.h`.

We convert field by field.

⟨Pack the materials for the device 142⟩ ≡
```
std::vector<MaterialData> materials;
for (const Material& m : g.materials)
{
    ⟨Conversions from host material fields 143⟩
    ⟨Fill in one device material 144⟩
}
m_materials.upload(materials.data(), materials.size() * sizeof(MaterialData));
```
This code is used in section 140.

143. Three small lambdas turn the host types into their device counterparts: a four-component array into a *float4*, a three-component one into a *float3*, and a scalar input (a constant, or a channel of a texture with a scale and a bias) into a *ScalarParam*.

⟨Conversions from host material fields 143⟩ ≡
```
auto f4 = [](const std::array<float, 4>& v) { return make_float4(v[0], v[1], v[2], v[3]); };
auto f3 = [](const std::array<float, 3>& v) { return make_float3(v[0], v[1], v[2]); };
auto scalar = [](const ScalarInput& in) {
    return ScalarParam{in.value, in.texture, in.channel, in.scale, in.bias};
};
```
This code is used in section 142.

144. The fields follow the UsdPreviewSurface inputs: the diffuse texture's scale and bias, those of the normal map and the specular color, the constant colors, texture indices (−1 meaning none), roughness and metallic, the alpha-cutout channel and threshold, the workflow flag and the index of refraction. Texture indices refer to the table built below.

⟨Fill in one device material 144⟩ ≡
```
MaterialData d = {};
d.texScale = f4(m.scale);
d.texBias = f4(m.bias);
d.normalScale = f4(m.normalScale);
d.normalBias = f4(m.normalBias);
d.specScale = f4(m.specularScale);
d.specBias = f4(m.specularBias);
d.color = f3(m.color);
d.texture = m.texture;
d.specColor = f3(m.specularColor);
d.specTexture = m.specularTexture;
d.roughness = scalar(m.roughness);
d.metallic = scalar(m.metallic);
d.opacityTexture = m.opacityTexture;
d.opacityChannel = m.opacityChannel;
d.opacityThreshold = m.opacityThreshold;
d.normalTexture = m.normalTexture;
d.specularWorkflow = m.specularWorkflow ? 1 : 0;
d.ior = m.ior;
materials.push_back(d);
```
This code is used in section 142.

145. With the textures uploaded, all device addresses are known and we store them in the launch parameters. A scene without triangles stops here: the handle stays zero, and *render* then simply paints the background.

⟨Upload the textures and publish the device pointers 145⟩ ≡
```
uploadTextures(g);
m_params.vertices = reinterpret_cast<const float3*>(m_vertices.ptr);
m_params.indices = reinterpret_cast<const uint3*>(m_indices.ptr);
m_params.triMesh = reinterpret_cast<const unsigned int*>(m_triMesh.ptr);
m_params.triMaterial = reinterpret_cast<const unsigned int*>(m_triMaterial.ptr);
m_params.triUVs = reinterpret_cast<const float2*>(m_triUVs.ptr);
m_params.triNormals = reinterpret_cast<const float3*>(m_triNormals.ptr);
m_params.materials = reinterpret_cast<const MaterialData*>(m_materials.ptr);
```

```
m_params.textures = reinterpret_cast<const cudaTextureObject_t*>(m_textureTable.ptr);
if (g.empty())
    return;
```

This code is used in section 140.

146. The acceleration structure. OptiX builds a bounding volume hierarchy over the triangles, the *geometry acceleration structure*, which the hardware ray-tracing cores traverse. The whole scene is one build input: an indexed triangle array of world-space *float3* vertices and *uint3* index triplets, with one SBT record.

The geometry flag deserves a word. The usual advice for opaque geometry is the flag

$$OPTIX_GEOMETRY_FLAG_DISABLE_ANYHIT,$$

which spares the traversal a call per candidate hit, and an early version of the viewer did exactly that. Alpha cutouts need the any-hit program, though, so the flag is now *OPTIX_GEOMETRY_FLAG_NONE*; for opaque materials the program returns at once.

⟨Describe the triangles to OptiX 146⟩ ≡

```
// any-hit stays enabled for alpha cutouts; it returns early for opaque materials
const unsigned int flags[] = {OPTIX_GEOMETRY_FLAG_NONE};
OptixBuildInput input = {};
input.type = OPTIX_BUILD_INPUT_TYPE_TRIANGLES;
OptixBuildInputTriangleArray& tri = input.triangleArray;
tri.vertexFormat = OPTIX_VERTEX_FORMAT_FLOAT3;
tri.vertexStrideInBytes = sizeof(float3);
tri.numVertices = unsigned(g.vertices.size() / 3);
tri.vertexBuffers = &m_vertices.ptr;
tri.indexFormat = OPTIX_INDICES_FORMAT_UNSIGNED_INT3;
tri.indexStrideInBytes = sizeof(uint3);
tri.numIndexTriplets = unsigned(g.indices.size() / 3);
tri.indexBuffer = m_indices.ptr;
tri.flags = flags;
tri.numSbtRecords = 1;
```

This code is used in section 140.

147. The scene is static once loaded, so we ask for the structure that traces fastest, at the price of a slower build, and do not request updates or compaction.

⟨Choose the acceleration structure build options 147⟩ ≡

```
OptixAccelBuildOptions accelOptions = {};
accelOptions.buildFlags = OPTIX_BUILD_FLAG_PREFER_FAST_TRACE;
accelOptions.operation = OPTIX_BUILD_OPERATION_BUILD;
```

This code is used in section 140.

148. OptiX first tells us how much memory the build needs: scratch space for the build itself and room for the result. The build runs on our stream and writes the *traversable handle* straight into the launch parameters; we wait for it before releasing the scratch buffer.

⟨Build the geometry acceleration structure 148⟩ ≡

```
OptixAccelBufferSizes sizes = {};
check(optixAccelComputeMemoryUsage(m_context, &accelOptions, &input, 1, &sizes),
      "optixAccelComputeMemoryUsage");
DeviceBuffer temp;
temp.resize(sizes.tempSizeInBytes);
m_gas.resize(sizes.outputSizeInBytes);
check(optixAccelBuild(m_context, m_stream, &accelOptions, &input, 1, temp.ptr, temp.size, m_gas.ptr, m_gas.size,
                      &m_params.handle, nullptr, 0),
      "optixAccelBuild");
check(cudaStreamSynchronize(m_stream), "accel build");
temp.free();
```

This code is used in section 140.

149. Textures. Every texture of the scene becomes a CUDA *mipmapped array* holding its full mip chain, and a *texture object*: a 64-bit handle that combines the array with a description of how to sample it. The

handles go into a table on the device, indexed by the texture numbers stored in the materials.

```
⟨Upload the textures 149⟩ ≡
    void OptixRenderer::uploadTextures(const FlatGeometry& g)
    {
        freeTextures();
        for (const Texture& t : g.textures)
        {
            ⟨Allocate a mipmapped array for the texture 150⟩
            ⟨Upload every mip level 151⟩
            ⟨Describe how the texture is sampled 152⟩
            ⟨Create the texture object 153⟩
        }
        m_textureTable.upload(m_textureObjects.data(), m_textureObjects.size() * sizeof(cudaTextureObject_t));
    }
```

This code is used in section 109.

150. A chain that halves down to 1×1 has $1 + \lfloor \log_2 \max(w, h) \rfloor$ levels. Texels are four unsigned bytes, as decoded by the scene.

```
⟨Allocate a mipmapped array for the texture 150⟩ ≡
    // full mip chain, down to 1x1
    const unsigned levels = 1u + unsigned(std::floor(std::log2(double(std::max(t.width, t.height)))));
    const cudaChannelFormatDesc format = cudaCreateChannelDesc<uchar4>();
    cudaMipmappedArray_t mipmaps = nullptr;
    check(cudaMallocMipmappedArray(&mipmaps, &format, make_cudaExtent(size_t(t.width), size_t(t.height), 0),
                                   levels),
          "cudaMallocMipmappedArray");
    m_textureArrays.push_back(mipmaps);
```

This code is used in section 149.

151. Level 0 is the image itself. After uploading each level we compute the next one from it with *downsample*, which halves the dimensions the same way CUDA does, never going below 1.

```
⟨Upload every mip level 151⟩ ≡
    std::vector<unsigned char> level = t.rgba;
    int w = t.width, h = t.height;
    for (unsigned l = 0; l < levels; ++l)
    {
        cudaArray_t array = nullptr;
        check(cudaGetMipmappedArrayLevel(&array, mipmaps, l), "cudaGetMipmappedArrayLevel");
        check(cudaMemcpy2DToArray(array, 0, 0, level.data(), size_t(w) * 4, size_t(w) * 4, size_t(h),
                                  cudaMemcpyHostToDevice),
              "texture upload");
        if (l + 1 < levels)
        {
            level = downsample(level, w, h, t.srgb);
            w = std::max(1, w / 2);
            h = std::max(1, h / 2);
        }
    }
```

This code is used in section 149.

152. The sampling description is where the texture hardware earns its keep.

- The USD wrap modes map onto CUDA address modes: `repeat` to wrap, `mirror` to mirror, `clamp` to clamp, and `black` to border, whose border color is zero.
- Filtering is *trilinear*: bilinear within a level, and linear between the two levels nearest to the requested one.
- Up to eight anisotropic samples are taken along the longer axis of a pixel's footprint. This only works with explicit gradients, which is why the device programs sample with *tex2DGrad*; it keeps floors seen at grazing angles sharp instead of blurring them to the coarsest level the short axis would allow.

- Texels are read as normalized floats in $[0, 1]$, addressed with normalized coordinates.
- For sRGB textures the hardware converts each texel to linear light *before* filtering, which is the correct order and costs nothing.

⟨Describe how the texture is sampled 152⟩ ≡
```
cudaResourceDesc resource = {};
resource.resType = cudaResourceTypeMipmappedArray;
resource.res.mipmap.mipmap = mipmaps;
cudaTextureDesc desc = {};
auto mode = [](TextureWrap w) {
    switch (w)
    {
    case TextureWrap::Mirror: return cudaAddressModeMirror;
    case TextureWrap::Clamp: return cudaAddressModeClamp;
    case TextureWrap::Black: return cudaAddressModeBorder;
    default: return cudaAddressModeWrap;
    }
};
desc.addressMode[0] = mode(t.wrapS);
desc.addressMode[1] = mode(t.wrapT);
desc.filterMode = cudaFilterModeLinear;
desc.mipmapFilterMode = cudaFilterModeLinear;
desc.maxMipmapLevelClamp = float(levels - 1);
desc.maxAnisotropy = 8;
desc.readMode = cudaReadModeNormalizedFloat;
desc.normalizedCoords = 1;
desc.sRGB = t.srgb ? 1 : 0;   // the hardware converts sRGB texels to linear
```
This code is used in section 149.

153.

⟨Create the texture object 153⟩ ≡
```
cudaTextureObject_t object = 0;
check(cudaCreateTextureObject(&object, &resource, &desc, nullptr), "cudaCreateTextureObject");
m_textureObjects.push_back(object);
```
This code is used in section 149.

154. Texture objects must be destroyed before the arrays they refer to.

⟨Free the textures 154⟩ ≡
```
void OptixRenderer::freeTextures()
{
    for (cudaTextureObject_t object : m_textureObjects)
        cudaDestroyTextureObject(object);
    for (cudaMipmappedArray_t mipmaps : m_textureArrays)
        cudaFreeMipmappedArray(mipmaps);
    m_textureObjects.clear();
    m_textureArrays.clear();
    m_textureTable.free();
}
```
This code is used in section 109.

155. The image and the camera. The output buffers follow the size of the viewport. A minimized or collapsed viewport may report a zero size, which we round up to one pixel so that every buffer stays valid. Resizing restarts the accumulation, since the old sums belong to other pixels. The accumulation buffer holds one *float4* per pixel: the sum of the sample colors, and in the fourth component the number of samples.

⟨Resize the output image 155⟩ ≡
```
void OptixRenderer::resize(unsigned int width, unsigned int height)
{
    width = width ? width : 1;
    height = height ? height : 1;
    if (width ≡ m_params.width ∧ height ≡ m_params.height)
```

```
        return;
    m_params.width = width;
    m_params.height = height;
    m_colorBuffer.resize(size_t(width) * height * sizeof(uchar4));
    m_accumBuffer.resize(size_t(width) * height * sizeof(float4));
    m_params.accumBuffer = reinterpret_cast<float4*>(m_accumBuffer.ptr);
    m_params.frameIndex = 0;
    m_meshIdBuffer.resize(size_t(width) * height * sizeof(unsigned int));
    m_params.colorBuffer = reinterpret_cast<uchar4*>(m_colorBuffer.ptr);
    m_params.meshIdBuffer = reinterpret_cast<unsigned int*>(m_meshIdBuffer.ptr);
    m_pixels.resize(size_t(width) * height);
    m_meshIds.resize(size_t(width) * height);
}
```

This code is used in section 109.

156. A pinhole camera is described to the device by its eye point and three vectors. Let $\mathbf{w}$ be the unit vector from the eye towards the target, $\mathbf{u} = \mathbf{w} \times \mathbf{up}$ normalized (pointing right), and $\mathbf{v} = \mathbf{u} \times \mathbf{w}$ (pointing up). If ϕ is the vertical field of view and a the aspect ratio, the image plane at distance 1 spans $\pm\tan(\phi/2)$ vertically and $\pm a\tan(\phi/2)$ horizontally. Scaling $\mathbf{u}$ and $\mathbf{v}$ by those half-extents lets the ray generation program write the direction through normalized device coordinates $(x, y) \in [-1, 1]^2$ simply as $\mathbf{w} + x\,\mathbf{u} + y\,\mathbf{v}$. The aspect ratio is taken from the current image size, so the camera must be set after *resize*.

⟨Set the camera 156⟩ ≡

```
void OptixRenderer::setCamera(const float eyeIn[3], const float targetIn[3], const float upIn[3], float fovYDeg)
{
    const float3 eye = make3(eyeIn);
    const float3 w = normalize(sub(make3(targetIn), eye));
    const float3 u = normalize(cross(w, make3(upIn)));
    const float3 v = cross(u, w);
    const float halfH = std::tan(fovYDeg * 0.5f * float(M_PI) / 180.0f);
    const float aspect = m_params.height ? float(m_params.width) / float(m_params.height) : 1.0f;
    m_params.camera = {eye, scale(u, halfH * aspect), scale(v, halfH), w};
}
```

This code is used in section 109.

157. The light is directional; the device programs want the unit vector that points towards it.

⟨Set the light direction 157⟩ ≡

```
void OptixRenderer::setLightDir(const float dir[3])
{
    m_params.lightDir = normalize(make3(dir));
}
```

This code is used in section 109.

158. Rendering. Each call adds one sample per pixel. If there is no image or no scene there is nothing to trace: an empty scene is painted in a dark gray and reports no mesh anywhere.

⟨Render one progressive sample 158⟩ ≡

```
void OptixRenderer::render()
{
    ⟨Handle an empty image or scene 159⟩
    ⟨Launch one sample per pixel 160⟩
    ⟨Download the image and the picking ids 161⟩
}
```

This code is used in section 109.

159.

⟨Handle an empty image or scene 159⟩ ≡

```
if (¬m_params.width ∨ ¬m_params.height)
    return;
const auto start = std::chrono::steady_clock::now();
if (¬m_params.handle)
```

```
{
    // Nothing to trace: clear to the background color.
    std::fill(m_pixels.begin(), m_pixels.end(), make_uchar4(40, 42, 48, 255));
    std::fill(m_meshIds.begin(), m_meshIds.end(), kNoMesh);
    return;
}
```

This code is used in section 158.

160. The host copy of the launch parameters goes to the device, and the pipeline is launched over a two-dimensional grid with one thread per pixel. Both operations are queued on the same stream, so the launch sees the new parameters; the frame index tells the ray generation program whether to start a new sum (frame 0) or add to the existing one.

```
⟨Launch one sample per pixel 160⟩ ≡
    check(cudaMemcpyAsync(reinterpret_cast<void*>(m_paramsBuffer.ptr), &m_params, sizeof(m_params),
                          cudaMemcpyHostToDevice, m_stream),
          "upload params");
    check(optixLaunch(m_pipeline, m_stream, m_paramsBuffer.ptr, sizeof(LaunchParams), &m_sbt, m_params.width,
                      m_params.height, 1),
          "optixLaunch");
```

This code is used in section 158.

161. The averaged image comes back after every sample, so the user sees it converge. The mesh ids are another story: the first sample shoots its rays through the pixel centers and is the only one that writes them, because later samples jitter their rays inside the pixel, and a jittered id would flicker along the edges of objects. We download them only after that first sample. Then we wait for the stream, count the sample, and record how long it took, for the status bar.

```
⟨Download the image and the picking ids 161⟩ ≡
    check(cudaMemcpyAsync(m_pixels.data(), reinterpret_cast<void*>(m_colorBuffer.ptr), m_colorBuffer.size,
                          cudaMemcpyDeviceToHost, m_stream),
          "download color");
    if (m_params.frameIndex ≡ 0)
        check(cudaMemcpyAsync(m_meshIds.data(), reinterpret_cast<void*>(m_meshIdBuffer.ptr), m_
                meshIdBuffer.size,
                              cudaMemcpyDeviceToHost, m_stream),
              "download ids");
    check(cudaStreamSynchronize(m_stream), "render");
    ++m_params.frameIndex;
    m_lastRenderMs = std::chrono::duration<float, std::milli>(std::chrono::steady_clock::now() − start).count();
```

This code is used in section 158.

162. Picking reads the host copy of the ids; coordinates outside the image hit nothing.

```
⟨Look up the mesh under a pixel 162⟩ ≡
    unsigned int OptixRenderer::meshIdAt(unsigned int x, unsigned int y) const
    {
        if (x ≥ m_params.width ∨ y ≥ m_params.height)
            return kNoMesh;
        return m_meshIds[size_t(y) * m_params.width + x];
    }
```

This code is used in section 109.

The device programs

163. This part runs on the GPU. OptiX organizes a ray tracer as a handful of small programs that it calls at well-defined moments: a *ray generation* program runs once per pixel and shoots the first rays; when a ray finds its nearest intersection, the *closest-hit* program computes the color it carries back; when a ray escapes, a *miss* program does; and for every candidate intersection an optional *any-hit* program may decide that the hit does not count. Our shading model follows UsdPreviewSurface: Lambertian diffuse reflection plus GGX specular reflection, lit by one directional light with hard shadows and by one glossy reflection bounce, under a simple gradient sky.

The file is compiled to PTX by `nvcc` and embedded in the executable, as described in the build-system part.

⟨`UsdOptixViewer/src/devicePrograms.cu` 163⟩ ≡
⟨Opening of the device programs file 164⟩
⟨Small vector algebra on the device 165⟩
⟨Random numbers on the device 166⟩
⟨Payload helpers and the sky 169⟩
⟨Texture sampling with ray differentials 172⟩
⟨Reading scalar material inputs 178⟩
⟨Normal mapping in the tangent frame 179⟩
⟨The GGX microfacet model 182⟩
⟨The ray generation program 186⟩
⟨The miss programs 190⟩
⟨The any-hit program for alpha cutouts 191⟩
⟨The closest-hit program 192⟩
This code is written to file `UsdOptixViewer/src/devicePrograms.cu`.

164. The launch parameters of the shared-parameters part are declared here as a *__constant__* variable called *params*. The name matters: the host tells OptiX, when it compiles the module, that the launch parameters live in a variable of that name, and OptiX copies the host's buffer into it before each launch. The **extern** `"C"` keeps the name from being mangled. As noted in the build system, `nvcc` remarks that this **extern** declaration is treated as a definition, which is precisely what we want.

The light intensity looks arbitrary, and in a sense it is. The very first version of the viewer shaded a lit surface with $0.7\,\rho\,(n \cdot l)$, where ρ is the diffuse color. A Lambertian surface reflects $\rho/\pi \cdot E\,(n \cdot l)$ under irradiance E, so choosing $E = 0.7\pi$ keeps that familiar brightness while the shading became physically based.

⟨Opening of the device programs file 164⟩ ≡
```
// OptiX programs: progressive pinhole camera, UsdPreviewSurface-style shading
// (Lambert diffuse + GGX specular) with a directional light, hard shadows,
// one glossy reflection bounce and a gradient sky.
#include <optix.h>

#include "LaunchParams.h"

extern "C" __constant__ LaunchParams params;

constexpr float kPi = 3.14159265358979f;
constexpr float kLightIntensity = 0.7f * kPi;   // keeps the diffuse look of the earlier versions
```
This code is used in section 163.

165. CUDA's vector types come without arithmetic. The CUDA samples ship a header of operators, but it is not part of the toolkit, so we write the few we need. Every function is *__forceinline__*: they are tiny, and calls in OptiX programs are costly. *normalize* uses the fast reciprocal square root and refuses to divide by a length below 10^{-10}, so a degenerate vector produces a huge but finite result instead of a NaN that would poison the accumulated image forever. *mix* is linear interpolation, $\mathrm{mix}(a, b, t) = (1 - t)\,a + t\,b$.

⟨Small vector algebra on the device 165⟩ ≡
```
static __forceinline__ __device__ float3 operator+(float3 a, float3 b) { return make_float3(a.x + b.x, a.y + b.y, a.z
```

```
    + b.z); }
static __forceinline__ __device__ float3 operator−(float3 a, float3 b) { return make_float3(a.x − b.x, a.y − b.y, a.z
    − b.z); }
static __forceinline__ __device__ float3 operator*(float3 a, float s) { return make_float3(a.x * s, a.y * s, a.z * s); }
static __forceinline__ __device__ float3 operator*(float3 a, float3 b) { return make_float3(a.x * b.x, a.y * b.y, a.z *
    b.z); }
static __forceinline__ __device__ float dot(float3 a, float3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
static __forceinline__ __device__ float3 cross(float3 a, float3 b)
{
    return make_float3(a.y * b.z − a.z * b.y, a.z * b.x − a.x * b.z, a.x * b.y − a.y * b.x);
}
static __forceinline__ __device__ float3 normalize(float3 a) { return a * rsqrtf(fmaxf(dot(a, a), 1e-20f)); }
static __forceinline__ __device__ float3 mix(float3 a, float3 b, float t) { return a * (1.0f − t) + b * t; }
```

This code is used in section 163.

166. Random numbers. Progressive rendering needs random numbers that differ from pixel to pixel and from frame to frame, yet cost almost nothing. We use the usual two-stage recipe of the OptiX samples. First, the pixel index and the frame number are hashed together with four rounds of TEA, the Tiny Encryption Algorithm of Wheeler and Needham; four rounds are far fewer than encryption would require, but enough to make the seeds of neighbouring pixels and consecutive frames look unrelated.

⟨Random numbers on the device 166⟩ ≡

```
// ---- random numbers ------------------------------------------------------
⟨The TEA hash 167⟩
⟨A linear congruential generator 168⟩
```

This code is used in section 163.

167. The constant 0x9e3779b9 is $2^{32}/\varphi$, with φ the golden ratio; the other four are TEA's key. All arithmetic is modulo 2^{32}, courtesy of unsigned overflow.

⟨The TEA hash 167⟩ ≡

```
static __forceinline__ __device__ unsigned int tea(unsigned int v0, unsigned int v1)
{
    unsigned int s0 = 0;
    for (int n = 0; n < 4; ++n)
    {
        s0 += 0x9e3779b9;
        v0 += ((v1 << 4) + 0xa341316c) ^ (v1 + s0) ^ ((v1 >> 5) + 0xc8013ea4);
        v1 += ((v0 << 4) + 0xad90777d) ^ (v0 + s0) ^ ((v0 >> 5) + 0x7e95761e);
    }
    return v0;
}
```

This code is used in section 166.

168. From the seed, a linear congruential generator with the constants of *Numerical Recipes* produces the numbers themselves. The low bits of an LCG are its weakest, but we keep 24 bits and divide by 2^{24}, which gives floats uniformly spaced in $[0, 1)$ that are exactly representable in single precision. The seed is passed by reference and advanced in place, so one seed can serve a whole path, even across calls to *optixTrace*.

⟨A linear congruential generator 168⟩ ≡

```
static __forceinline__ __device__ float rnd(unsigned int& seed)
{
    seed = 1664525u * seed + 1013904223u;
    return float(seed & 0x00ffffffu) / float(0x01000000u);
}
```

This code is used in section 166.

169. The payload. A ray carries a few 32-bit registers, the *payload*, that the programs it reaches can read and write. Radiance rays use six: the red, green and blue components of the returned color, stored as the bit patterns of **float**s; the id of the mesh that was hit, for picking; the recursion depth of the ray, which tells primary rays from reflection rays; and the random seed, which travels with the ray so that a reflection

continues the same random sequence instead of starting a correlated one. Shadow rays use a single register that the shadow miss program sets to 1.

⟨Payload helpers and the sky 169⟩ ≡

```
// ---- payload ----------------------------------------------------------------
// Radiance rays: p0..p2 = rgb (float bits), p3 = mesh id, p4 = depth, p5 = rng seed.
// Shadow rays: p0 = 1 when the light is visible.
static __forceinline__ __device__ void setColor(float3 c, unsigned int meshId)
{
    optixSetPayload_0(__float_as_uint(c.x));
    optixSetPayload_1(__float_as_uint(c.y));
    optixSetPayload_2(__float_as_uint(c.z));
    optixSetPayload_3(meshId);
}
```

⟨Recognize shadow rays 170⟩

⟨The sky 171⟩

This code is used in section 163.

170. Some code, such as texture sampling for the cutout test, runs for both kinds of rays and must know which one it serves. Shadow rays are the only rays traced with *OPTIX_RAY_FLAG_DISABLE_CLOSESTHIT*, so that flag identifies them. The distinction matters because a shadow ray has only one payload register: reading register 4 of a shadow ray would be an error.

⟨Recognize shadow rays 170⟩ ≡

```
static __forceinline__ __device__ bool isShadowRay()
{
    return optixGetRayFlags() & OPTIX_RAY_FLAG_DISABLE_CLOSESTHIT;
}
```

This code is used in section 169.

171. The sky is a vertical gradient, from a dark grey below the horizon to a greyish blue overhead, interpolated by the height d_y of the direction. It is the background of the image and, through reflections, the only environment lighting of the scene. It always uses the world's y axis, even for stages whose up axis is z.

⟨The sky 171⟩ ≡

```
static __forceinline__ __device__ float3 sky(float3 dir)
{
    return mix(make_float3(0.10f, 0.10f, 0.12f), make_float3(0.32f, 0.38f, 0.48f), 0.5f * (dir.y + 1.0f));
}
```

This code is used in section 169.

172. Texture sampling with ray differentials. A texture seen from afar covers many texels within one pixel; sampling it at a single point then gives the shimmering moiré patterns of aliasing. GPUs avoid this with *mipmaps*, prefiltered copies of the texture at half, quarter, … resolution, and pick the level whose texels match the pixel's footprint. A rasterizer knows the footprint for free, from the differences between neighbouring pixels. A ray tracer must compute it, and we do so with *ray differentials*: we ask where the rays through the next pixel to the right and the next pixel below would meet the plane of the triangle, and how far apart in texture space those points are from the hit.

⟨Texture sampling with ray differentials 172⟩ ≡

```
// ---- textures ---------------------------------------------------------------
// Samples a texture at the current hit, flipping USD's bottom-left st origin
// to the image's top-left row order. For camera rays the mip level comes from
// ray differentials: the rays through the neighbouring pixels are intersected
// with the triangle's plane, and the offsets are converted to st gradients.
// Shadow and reflection rays sample the finest level.
static __forceinline__ __device__ float4 sampleTexture(int texture, unsigned int prim)
{
    ⟨Interpolate the texture coordinates at the hit 173⟩
    float2 dx = make_float2(0.0f, 0.0f), dy = make_float2(0.0f, 0.0f);
    if (params.mipmaps ∧ ¬isShadowRay() ∧ optixGetPayload_4() ≡ 0)
    {
```

```
        ⟨Compute the st gradients of the pixel footprint 174⟩
    }
    return tex2DGrad<float4>(params.textures[texture], s, 1.0f − t, dx, dy);
}
```

This code is used in section 163.

173. OptiX reports a triangle hit with barycentric coordinates (β, γ): the hit point is $(1-\beta-\gamma)\,a+\beta\,b+\gamma\,c$, and any quantity stored at the corners is interpolated with the same weights. The texture coordinates (s, t) are read from the per-corner array of the triangle.

The last line of ⟨Texture sampling with ray differentials 172⟩ deserves a word. USD places the origin of texture space at the bottom-left corner of the image, with t growing upward, while images are stored, and CUDA addresses them, from the top row down. Sampling at $1 - t$ reconciles the two conventions. The function *tex2DGrad* takes the screen-space derivatives of the texture coordinates; from their lengths the hardware chooses the mipmap level, and from their ratio and directions the anisotropic filter. With zero derivatives it samples the finest level, which is what happens for shadow rays, reflection rays (depth > 0), and whenever the *Texture mipmaps* switch is off. The test of *isShadowRay* must come before the payload is read, because shadow rays have no fifth payload register.

```
⟨Interpolate the texture coordinates at the hit 173⟩ ≡
    const float2 bary = optixGetTriangleBarycentrics();
    const float2* uv = params.triUVs + 3 * prim;
    const float w0 = 1.0f − bary.x − bary.y;
    const float s = w0 * uv[0].x + bary.x * uv[1].x + bary.y * uv[2].x;
    const float t = w0 * uv[0].y + bary.x * uv[1].y + bary.y * uv[2].y;
```

This code is used in section 172.

174. The computation of the gradients has three steps: gather the triangle and the hit point, build the directions of the neighbouring rays, and turn each of them into an offset in texture space.

```
⟨Compute the st gradients of the pixel footprint 174⟩ ≡
    ⟨Gather the triangle and the hit point 175⟩
    ⟨Build the rays through the neighbouring pixels 176⟩
    ⟨Turn a neighbouring ray into an st offset 177⟩
    dx = gradient(dirX);
    dy = gradient(dirY);
```

This code is used in section 172.

175. We need the triangle's corner a, its edges $e_1 = b - a$ and $e_2 = c - a$, the normal $n = e_1 \times e_2$ of its plane (its length is irrelevant here), the ray origin o and the hit point $p = o + t_{\max}\, d$, where $t_{\max}$ is the distance along the ray at which the closest hit was found.

```
⟨Gather the triangle and the hit point 175⟩ ≡
    const uint3 tri = params.indices[prim];
    const float3 a = params.vertices[tri.x];
    const float3 e1 = params.vertices[tri.y] − a, e2 = params.vertices[tri.z] − a;
    const float3 n = cross(e1, e2);
    const float3 o = optixGetWorldRayOrigin();
    const float3 p = o + optixGetWorldRayDirection() * optixGetRayTmax();
```

This code is used in section 174.

176. The rays through the neighbouring pixels are built exactly as in the ray generation program, from normalized device coordinates. One pixel is $2/\mathit{width}$ wide in those coordinates, and $2/\mathit{height}$ high; the vertical step is subtracted because the y axis of the image points down. We use the pixel centre even when the actual sample was jittered: the footprint changes very little within a pixel. The directions need not be normalized, since only the point where they meet the plane matters.

```
⟨Build the rays through the neighbouring pixels 176⟩ ≡
    const uint3 idx = optixGetLaunchIndex();
    const float nx = 2.0f * (idx.x + 0.5f) / params.width − 1.0f;
    const float ny = 1.0f − 2.0f * (idx.y + 0.5f) / params.height;
    const Camera& cam = params.camera;
    const float3 dirX = cam.w + cam.u * (nx + 2.0f / params.width) + cam.v * ny;
```

```
const float3 dirY = cam.w + cam.u * nx + cam.v * (ny − 2.0f / params.height);
```

This code is used in section 174.

177. Now the geometry. A ray $o + \tau d'$ meets the plane through p with normal n where $(o + \tau d' - p) \cdot n = 0$, that is, at

$$\tau = \frac{(p - o) \cdot n}{d' \cdot n}.$$

The offset $\Delta = o + \tau d' - p$ lies in the plane of the triangle, so it can be written as $\Delta = \beta\, e_1 + \gamma\, e_2$. Taking dot products with e_1 and e_2 gives two linear equations,

$$\begin{pmatrix} e_1 \cdot e_1 & e_1 \cdot e_2 \\ e_1 \cdot e_2 & e_2 \cdot e_2 \end{pmatrix} \begin{pmatrix} \beta \\ \gamma \end{pmatrix} = \begin{pmatrix} \Delta \cdot e_1 \\ \Delta \cdot e_2 \end{pmatrix},$$

which Cramer's rule solves with the determinant $d_{00}d_{11} - d_{01}^2$, computed once for both rays. Since texture coordinates vary linearly over the triangle, the offset in texture space is $\beta\,(uv_1 - uv_0) + \gamma\,(uv_2 - uv_0)$; its t component changes sign because we sample at $1 - t$. A ray parallel to the plane, or a degenerate triangle, gives no usable answer, and we fall back to the finest level.

⟨Turn a neighbouring ray into an st offset 177⟩ ≡

```
const float d00 = dot(e1, e1), d01 = dot(e1, e2), d11 = dot(e2, e2);
const float denom = d00 * d11 − d01 * d01;
auto gradient = [&](float3 dir) {
    const float dn = dot(dir, n);
    if (fabsf(dn) < 1e-20f ∨ fabsf(denom) < 1e-30f)
        return make_float2(0.0f, 0.0f);
    const float3 offset = o + dir * (dot(p − o, n) / dn) − p;
    const float d20 = dot(offset, e1), d21 = dot(offset, e2);
    const float b1 = (d11 * d20 − d01 * d21) / denom, b2 = (d00 * d21 − d01 * d20) / denom;
    return make_float2(b1 * (uv[1].x − uv[0].x) + b2 * (uv[2].x − uv[0].x),
                       −(b1 * (uv[1].y − uv[0].y) + b2 * (uv[2].y − uv[0].y)));   // t is flipped
};
```

This code is used in section 174.

178. Scalar inputs. A *ScalarParam* is either a constant or one channel of a texture, remapped by that channel's scale and bias. *channel* picks a component of a *float4* by number; the conditional chain compiles to a few selects, without any indexing into local memory.

⟨Reading scalar material inputs 178⟩ ≡

```
static __forceinline__ __device__ float channel(float4 v, int c)
{
    return c ≡ 0 ? v.x : c ≡ 1 ? v.y : c ≡ 2 ? v.z : v.w;
}
static __forceinline__ __device__ float evalScalar(const ScalarParam& p, unsigned int prim)
{
    if (p.texture < 0)
        return p.value;
    return channel(sampleTexture(p.texture, prim), p.channel) * p.scale + p.bias;
}
```

This code is used in section 163.

179. Normal mapping. A normal map stores, in each texel, a direction expressed in a frame attached to the surface: the first axis follows increasing s (the *tangent*), the second follows increasing t (the *bitangent*, pointing to the top of the image), and the third is the surface normal. To use it we must know that frame in world space.

⟨Normal mapping in the tangent frame 179⟩ ≡

```
// Shading normal from a tangent-space normal map. The tangent frame comes from
// the triangle's positions and st (tangent along +s, bitangent along +t, the
// image's "up"), orthonormalized around the base normal `n`.
static __forceinline__ __device__ float3 normalMapped(const MaterialData& mat, unsigned int prim, float3 n)
{
    ⟨Build the tangent frame from positions and texture coordinates 180⟩
```

⟨Decode the normal map and leave tangent space 181⟩
}

This code is used in section 163.

180. Over a triangle, position is a linear function of the texture coordinates. If T and B are the derivatives of position with respect to s and t, the edges satisfy $e_1 = \Delta s_1\, T + \Delta t_1\, B$ and $e_2 = \Delta s_2\, T + \Delta t_2\, B$, where the Δs are the differences of texture coordinates along the edges. Solving this 2×2 system gives

$$T = \frac{e_1\,\Delta t_2 - e_2\,\Delta t_1}{\delta}, \qquad B = \frac{e_2\,\Delta s_1 - e_1\,\Delta s_2}{\delta}, \qquad \delta = \Delta s_1\,\Delta t_2 - \Delta s_2\,\Delta t_1.$$

When δ vanishes the texture is degenerate on this triangle and we keep the normal we were given. Otherwise we make the frame orthonormal around the base normal n (which may be a smooth, interpolated normal rather than the flat one): the tangent loses its component along n and is normalized, and the bitangent becomes $n \times t$. That cross product fixes the handedness, so we compare it with the true B and flip it when the texture is mirrored on this triangle; without that test the bumps of a mirrored texture would appear lit from the wrong side.

⟨Build the tangent frame from positions and texture coordinates 180⟩ ≡
```
const uint3 tri = params.indices[prim];
const float3 a = params.vertices[tri.x];
const float3 e1 = params.vertices[tri.y] − a, e2 = params.vertices[tri.z] − a;
const float2* uv = params.triUVs + 3 * prim;
const float du1 = uv[1].x − uv[0].x, dv1 = uv[1].y − uv[0].y;
const float du2 = uv[2].x − uv[0].x, dv2 = uv[2].y − uv[0].y;
const float det = du1 * dv2 − du2 * dv1;
if (fabsf(det) < 1e-12f)
    return n;  // degenerate st: no usable frame
const float3 tangent = (e1 * dv2 − e2 * dv1) * (1.0f / det);
const float3 bitangent = (e2 * du1 − e1 * du2) * (1.0f / det);
const float3 t = normalize(tangent − n * dot(n, tangent));
float3 b = cross(n, t);
if (dot(b, bitangent) < 0.0f)
    b = b * -1.0f;  // mirrored st
```

This code is used in section 179.

181. The texel is sampled like any other texture, mipmaps included, and decoded with the scale and bias authored in the file, usually $2x - 1$. The decoded vector is expressed in the frame (t, b, n) and normalized: filtering averages neighbouring normals, which shortens them.

⟨Decode the normal map and leave tangent space 181⟩ ≡
```
const float4 texel = sampleTexture(mat.normalTexture, prim);
const float3 local = make_float3(texel.x * mat.normalScale.x + mat.normalBias.x,
                                 texel.y * mat.normalScale.y + mat.normalBias.y,
                                 texel.z * mat.normalScale.z + mat.normalBias.z);
return normalize(t * local.x + b * local.y + n * local.z);
```

This code is used in section 179.

182. The GGX microfacet model. A glossy surface is modelled as a myriad of tiny mirrors, the *microfacets*, whose normals scatter around the surface normal n. Light from direction l reaches the viewer in direction v only through facets whose normal is the half vector $h = (v + l)/|v + l|$, and the specular part of the reflectance (the BRDF) is

$$f_s(l, v) = \frac{D(h)\, G(l, v)\, F(v \cdot h)}{4\,(n \cdot l)\,(n \cdot v)}.$$

D is the density of facets oriented along h, G the fraction of them that are neither shadowed nor masked by others, and F the Fresnel reflectance of one facet. We use the GGX (Trowbridge–Reitz) distribution with the Smith masking function, as UsdPreviewSurface does. Its width α is the square of the authored roughness, which makes the roughness slider perceptually even. All three functions take α^2, called *a2*.

⟨The GGX microfacet model 182⟩ ≡
```
// ---- GGX microfacet model ----------------------------------------------------
```
⟨The GGX normal distribution 183⟩

⟨The Smith masking function 184⟩

⟨Schlick's Fresnel approximation 185⟩

This code is used in section 163.

183. The GGX distribution is

$$D(h) = \frac{\alpha^2}{\pi \left((n \cdot h)^2(\alpha^2 - 1) + 1\right)^2}.$$

For small α it is a sharp peak around $h = n$; for $\alpha = 1$ it is constant, $1/\pi$, over the hemisphere.

⟨The GGX normal distribution 183⟩ ≡

```
static __forceinline__ __device__ float ggxD(float nDotH, float a2)
{
    const float d = nDotH * nDotH * (a2 − 1.0f) + 1.0f;
    return a2 / (kPi * d * d);
}
```

This code is used in section 182.

184. Smith's masking function for GGX, for a direction x making the cosine $n \cdot x$ with the normal, is

$$G_1(x) = \frac{2\,(n \cdot x)}{n \cdot x + \sqrt{\alpha^2 + (1 - \alpha^2)(n \cdot x)^2}}.$$

It is 1 for directions along the normal and falls to 0 at grazing angles. We use the separable form $G(l, v) = G_1(l)\,G_1(v)$.

⟨The Smith masking function 184⟩ ≡

```
static __forceinline__ __device__ float smithG1(float nDotX, float a2)
{
    return 2.0f * nDotX / (nDotX + sqrtf(a2 + (1.0f − a2) * nDotX * nDotX));
}
```

This code is used in section 182.

185. Schlick's approximation of the Fresnel equations,

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos\theta)^5,$$

interpolates from the reflectance at normal incidence, F_0, to total reflection at grazing incidence. F_0 is a color, because metals tint their reflections. The cosine is clamped to $[0, 1]$ so that rounding errors cannot push the result outside $[F_0, 1]$.

⟨Schlick's Fresnel approximation 185⟩ ≡

```
static __forceinline__ __device__ float3 fresnel(float3 f0, float cosTheta)
{
    const float m = 1.0f − fminf(fmaxf(cosTheta, 0.0f), 1.0f);
    const float m5 = m * m * m * m * m;
    return f0 + (make_float3(1.0f, 1.0f, 1.0f) − f0) * m5;
}
```

This code is used in section 182.

186. Ray generation. OptiX launches one thread per pixel, and each runs the ray generation program. Its name, like the names of all programs, begins with a prefix that tells OptiX its role (*__raygen__*, *__miss__*, *__closesthit__*, *__anyhit__*); the rest is the name by which the host refers to it when it creates the program groups.

⟨The ray generation program 186⟩ ≡

```
// ---- programs ----------------------------------------------------------
extern "C" __global__ void __raygen__renderFrame()
{
    ⟨Seed the random generator for this pixel and frame 187⟩
    ⟨Shoot the camera ray through a jittered point of the pixel 188⟩
    ⟨Accumulate the sample and write the pixel 189⟩
```

```
}
```

This code is used in section 163.

187. Each pixel and each frame get their own seed.

```
⟨Seed the random generator for this pixel and frame 187⟩ ≡
    const uint3 idx = optixGetLaunchIndex();
    const unsigned int pixel = idx.y * params.width + idx.x;
    unsigned int seed = tea(pixel, params.frameIndex);
```

This code is used in section 186.

188. Averaging samples taken at random points of the pixel is what removes the jagged edges of the image; it is antialiasing for free. The first frame, however, aims at the exact centre of the pixel: its mesh ids serve for picking, and its image, shown while the reader drags the camera, should be stable rather than noisy. The point in the pixel is converted to normalized device coordinates, with y flipped because image rows go down, and then to a direction with the camera vectors of the shared parameters.

The ray starts at the eye with no minimum distance and practically no maximum one. The six payload variables follow the layout of ⟨Payload helpers and the sky 169⟩; *depth* starts at zero, which marks a primary ray.

```
⟨Shoot the camera ray through a jittered point of the pixel 188⟩ ≡
    // frame 0 goes through the pixel center (also used for picking), later frames jitter
    const float jx = params.frameIndex ≡ 0 ? 0.5f : rnd(seed);
    const float jy = params.frameIndex ≡ 0 ? 0.5f : rnd(seed);
    const float2 ndc = make_float2(2.0f * (idx.x + jx) / params.width − 1.0f,
                                   1.0f − 2.0f * (idx.y + jy) / params.height);
    const Camera& cam = params.camera;
    const float3 dir = normalize(cam.w + cam.u * ndc.x + cam.v * ndc.y);

    unsigned int r, g, b, meshId, depth = 0;
    optixTrace(params.handle, cam.eye, dir, 0.0f, 1e16f, 0.0f, OptixVisibilityMask(255),
               OPTIX_RAY_FLAG_NONE, RAY_TYPE_RADIANCE, RAY_TYPE_COUNT, RAY_TYPE_
                   RADIANCE,
               r, g, b, meshId, depth, seed);
```

This code is used in section 186.

189. The new sample is added to the sum kept for the pixel, together with a count in the fourth component; on frame 0 the sum is simply replaced, which is how a reset takes effect. The displayed color is the average, $\bar{c} = \sum c_i / N$, whose noise falls as $1/\sqrt{N}$.

The conversion to 8 bits is a deliberate approximation. Displays expect sRGB, whose transfer curve is close to a power of 1/2.2; we use 1/2, a square root, which is cheaper and was the look of the viewer from its very first version. Values above 1 are clamped by *__saturatef*. Finally, the mesh id is stored on frame 0 only, the frame the host reads back for picking.

```
⟨Accumulate the sample and write the pixel 189⟩ ≡
    const float4 sample = make_float4(__uint_as_float(r), __uint_as_float(g), __uint_as_float(b), 1.0f);
    float4 sum = sample;
    if (params.frameIndex > 0)
    {
        const float4 prev = params.accumBuffer[pixel];
        sum = make_float4(prev.x + sample.x, prev.y + sample.y, prev.z + sample.z, prev.w + 1.0f);
    }
    params.accumBuffer[pixel] = sum;
    const float inv = 1.0f / sum.w;
    // approximate sRGB encoding
    params.colorBuffer[pixel] = make_uchar4((unsigned char)(255.0f * __saturatef(sqrtf(sum.x * inv))),
                                            (unsigned char)(255.0f * __saturatef(sqrtf(sum.y * inv))),
                                            (unsigned char)(255.0f * __saturatef(sqrtf(sum.z * inv))), 255);
    if (params.frameIndex ≡ 0)
        params.meshIdBuffer[pixel] = meshId;
```

This code is used in section 186.

190. The miss programs. A radiance ray that escapes returns the sky in its direction and no mesh. A shadow

ray that escapes has reached the light; its caller initializes the payload to 0, so the light counts as blocked unless this program runs.

⟨The miss programs 190⟩ ≡
```
extern "C" __global__ void __miss__radiance()
{
    setColor(sky(optixGetWorldRayDirection()), kNoMesh);
}
extern "C" __global__ void __miss__shadow()
{
    optixSetPayload_0(1u);   // light is visible
}
```
This code is used in section 163.

191. Alpha cutouts. Leaves, fences and rails are often drawn as simple quads whose texture is transparent outside the shape. UsdPreviewSurface describes them with an opacity, usually the alpha channel of the diffuse texture, and an *opacityThreshold*: where the opacity falls below the threshold, the surface is not there at all. The any-hit program runs for every candidate intersection, before OptiX decides which is closest, and calls *optixIgnoreIntersection* for the transparent ones, so that the ray goes on as if the triangle did not exist.

Because the same hit-group record serves both ray types, the test applies to shadow rays too: the shadow of a fern is the shape of its leaves, not of its quads. For materials without a cutout the program returns at once; the cost of calling it at all is the price of not splitting the geometry into opaque and cutout parts.

⟨The any-hit program for alpha cutouts 191⟩ ≡
```
// Alpha cutouts (UsdPreviewSurface opacity + opacityThreshold), for camera,
// reflection and shadow rays.
extern "C" __global__ void __anyhit__cutout()
{
    const unsigned int prim = optixGetPrimitiveIndex();
    const MaterialData& mat = params.materials[params.triMaterial[prim]];
    if (mat.opacityTexture < 0)
        return;
    const float4 texel = sampleTexture(mat.opacityTexture, prim);
    if (channel(texel, mat.opacityChannel) < mat.opacityThreshold)
        optixIgnoreIntersection();
}
```
This code is used in section 163.

192. Shading. The closest-hit program is the heart of the renderer. It finds the normals at the hit, evaluates the material, and adds up three contributions: an ambient fill, the directional light, and a glossy reflection. It then hands the color, the mesh id and the advanced random seed back to the ray's caller.

⟨The closest-hit program 192⟩ ≡
```
extern "C" __global__ void __closesthit__radiance()
{
    ⟨Fetch the hit triangle and its material 193⟩
    ⟨Compute the geometric and shading normals 194⟩
    ⟨Evaluate the material at the hit 198⟩
    ⟨Add the ambient fill 202⟩
    ⟨Add the directional light 203⟩
    ⟨Add a glossy reflection 207⟩
    ⟨Return the color and the seed to the caller 213⟩
}
```
This code is used in section 163.

193. The primitive index identifies the triangle in all the per-triangle arrays. The payload tells us how deep in the recursion we are and gives us the seed.

⟨Fetch the hit triangle and its material 193⟩ ≡
```
const unsigned int prim = optixGetPrimitiveIndex();
```

```
const unsigned int depth = optixGetPayload_4();
unsigned int seed = optixGetPayload_5();
const uint3 tri = params.indices[prim];
const float3 a = params.vertices[tri.x];
const float3 b = params.vertices[tri.y];
const float3 c = params.vertices[tri.z];
const float3 rayDir = optixGetWorldRayDirection();
const unsigned int meshId = params.triMesh[prim];
const MaterialData& mat = params.materials[params.triMaterial[prim]];
```

This code is used in section 192.

194. Normals. We keep two of them. The *geometric* normal n is the true normal of the flat triangle; it decides which side of the surface we are on, where secondary rays start, and whether they would leave the surface. The *shading* normal ns is the one used for lighting; it may come from smooth or authored normals and be perturbed by a normal map, so that a mesh of flat triangles looks curved and bumpy.

```
⟨Compute the geometric and shading normals 194⟩ ≡
    // geometric normal: outward for counter-clockwise triangles (see Scene.cpp)
    float3 n = normalize(cross(b − a, c − a));
    ⟨Interpolate the shading normal from the corner normals 195⟩
    ⟨Perturb the shading normal with the normal map 196⟩
    ⟨Turn both normals towards the viewer 197⟩
```

This code is used in section 192.

195. The scene code orders the corners of every triangle counter-clockwise as seen from outside, so $(b-a)\times(c-a)$ points outward. When the triangle has shading normals, they are interpolated with the barycentric coordinates and normalized. Should they disagree with the winding, as happens with meshes whose authored normals were made for the opposite orientation, the authored normals win and the geometric normal is turned around.

```
⟨Interpolate the shading normal from the corner normals 195⟩ ≡
    // interpolated shading normal (smooth/authored normals); they win if the winding disagrees
    float3 ns = n;
    const float3* cn = params.triNormals + 3 * prim;
    if (dot(cn[0], cn[0]) > 0.0f)
    {
        const float2 bary = optixGetTriangleBarycentrics();
        ns = normalize(cn[0] * (1.0f − bary.x − bary.y) + cn[1] * bary.x + cn[2] * bary.y);
        if (dot(ns, n) < 0.0f)
            n = n * -1.0f;
    }
```

This code is used in section 194.

196. The normal map, when present and enabled, bends the shading normal; the tangent frame is built around the smooth normal, not the flat one, so that the bumps follow the curvature of the surface.

```
⟨Perturb the shading normal with the normal map 196⟩ ≡
    if (params.normalMaps ∧ mat.normalTexture ≥ 0)
        ns = normalMapped(mat, prim, ns);
```

This code is used in section 194.

197. All surfaces are two-sided: a ray that hits the back of a triangle sees it lit as if it were the front. So when the geometric normal points away from the viewer, both normals are flipped.

Even then the shading normal may face away from the viewer: an interpolated or normal-mapped normal can tilt beyond the silhouette, which would give negative cosines and black, or NaN, pixels. We nudge it towards the view direction $v = -d$ just enough to make $n_s \cdot v$ at least 10^{-3}, and keep $n_s \cdot v$, which several formulas divide by, away from zero.

```
⟨Turn both normals towards the viewer 197⟩ ≡
    if (dot(n, rayDir) > 0.0f)
    {
        n = n * -1.0f;   // two-sided
        ns = ns * -1.0f;
```

```
}
// keep the shading normal facing the viewer
const float3 v = rayDir * -1.0f;
const float facing = dot(ns, v);
if (facing < 1e-3f)
    ns = normalize(ns + v * (1e-3f - facing));
const float nDotV = fmaxf(dot(ns, v), 1e-4f);
```

This code is used in section 194.

198. The material. We need three things from it: the diffuse color, the roughness, and the specular reflectance at normal incidence, F_0.

```
⟨Evaluate the material at the hit 198⟩ ≡
    // material
    ⟨Look up the base color 199⟩
    ⟨Derive the roughness and the reflectance at normal incidence 200⟩
    ⟨Split the energy between specular and diffuse reflection 201⟩
```

This code is used in section 192.

199. The base color, or *albedo*, is the texture remapped by its scale and bias, or the constant color. When the mesh is the current selection, the albedo is blended 60% towards orange; since the metallic workflow derives the reflections of metals from the albedo, a selected metal turns orange too.

```
⟨Look up the base color 199⟩ ≡
    float3 albedo = mat.color;
    if (mat.texture ≥ 0)
    {
        const float4 texel = sampleTexture(mat.texture, prim);
        albedo = make_float3(texel.x * mat.texScale.x + mat.texBias.x, texel.y * mat.texScale.y + mat.texBias.y,
                             texel.z * mat.texScale.z + mat.texBias.z);
    }
    if (meshId ≡ params.selectedMesh)
        albedo = mix(albedo, make_float3(1.0f, 0.55f, 0.1f), 0.6f);
```

This code is used in section 198.

200. UsdPreviewSurface has two ways to describe specular reflection. In the *specular workflow*, F_0 is given directly as *specularColor*, constant or textured, and the diffuse color is the albedo. In the more common *metallic workflow*, a dielectric such as plastic reflects $F_0 = \big((\eta-1)/(\eta+1)\big)^2$, about 4% for the default index of refraction $\eta = 1.5$, without tint, while a metal reflects with the color of its albedo and has no diffuse reflection at all. A *metallic* value between 0 and 1 blends the two. Roughness and metallic are clamped to $[0, 1]$, since a texture's scale and bias could carry them outside.

```
⟨Derive the roughness and the reflectance at normal incidence 200⟩ ≡
    const float roughness = __saturatef(evalScalar(mat.roughness, prim));
    float3 f0, diffuse;
    if (mat.specularWorkflow)
    {
        f0 = mat.specColor;
        if (mat.specTexture ≥ 0)
        {
            const float4 texel = sampleTexture(mat.specTexture, prim);
            f0 = make_float3(texel.x * mat.specScale.x + mat.specBias.x, texel.y * mat.specScale.y + mat.specBias.y,
                             texel.z * mat.specScale.z + mat.specBias.z);
        }
        diffuse = albedo;
    }
    else
    {
        const float metallic = __saturatef(evalScalar(mat.metallic, prim));
        const float r0 = (mat.ior - 1.0f) / (mat.ior + 1.0f);
        f0 = mix(make_float3(r0 * r0, r0 * r0, r0 * r0), albedo, metallic);
        diffuse = albedo * (1.0f - metallic);
    }
```

This code is used in section 198.

201. Light that is reflected specularly cannot also enter the surface and be reflected diffusely. We account for that, simply, by weighting the diffuse term with $k_d = 1 - F(n \cdot v)$, the fraction not reflected at the viewing angle. This is the reason glossy objects show less of their color at grazing angles.

⟨Split the energy between specular and diffuse reflection 201⟩ ≡
```
const float3 fv = fresnel(f0, nDotV);
const float3 kd = make_float3(1.0f − fv.x, 1.0f − fv.y, 1.0f − fv.z);
```
This code is used in section 198.

202. The viewer does not compute indirect diffuse lighting, light that bounces off one surface onto another. Without it, the faces turned away from the light would be black. A constant ambient term of 0.15 and a "headlight" term that grows with $n \cdot v$ keep them readable and give a sense of shape, as in the earliest versions of the viewer.

⟨Add the ambient fill 202⟩ ≡
```
// ambient + headlight fill on the diffuse part (no indirect diffuse)
float3 color = diffuse * kd * (0.15f + 0.25f * nDotV);
```
This code is used in section 192.

203. The directional light. A directional light, like the sun, lights every point from the same direction l. The surface receives it only if l is above both the shading and the geometric horizons: with a bent shading normal alone, light could leak onto faces that actually look away from it.

⟨Add the directional light 203⟩ ≡
```
// directional light: Lambert + GGX
⟨Find the hit point and the offset for secondary rays 204⟩
const float3 l = params.lightDir;
const float nDotL = dot(ns, l);
if (nDotL > 0.0f ∧ dot(n, l) > 0.0f)
{
    ⟨Trace a shadow ray towards the light 205⟩
    if (visible)
    {
        ⟨Add the diffuse and specular reflection of the light 206⟩
    }
}
```
This code is used in section 192.

204. Secondary rays start at the hit point, but not exactly there: rounding errors would let them hit the very triangle they leave, speckling the surface with false shadows ("shadow acne"). We move the origin a little along the geometric normal, on the side the ray came from. Floating-point error grows with distance, so the offset does too, from 10^{-3} near the origin by a thousandth of the distance travelled.

⟨Find the hit point and the offset for secondary rays 204⟩ ≡
```
const float3 hitPos = optixGetWorldRayOrigin() + rayDir * optixGetRayTmax();
const float3 offset = n * (1e-3f * (1.0f + fabsf(optixGetRayTmax()) * 1e-3f));
```
This code is used in section 203.

205. The shadow ray only needs to know whether *anything* blocks the light, so it stops at the first hit it finds and runs no closest-hit program; if it escapes, the shadow miss program sets *visible*. It uses the radiance hit-group record, whose any-hit program applies the alpha cutouts, and the shadow miss program. With shadows switched off, the light is always visible.

⟨Trace a shadow ray towards the light 205⟩ ≡
```
unsigned int visible = 1u;
if (params.shadows)
{
    visible = 0u;
    optixTrace(params.handle, hitPos + offset, l, 0.0f, 1e16f, 0.0f, OptixVisibilityMask(255),
            OPTIX_RAY_FLAG_TERMINATE_ON_FIRST_HIT | OPTIX_RAY_FLAG_DISABLE_
                CLOSESTHIT,
```

RAY_TYPE_RADIANCE, *RAY_TYPE_COUNT*, *RAY_TYPE_SHADOW*, *visible*);
}

This code is used in section 203.

206. The reflected light is the BRDF times the irradiance $E\,(n \cdot l)$:

$$L = \Big(k_d\,\frac{\rho_d}{\pi} + \frac{D\,G\,F}{4\,(n\cdot l)(n\cdot v)}\Big)E\,(n\cdot l) = E\Big(k_d\,\rho_d\,\frac{n\cdot l}{\pi} + F\,\frac{D\,G}{4\,(n\cdot v)}\Big),$$

and the factor $n \cdot l$ cancels in the specular term, which *spec* holds. One detail needs care. A directional light is a point on the sphere of directions, and the highlight of such a light on a perfect mirror is an infinitely bright point of zero size: either invisible or a single white pixel. We therefore never let the GGX width for this light fall below $\alpha = 0.01$, which turns the highlight into a small, finite spot.

⟨Add the diffuse and specular reflection of the light 206⟩ ≡
```
const float3 h = normalize(v + l);
const float aLight = fmaxf(roughness * roughness, 0.01f);  // a delta light needs a minimum width
const float a2 = aLight * aLight;
const float3 f = fresnel(f0, dot(v, h));
const float spec = ggxD(fmaxf(dot(ns, h), 0.0f), a2) * smithG1(nDotV, a2) * smithG1(nDotL, a2) /
                   (4.0f * nDotV);  // BRDF * nDotL
color = color + (diffuse * kd * (nDotL / kPi) + f * spec) * kLightIntensity;
```
This code is used in section 203.

207. Glossy reflections. Light also arrives from every other direction: from the sky, and from the other objects of the scene. A smooth surface reflects it like a mirror and a rough one blurs it. Computing that blur exactly would mean integrating the BRDF against all incoming light; instead each sample of each pixel traces *one* reflection ray in a random direction, chosen in proportion to the glossy lobe, and the progressive averaging of the ray generation program does the integration over the frames.

The choice of the random direction matters a great deal. Our first attempt drew microfacet normals from D alone; the result was correct on average, but rough metals glittered with noise even after 256 samples, because the weight of each sample, $F\,G\,(v\cdot h)/\big((n\cdot h)(n\cdot v)\big)$, becomes very large at grazing angles. We now sample the distribution of *visible* normals, the facets that the viewer can actually see, following Eric Heitz's "Sampling the GGX Distribution of Visible Normals" (2018).

⟨Add a glossy reflection 207⟩ ≡
```
// glossy reflection: one direction per sample, accumulated over frames.
// Visible-normal sampling of GGX (Heitz 2018): the weight is F * G1(r) <= 1,
// which keeps rough and grazing reflections free of fireflies.
{
    ⟨Express the view direction in a frame around the shading normal 208⟩
    ⟨Sample a visible normal of the GGX distribution 209⟩
    ⟨Reflect the view direction about the sampled normal 210⟩
    if (vDotH > 0.0f ∧ nDotR > 0.0f ∧ dot(n, r) > 0.0f)
    {
        ⟨Weight the reflected light and fetch it 211⟩
    }
}
```
This code is used in section 192.

208. The sampling works in a local frame whose z axis is the shading normal. Any two unit vectors perpendicular to n_s and to each other will do; we take the cross product with the z axis of the world, or with the x axis when n_s is nearly parallel to z. A minimum width $\alpha = 10^{-3}$ keeps the formulas finite for perfect mirrors.

⟨Express the view direction in a frame around the shading normal 208⟩ ≡
```
const float alpha = fmaxf(roughness * roughness, 1e-3f), a2 = alpha * alpha;
const float3 up = fabsf(ns.z) < 0.999f ? make_float3(0.0f, 0.0f, 1.0f) : make_float3(1.0f, 0.0f, 0.0f);
const float3 tx = normalize(cross(up, ns)), ty = cross(ns, tx);
const float3 ve = make_float3(dot(v, tx), dot(v, ty), dot(v, ns));  // view in tangent space
```
This code is used in section 207.

209. Heitz's method rests on a geometric observation. Stretch the GGX surface by $1/\alpha$ horizontally and its

microfacets become those of a hemisphere; the visible facets of a hemisphere are distributed like its projected area as seen from the viewer, which is easy to sample. So we stretch the view vector into v_h, build an orthonormal basis (t_1, t_2, v_h), pick a uniform point (p_1, p_2) on the unit disk in polar coordinates, and squeeze its second coordinate towards the visible half of the disk with the factor $s = (1 + v_{h,z})/2$; the point is then lifted onto the hemisphere and the resulting normal unstretched by α. The final line takes it back from the local frame to the world.

```
⟨Sample a visible normal of the GGX distribution 209⟩ ≡
    const float3 vh = normalize(make_float3(alpha * ve.x, alpha * ve.y, ve.z));
    const float lenSq = vh.x * vh.x + vh.y * vh.y;
    const float3 t1 = lenSq > 0.0f ? make_float3(−vh.y, vh.x, 0.0f) * rsqrtf(lenSq) : make_float3(1.0f, 0.0f, 0.0f);
    const float3 t2 = cross(vh, t1);
    const float rad = sqrtf(rnd(seed)), phi = 2.0f * kPi * rnd(seed);
    const float p1 = rad * cosf(phi), s = 0.5f * (1.0f + vh.z);
    const float p2 = (1.0f − s) * sqrtf(fmaxf(0.0f, 1.0f − p1 * p1)) + s * rad * sinf(phi);
    const float3 nh = t1 * p1 + t2 * p2 + vh * sqrtf(fmaxf(0.0f, 1.0f − p1 * p1 − p2 * p2));
    const float3 hl = normalize(make_float3(alpha * nh.x, alpha * nh.y, fmaxf(nh.z, 0.0f)));
    const float3 h = tx * hl.x + ty * hl.y + ns * hl.z;
```

This code is used in section 207.

210. The reflection of v about the facet normal h is $r = 2\,(v \cdot h)\,h - v$.

```
⟨Reflect the view direction about the sampled normal 210⟩ ≡
    const float vDotH = dot(v, h);
    const float3 r = h * (2.0f * vDotH) − v;
    const float nDotR = dot(ns, r);
```

This code is used in section 207.

211. A sample is used only if the facet faces the viewer and the reflected ray leaves the surface, above both the shading and the geometric horizons; otherwise the light it would carry is simply lost, as the masking of real microfacets loses it.

Then comes the payoff of visible-normal sampling. The probability density of r is $D_v(h)/\big(4\,(v \cdot h)\big)$, where $D_v(h) = G_1(v)\,(v \cdot h)\,D(h)/(n \cdot v)$ is the density of visible normals. The Monte Carlo weight of the sample, BRDF times cosine over density, is

$$\frac{D\,G_1(v)\,G_1(r)\,F}{4\,(n \cdot v)(n \cdot r)}\,(n \cdot r) \cdot \frac{4\,(v \cdot h)\,(n \cdot v)}{G_1(v)\,(v \cdot h)\,D} = F\,G_1(r),$$

which never exceeds 1. Both D and $G_1(v)$ have cancelled.

```
⟨Weight the reflected light and fetch it 211⟩ ≡
    const float3 weight = fresnel(f0, vDotH) * smithG1(nDotR, a2);
    float3 incoming;
    if (params.reflections ∧ depth ≡ 0)
    {
        ⟨Trace the reflection ray and clamp fireflies 212⟩
    }
    else
        incoming = sky(r);
    color = color + weight * incoming;
```

This code is used in section 207.

212. Only primary rays spawn a reflection ray; the reflection rays themselves, at depth 1, take the sky in the reflected direction instead, as does every hit when the *Reflections* switch is off. This bounds the recursion: a camera ray, a reflection ray, and the shadow ray traced from the reflection hit, three levels in all, which is the maximum trace depth the host declares for the pipeline. The reflection ray carries the seed, and receives it back advanced.

One more problem remained after visible-normal sampling: rare, very bright pixels, the “fireflies”. They appeared when a rough reflection happened to land on the highlight of the light on another, smooth surface. That highlight is thousands of times brighter than its surroundings, so one such sample in 256 still shows as a white dot. Any value above 2 is displayed as white anyway, so we clamp the brightest component of the reflected light to 2, scaling the whole color to preserve its hue. The image loses nothing visible and the fireflies disappear.

⟨Trace the reflection ray and clamp fireflies 212⟩ ≡

```
unsigned int rr, rg, rb, rMesh, rDepth = depth + 1;
optixTrace(params.handle, hitPos + offset, r, 0.0f, 1e16f, 0.0f, OptixVisibilityMask(255),
        OPTIX_RAY_FLAG_NONE, RAY_TYPE_RADIANCE, RAY_TYPE_COUNT, RAY_TYPE_
            RADIANCE,
        rr, rg, rb, rMesh, rDepth, seed);
incoming = make_float3(__uint_as_float(rr), __uint_as_float(rg), __uint_as_float(rb));
// Firefly clamp: a rough reflection that lands on another surface's sharp
// highlight of the (delta) light returns huge, rare values. Anything above
// 2 already displays as white, so clamping there only removes the spikes.
const float peak = fmaxf(incoming.x, fmaxf(incoming.y, incoming.z));
if (peak > 2.0f)
    incoming = incoming * (2.0f / peak);
```

This code is used in section 211.

213. Finally the program writes the advanced seed back into the payload, so that a reflection's caller continues the random sequence, and returns the color with the id of the mesh that was hit.

⟨Return the color and the seed to the caller 213⟩ ≡

```
optixSetPayload_5(seed);
setColor(color, meshId);
```

This code is used in section 192.

The application

214. We have a scene that knows how to read a USD stage and a renderer that knows how to turn triangles into pixels. What remains is the program that a person actually sees: a window with a menu bar, a tree of prims on the left, an inspector on the right, the ray-traced image in the middle and a status line at the bottom. This part builds that program with SDL3, which gives us a window, input events and a simple 2D renderer, and Dear ImGui, an "immediate mode" user interface library in which the whole interface is described anew in every frame by ordinary function calls.

The immediate-mode style shapes the code that follows. There are no widget objects to create, connect and destroy; instead, each frame the application asks ImGui to draw, say, a menu item, and ImGui answers whether it was clicked. The state of the interface therefore lives in a handful of plain member variables of one class, *App*, and every panel is a member function that reads them and changes them.

215. The main program. The entry point is as short as it can be: it creates the application, initializes it with the command-line arguments, and runs it until the window is closed. Everything interesting happens inside *App*.

⟨`UsdOptixViewer/src/main.cpp` 215⟩ ≡

```
// USD OptiX Viewer: loads a USD stage, shows its prim hierarchy and
// properties, and ray traces its geometry with OptiX.
//
// UsdOptixViewer [file.usd]
#include "App.h"

int main(int argc, char** argv)
{
    App app;
    if (¬app.init(argc, argv))
        return 1;
    app.run();
    return 0;
}
```

This code is written to file `UsdOptixViewer/src/main.cpp`.

216. The application class. The header `App.h` declares the class. Its public face is tiny: initialization, the main loop, and one function, *requestOpen*, that other threads may call. All the rest is private, and falls into two groups: the functions that draw the parts of the interface or act on the scene, and the state they share.

⟨`UsdOptixViewer/src/App.h` 216⟩ ≡

```
// Application: SDL3 window, Dear ImGui panels (hierarchy, inspector,
// viewport) and the OptiX renderer.
#pragma once

#include <mutex>
#include <string>
#include <unordered_set>

#include <SDL3/SDL.h>

#include "OptixRenderer.h"
#include "Scene.h"

class App
{
public:
    ⟨Public interface of the application 217⟩

private:
    ⟨Private member functions of the application 218⟩

    ⟨Window and SDL objects of the application 219⟩

    ⟨Scene and renderer of the application 220⟩

    ⟨Camera and display options of the application 221⟩
```

⟨Selection and hierarchy filter state 222⟩

⟨Text input and dialog state 223⟩

⟨File requests from other threads 224⟩

};

This code is written to file `UsdOptixViewer/src/App.h`.

217. The destructor releases ImGui and SDL in the reverse order of their creation. The file dialog of SDL may run its callback on a thread of its own, which is why *requestOpen* is public and documented as thread-safe.

⟨Public interface of the application 217⟩ ≡

```
~App();
bool init(int argc, char** argv);
void run();
// Called from the SDL file dialog callback (possibly on another thread).
void requestOpen(const std::string& path);
```

This code is used in section 216.

218. The private functions follow the layout of the screen: one function per panel, plus a few actions on the camera and the selection. Each is presented below, in the section that deals with its panel.

⟨Private member functions of the application 218⟩ ≡

```
void loadFile(const std::string& path);
void processEvent(const SDL_Event& event);
void drawUi();
void drawMenuBar();
void drawHierarchy();
void drawHierarchyNode(entt::entity e);
void drawInspector();
void drawViewport();
void drawStatusBar();
void drawPopups();
void select(entt::entity e, bool reveal);
void frameBounds(const float lo[3], const float hi[3]);
void frameSelection();
void updateFilter();
void updateCamera();
void openFileDialog();
```

This code is used in section 216.

219. SDL gives us three objects: the window, a 2D renderer attached to it, and a texture that holds the latest ray-traced image. The flag *m_running* is cleared when the user asks to quit.

⟨Window and SDL objects of the application 219⟩ ≡

```
SDL_Window* m_window = nullptr;
SDL_Renderer* m_sdlRenderer = nullptr;
SDL_Texture* m_texture = nullptr;
bool m_running = true;
```

This code is used in section 216.

220. The scene and the OptiX renderer are plain members. *m_rendererOk* stays false if OptiX could not be initialized, for instance on a machine without a suitable GPU; the rest of the interface still works, so the program can at least be used to inspect a stage. *m_needsRender* is the one flag through which every part of the interface says “something that affects the image has changed”.

⟨Scene and renderer of the application 220⟩ ≡

```
Scene m_scene;
OptixRenderer m_renderer;
bool m_rendererOk = false;
bool m_needsRender = true;
```

This code is used in section 216.

221. The camera is an *orbit* camera: it always looks at a target point, from a given distance, along a direction

given by two angles, the yaw around the up axis and the pitch above the horizon. The field of view is vertical and measured in degrees. The light direction and the four switches of the View menu complete the parameters of the image.

⟨Camera and display options of the application 221⟩ ≡
```
// orbit camera
float m_target[3] = {0, 0, 0};
float m_distance = 5.0f, m_yaw = 0.6f, m_pitch = 0.4f, m_fovY = 45.0f;
float m_light[3] = {0.4f, 0.8f, 0.45f};
bool m_shadows = true;
bool m_mipmaps = true;
bool m_normalMaps = true;
bool m_reflections = true;
```
This code is used in section 216.

222. The selection is an EnTT entity, one of those created by the scene for each prim. When a prim is selected by clicking on the image, the tree must open all its ancestors so that it becomes visible; *m_reveal* holds them until the tree has been drawn once. The filter of the tree keeps the text being typed, the text for which the filter was last computed, and the resulting set of entities to show.

⟨Selection and hierarchy filter state 222⟩ ≡
```
entt::entity m_selected = entt::null;
bool m_revealSelection = false;
std::unordered_set<entt::entity> m_reveal;   // ancestors to open in the tree
char m_filter[128] = "";
std::string m_appliedFilter;
std::unordered_set<entt::entity> m_visibleByFilter;
```
This code is used in section 216.

223. ImGui's text fields edit fixed character buffers in place, hence the **char** arrays. *m_error* holds a message to be shown in a modal popup; an empty string means there is none.

⟨Text input and dialog state 223⟩ ≡
```
char m_pathInput[1024] = "";
std::string m_error;
bool m_showAbout = false;
```
This code is used in section 216.

224. A path to open, posted by the file dialog or by a dropped file, waits here until the main loop picks it up. The mutex protects it against the dialog's thread.

⟨File requests from other threads 224⟩ ≡
```
std::mutex m_pendingMutex;
std::string m_pendingOpen;
```
This code is used in section 216.

225. The implementation. The file `App.cpp` contains, after its headers and two helper functions, the member functions in the order of the life of the program: creation, the main loop, and then the panels from left to right.

⟨`UsdOptixViewer/src/App.cpp` 225⟩ ≡
```
⟨Headers of the application 226⟩

PXR_NAMESPACE_USING_DIRECTIVE

namespace {

⟨Constants of the application 227⟩

⟨Format an attribute value for the inspector 264⟩

⟨Search a name ignoring case 250⟩

}  // namespace

⟨Destroy the application 234⟩

⟨Initialize the application 228⟩
```

⟨Request a file to be opened 235⟩

⟨Load a USD file into the scene and the renderer 236⟩

⟨The main loop of the application 238⟩

⟨Handle an SDL event 239⟩

```
// ---- layout ----------------------------------------------------------
```
⟨Lay out the panels and handle shortcuts 240⟩

⟨Draw the menu bar 244⟩

⟨Show the file dialog 237⟩

```
// ---- hierarchy -------------------------------------------------------
```
⟨Recompute the hierarchy filter 249⟩

⟨Draw the hierarchy panel 248⟩

⟨Draw one node of the hierarchy 251⟩

⟨Select an entity 254⟩

```
// ---- inspector -------------------------------------------------------
```
⟨Draw the inspector 255⟩

```
// ---- viewport --------------------------------------------------------
```
⟨Send the camera and the options to the renderer 266⟩

⟨Frame a bounding box 267⟩

⟨Frame the selection 268⟩

⟨Draw the viewport 269⟩

⟨Draw the status bar 280⟩

⟨Draw the popups 281⟩

This code is written to file `UsdOptixViewer/src/App.cpp`.

226. Besides ImGui and its two back ends, the implementation needs a few pieces of OpenUSD for the inspector, and `optix.h` only for the version number that the status bar and the About box display. The macro *PXR_NAMESPACE_USING_DIRECTIVE* brings OpenUSD's versioned namespace into scope, as everywhere else in the program.

⟨Headers of the application 226⟩ ≡
```
#include "App.h"

#include <algorithm>
#include <cmath>

#include <imgui.h>
#include <imgui_impl_sdl3.h>
#include <imgui_impl_sdlrenderer3.h>
#include <pxr/base/tf/stringUtils.h>
#include <pxr/usd/usd/attribute.h>
#include <pxr/usd/usd/modelAPI.h>
#include <pxr/usd/usd/relationship.h>
#include <pxr/usd/usdGeom/metrics.h>
#include <optix.h>
```
This code is used in section 225.

227. Two constants govern the whole interface. The renderer is progressive: it adds one sample per pixel and frame and averages them. After *kMaxSamples* samples the image has converged well enough, antialiased edges and glossy reflections included, and we stop spending the GPU's time on it. At the display's refresh rate of 60 frames per second, 256 samples take a little over four seconds.

The panels are not meant to be moved, resized or folded by the user; their places are computed every frame from the size of the window (⟨Lay out the panels and handle shortcuts 240⟩). *kPanelFlags* says so, and also asks ImGui not to save their positions in a settings file.

⟨Constants of the application 227⟩ ≡
```
constexpr unsigned int kMaxSamples = 256;   // progressive samples per pixel

constexpr ImGuiWindowFlags kPanelFlags =
```

ImGuiWindowFlags_NoMove | *ImGuiWindowFlags_NoResize* | *ImGuiWindowFlags_NoCollapse* |
ImGuiWindowFlags_NoSavedSettings;

This code is used in section 225.

228. Creation and destruction. Initialization proceeds in four steps: the window, the user interface library, the ray tracer, and finally whatever the command line asks for. A failure of SDL is fatal, but a failure of OptiX is not; it is reported in a popup and the viewport stays empty.

⟨Initialize the application 228⟩ ≡
```
bool App::init(int argc, char** argv)
{
    ⟨Create the window and the SDL renderer 229⟩
    ⟨Set up Dear ImGui 230⟩
    ⟨Create the OptiX renderer 231⟩
    ⟨Parse the command line 232⟩
    ⟨Open the requested file and select the requested prim 233⟩
    return true;
}
```
This code is used in section 225.

229. On a high-density display, a window of 1600 by 960 *logical* units may cover twice as many physical pixels. SDL reports the ratio the desktop uses for its own text as the display's *content scale*; we size the window by it so that it occupies the same fraction of the screen everywhere, and ask for a framebuffer at full pixel density with *SDL_WINDOW_HIGH_PIXEL_DENSITY*.

For drawing we use SDL's own 2D renderer rather than OpenGL. The interface is just textured triangles, which *SDL_Renderer* draws with whatever back end the platform offers, and our one large image—the ray-traced view—arrives as an ordinary array of pixels in main memory (the renderer copies it there anyway, because it also reads back the mesh identifiers used for picking). A streaming SDL texture is the simplest home for such an array; sharing a buffer between CUDA and OpenGL would avoid one copy per frame, but would tie the program to one graphics API for a saving that does not matter at these resolutions. Vertical synchronization paces the main loop at the display's refresh rate, which also paces the progressive rendering.

⟨Create the window and the SDL renderer 229⟩ ≡
```
if (¬SDL_Init(SDL_INIT_VIDEO))
{
    SDL_Log("SDL_Init failed: %s", SDL_GetError());
    return false;
}
const float scale = SDL_GetDisplayContentScale(SDL_GetPrimaryDisplay());
m_window = SDL_CreateWindow("USD OptiX Viewer", int(1600 * scale), int(960 * scale),
                            SDL_WINDOW_RESIZABLE | SDL_WINDOW_HIGH_PIXEL_DENSITY);
if (¬m_window)
{
    SDL_Log("SDL_CreateWindow failed: %s", SDL_GetError());
    return false;
}
m_sdlRenderer = SDL_CreateRenderer(m_window, nullptr);
if (¬m_sdlRenderer)
{
    SDL_Log("SDL_CreateRenderer failed: %s", SDL_GetError());
    return false;
}
SDL_SetRenderVSync(m_sdlRenderer, 1);
```
This code is used in section 228.

230. ImGui is set up as in its SDL3 example, with three changes. Keyboard navigation is enabled; the settings file `imgui.ini` is disabled (*IniFilename* set to null), since our panels have fixed places and there is nothing worth remembering; and window corners are square, because the panels tile the window. The same content scale enlarges all sizes and fonts, so that the interface is as legible on a high-density screen as on an ordinary one. The two back ends connect ImGui to SDL's events and to SDL's renderer.

⟨Set up Dear ImGui 230⟩ ≡
```
IMGUI_CHECKVERSION();
ImGui::CreateContext();
ImGuiIO& io = ImGui::GetIO();
io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard;
io.IniFilename = nullptr;
ImGui::StyleColorsDark();
ImGuiStyle& style = ImGui::GetStyle();
style.ScaleAllSizes(scale);
style.FontScaleDpi = scale;
style.WindowRounding = 0.0f;
ImGui_ImplSDL3_InitForSDLRenderer(m_window, m_sdlRenderer);
ImGui_ImplSDLRenderer3_Init(m_sdlRenderer);
```
This code is used in section 228.

231. If OptiX cannot start, the message goes into *m_error*, which the next frame shows in a popup (⟨Draw the popups 281⟩).

⟨Create the OptiX renderer 231⟩ ≡
```
std::string error;
m_rendererOk = m_renderer.init(error);
if (¬m_rendererOk)
    m_error = "OptiX initialization failed: " + error;
```
This code is used in section 228.

232. The command line names at most one file, and may add options. Three of them switch off a feature of the renderer from the start, exactly as the corresponding items of the View menu do; they make it easy to compare images with and without mipmaps, normal maps or traced reflections.

The fourth, `--select`, selects a prim by its path and frames the camera on it. It exists mostly for testing: while this program was developed, synthetic mouse clicks sent to its window by `xdotool` never reached SDL on the development machine's display, so the only way to check the selection, the tree and the inspector from a script was to select a prim from the command line and take a screenshot.

⟨Parse the command line 232⟩ ≡
```
// UsdOptixViewer [file.usd] [--select /Prim/Path] [--no-mipmaps] [--no-normal-maps] [--no-reflections]
std::string file, selectPath;
for (int i = 1; i < argc; ++i)
{
    const std::string arg = argv[i];
    if (arg ≡ "--select" ∧ i + 1 < argc)
        selectPath = argv[++i];
    else if (arg ≡ "--no-mipmaps")
        m_mipmaps = false;
    else if (arg ≡ "--no-normal-maps")
        m_normalMaps = false;
    else if (arg ≡ "--no-reflections")
        m_reflections = false;
    else
        file = arg;
}
```
This code is used in section 228.

233. The prim to select is looked up among the scene's entities by its USD path, after checking that the string is a valid path at all (the constructor of *SdfPath* would complain loudly otherwise). Selecting with *reveal* set opens its ancestors in the tree, and *frameSelection* points the camera at it.

⟨Open the requested file and select the requested prim 233⟩ ≡
```
if (¬file.empty())
    loadFile(file);
if (¬selectPath.empty() ∧ m_scene.loaded() ∧ SdfPath::IsValidPathString(selectPath))
{
    const SdfPath path(selectPath);
    for (auto [e, info] : m_scene.registry().view<PrimInfo>().each())
```

```
        if (info.path ≡ path)
        {
            select(e, true);
            frameSelection();
        }
    }
```

This code is used in section 228.

234. Destruction undoes initialization in reverse. ImGui is shut down only if it was created, because *init* may have failed before reaching that point. The scene and the OptiX renderer are members and clean up after themselves when the object's lifetime ends.

```
⟨Destroy the application 234⟩ ≡
    App::~App()
    {
        if (ImGui::GetCurrentContext())
        {
            ImGui_ImplSDLRenderer3_Shutdown();
            ImGui_ImplSDL3_Shutdown();
            ImGui::DestroyContext();
        }
        if (m_texture)
            SDL_DestroyTexture(m_texture);
        if (m_sdlRenderer)
            SDL_DestroyRenderer(m_sdlRenderer);
        if (m_window)
            SDL_DestroyWindow(m_window);
        SDL_Quit();
    }
```

This code is used in section 225.

235. Opening files. A file may be requested in four ways: on the command line, through the file dialog, by typing a path in the File menu, or by dropping it on the window. All but the first go through *requestOpen*, which merely records the path; the main loop loads it at a quiet moment, between two frames. This matters for the file dialog: SDL shows it asynchronously and may call us back on another thread, and loading a stage there would race with the drawing of the interface. A mutex makes the hand-over safe. If two requests arrive in the same frame, the later one wins, which is what a user who changes their mind expects.

```
⟨Request a file to be opened 235⟩ ≡
    void App::requestOpen(const std::string& path)
    {
        std::lock_guard<std::mutex> lock(m_pendingMutex);
        m_pendingOpen = path;
    }
```

This code is used in section 225.

236. Loading forgets everything that belonged to the previous stage: the selection, the entities to reveal and the filter of the tree. If the scene cannot open the file, its message is shown in the error popup. Otherwise the path is copied into the text field of the File menu (so that it can be edited and reloaded), the window title names the file, the flattened geometry is sent to the renderer, and the camera frames the whole stage.

```
⟨Load a USD file into the scene and the renderer 236⟩ ≡
    void App::loadFile(const std::string& path)
    {
        std::string error;
        m_selected = entt::null;
        m_reveal.clear();
        m_appliedFilter.clear();
        m_filter[0] = '\0';
        if (¬m_scene.load(path, error))
        {
            m_error = error;
            return;
```

```
    }
    SDL_strlcpy(m_pathInput, path.c_str(), sizeof(m_pathInput));
    SDL_SetWindowTitle(m_window, ("USD OptiX Viewer - " + path).c_str());
    if (m_rendererOk)
    {
        m_renderer.setGeometry(m_scene.geometry());
        m_renderer.setSelectedMesh(kNoMesh);
    }
    const FlatGeometry& g = m_scene.geometry();
    frameBounds(g.boundsMin, g.boundsMax);
}
```

This code is used in section 225.

237. The dialog is SDL's portable one; it uses the desktop's native dialog where there is one. Its callback receives a list of chosen files, an empty list if the user cancelled, or a null pointer if something went wrong. The filter list offers the four USD extensions first.

⟨Show the file dialog 237⟩ ≡

```
void App::openFileDialog()
{
    static const SDL_DialogFileFilter filters[] = {{"USD files", "usd;usda;usdc;usdz"}, {"All files", "*"}};
    SDL_ShowOpenFileDialog(
        [](void* userdata, const char* const* files, int) {
            if (files ∧ files[0])
                static_cast<App*>(userdata)→requestOpen(files[0]);
            else if (¬files)
                SDL_Log("File dialog error: %s", SDL_GetError());
        },
        this, m_window, filters, 2, nullptr, false);
}
```

This code is used in section 225.

238. The main loop. Each turn of the loop handles the pending events, loads a requested file if there is one, and then draws one frame. The pending path is taken out under the lock by swapping it with an empty string, and loaded after the lock has been released, so that the dialog's thread never waits for a stage to load. While the window is minimized we draw nothing and sleep a little, rather than spin.

Drawing a frame is ImGui's usual ritual: start a new frame in both back ends and in ImGui, describe the whole interface (⟨Lay out the panels and handle shortcuts 240⟩), let ImGui turn the description into triangles, and have SDL draw them. On a high-density display, ImGui works in logical units and SDL must scale its drawing by the framebuffer scale to match.

⟨The main loop of the application 238⟩ ≡

```
void App::run()
{
    while (m_running)
    {
        SDL_Event event;
        while (SDL_PollEvent(&event))
            processEvent(event);

        std::string pending;
        {
            std::lock_guard<std::mutex> lock(m_pendingMutex);
            pending.swap(m_pendingOpen);
        }
        if (¬pending.empty())
            loadFile(pending);

        if (SDL_GetWindowFlags(m_window) & SDL_WINDOW_MINIMIZED)
        {
            SDL_Delay(10);
            continue;
        }
```

```
        ImGui_ImplSDLRenderer3_NewFrame();
        ImGui_ImplSDL3_NewFrame();
        ImGui::NewFrame();
        drawUi();
        ImGui::Render();
        const ImGuiIO& io = ImGui::GetIO();
        SDL_SetRenderScale(m_sdlRenderer, io.DisplayFramebufferScale.x, io.DisplayFramebufferScale.y);
        SDL_SetRenderDrawColorFloat(m_sdlRenderer, 0.08f, 0.08f, 0.09f, 1.0f);
        SDL_RenderClear(m_sdlRenderer);
        ImGui_ImplSDLRenderer3_RenderDrawData(ImGui::GetDrawData(), m_sdlRenderer);
        SDL_RenderPresent(m_sdlRenderer);
    }
}
```

This code is used in section 225.

239. Every event goes first to ImGui, which keeps its own record of the mouse and the keyboard. Only three events concern the application itself: quitting, closing the window, and a file dropped on it. A dropped file takes the same road as a file chosen in the dialog.

⟨Handle an SDL event 239⟩ ≡

```
void App::processEvent(const SDL_Event& event)
{
    ImGui_ImplSDL3_ProcessEvent(&event);
    switch (event.type)
    {
    case SDL_EVENT_QUIT:
        m_running = false;
        break;
    case SDL_EVENT_WINDOW_CLOSE_REQUESTED:
        if (event.window.windowID ≡ SDL_GetWindowID(m_window))
            m_running = false;
        break;
    case SDL_EVENT_DROP_FILE:
        if (event.drop.data)
            requestOpen(event.drop.data);
        break;
    default:
        break;
    }
}
```

This code is used in section 225.

240. The layout. The window is divided into fixed regions. The menu bar sits on top, and ImGui reports the rest of the window as the main viewport's *work area*, whose top edge *WorkPos.y* is just below the menu. A status bar one line high takes the bottom. Between them, the hierarchy takes 22% of the width (but at least 260 units), the inspector 28% (at least 320), and the viewport the rest. Each panel's position and size are set just before it is drawn.

⟨Lay out the panels and handle shortcuts 240⟩ ≡

```
void App::drawUi()
{
    drawMenuBar();
    ⟨Compute the regions of the window 241⟩
    ⟨Place and draw the four panels 242⟩
    drawPopups();
    ⟨Handle the keyboard shortcuts 243⟩
}
```

This code is used in section 225.

241.

⟨Compute the regions of the window 241⟩ ≡
```
const ImGuiViewport* vp = ImGui::GetMainViewport();
const float statusH = ImGui::GetFrameHeight() + ImGui::GetStyle().WindowPadding.y;
const float top = vp→WorkPos.y, bottom = vp→Pos.y + vp→Size.y − statusH;
const float leftW = std::max(260.0f, vp→Size.x * 0.22f), rightW = std::max(320.0f, vp→Size.x * 0.28f);
```
This code is used in section 240.

242.

⟨Place and draw the four panels 242⟩ ≡
```
ImGui::SetNextWindowPos(ImVec2(vp→Pos.x, top));
ImGui::SetNextWindowSize(ImVec2(leftW, bottom − top));
drawHierarchy();

ImGui::SetNextWindowPos(ImVec2(vp→Pos.x + vp→Size.x − rightW, top));
ImGui::SetNextWindowSize(ImVec2(rightW, bottom − top));
drawInspector();

ImGui::SetNextWindowPos(ImVec2(vp→Pos.x + leftW, top));
ImGui::SetNextWindowSize(ImVec2(vp→Size.x − leftW − rightW, bottom − top));
drawViewport();

ImGui::SetNextWindowPos(ImVec2(vp→Pos.x, bottom));
ImGui::SetNextWindowSize(ImVec2(vp→Size.x, statusH));
drawStatusBar();
```
This code is used in section 240.

243. Three keys act on the whole application: F frames the selection (or the whole stage when nothing is selected), Esc clears the selection, and Ctrl+O opens the file dialog. They must not fire while the user is typing in a text field—typing “foo” in the filter of the tree should not move the camera—so ImGui’s *WantTextInput* guards them.

⟨Handle the keyboard shortcuts 243⟩ ≡
```
if (¬ImGui::GetIO().WantTextInput)
{
    if (ImGui::IsKeyPressed(ImGuiKey_F))
        frameSelection();
    if (ImGui::IsKeyPressed(ImGuiKey_Escape))
        select(entt::null, false);
    if (ImGui::IsKeyDown(ImGuiMod_Ctrl) ∧ ImGui::IsKeyPressed(ImGuiKey_O))
        openFileDialog();
}
```
This code is used in section 240.

244. Menus. The menu bar has three menus.

⟨Draw the menu bar 244⟩ ≡
```
void App::drawMenuBar()
{
    if (¬ImGui::BeginMainMenuBar())
        return;
    ⟨The File menu 245⟩
    ⟨The View menu 246⟩
    ⟨The Help menu 247⟩
    ImGui::EndMainMenuBar();
}
```
This code is used in section 225.

245. The File menu offers the dialog, a submenu with a text field in which a path can be typed or pasted (useful when no desktop dialog is available), reloading the current file, and quitting. The text field returns true when Enter is pressed, which counts as pressing Load. Reloading reads the file again from disk, which is handy while editing a `.usda` file in another window.

⟨The File menu 245⟩ ≡
```
if (ImGui::BeginMenu("File"))
```

```
{
    if (ImGui::MenuItem("Open...", "Ctrl+O"))
        openFileDialog();
    if (ImGui::BeginMenu("Open path"))
    {
        ImGui::SetNextItemWidth(420.0f);
        const bool enter = ImGui::InputText("##path", m_pathInput, sizeof(m_pathInput),
                                            ImGuiInputTextFlags_EnterReturnsTrue);
        ImGui::SameLine();
        if ((ImGui::Button("Load") ∨ enter) ∧ m_pathInput[0])
            requestOpen(m_pathInput);
        ImGui::EndMenu();
    }
    if (ImGui::MenuItem("Reload", nullptr, false, m_scene.loaded()))
        requestOpen(m_scene.filePath());
    ImGui::Separator();
    if (ImGui::MenuItem("Quit"))
        m_running = false;
    ImGui::EndMenu();
}
```

This code is used in section 244.

246. The View menu frames the stage or the selection and controls the image: shadows, texture mipmaps, normal maps, traced reflections, the direction of the light and the field of view. ImGui's check-mark items toggle the **bool** they are given and report the change, and every change sets *m_needsRender*, so that the progressive rendering starts over with the new settings.

⟨The View menu 246⟩ ≡

```
if (ImGui::BeginMenu("View"))
{
    if (ImGui::MenuItem("Frame all"))
    {
        const FlatGeometry& g = m_scene.geometry();
        frameBounds(g.boundsMin, g.boundsMax);
    }
    if (ImGui::MenuItem("Frame selection", "F", false, m_selected ≠ entt::null))
        frameSelection();
    if (ImGui::MenuItem("Shadows", nullptr, &m_shadows))
        m_needsRender = true;
    if (ImGui::MenuItem("Texture mipmaps", nullptr, &m_mipmaps))
        m_needsRender = true;
    if (ImGui::MenuItem("Normal maps", nullptr, &m_normalMaps))
        m_needsRender = true;
    if (ImGui::MenuItem("Reflections", nullptr, &m_reflections))
        m_needsRender = true;
    if (ImGui::SliderFloat3("Light direction", m_light, -1.0f, 1.0f))
        m_needsRender = true;
    if (ImGui::SliderFloat("Field of view", &m_fovY, 15.0f, 100.0f))
        m_needsRender = true;
    ImGui::EndMenu();
}
```

This code is used in section 244.

247. The About box cannot be opened from inside the menu, because ImGui's popups are identified by the stack of windows that opens them; the menu only raises a flag, and ⟨Draw the popups 281⟩ opens the box at the top level.

⟨The Help menu 247⟩ ≡

```
if (ImGui::BeginMenu("Help"))
{
    if (ImGui::MenuItem("About"))
        m_showAbout = true;
    ImGui::EndMenu();
```

```
}
```

This code is used in section 244.

248. The hierarchy. The left panel shows the prims of the stage as a tree. The scene has already mirrored the stage into the EnTT registry: every prim is an entity with a *PrimInfo* component (path, name, type) and a *Hierarchy* component (parent and children). The tree is drawn by a recursive function, starting from the children of the pseudo-root, the invisible prim at the top of every USD stage.

Above the tree, a text field filters the prims by name. The filter is recomputed only when its text changes (⟨Recompute the hierarchy filter 249⟩), not in every frame.

```
⟨Draw the hierarchy panel 248⟩ ≡
    void App::drawHierarchy()
    {
        ImGui::Begin("Hierarchy", nullptr, kPanelFlags);
        if (¬m_scene.loaded())
        {
            ImGui::TextWrapped("No stage loaded.\n\nUse File > Open, drop a .usd/.usda/.usdc/.usdz file
                    on the "
                                "window, or pass a file on the command line.");
            ImGui::End();
            return;
        }
        ImGui::SetNextItemWidth(−FLT_MIN);
        ImGui::InputTextWithHint("##filter", "Filter by name", m_filter, sizeof(m_filter));
        updateFilter();
        ImGui::Separator();
        ImGui::BeginChild("tree");
        for (entt::entity child : m_scene.registry().get<Hierarchy>(m_scene.root()).children)
            drawHierarchyNode(child);
        m_revealSelection = false;
        ImGui::EndChild();
        ImGui::End();
    }
```

This code is used in section 225.

249. A filtered tree must show not only the prims whose names match, but also every ancestor of each match; otherwise the matches would have no branch to hang from. So for each matching entity we walk up the parents, inserting each one in the set. The walk stops at the root, or as soon as an insertion fails: an entity already in the set has had its own ancestors inserted before, by the same walk, so there is no point in climbing further. The total work is therefore proportional to the number of prims, however many of them match.

```
⟨Recompute the hierarchy filter 249⟩ ≡
    void App::updateFilter()
    {
        if (m_appliedFilter ≡ m_filter)
            return;
        m_appliedFilter = m_filter;
        m_visibleByFilter.clear();
        if (m_appliedFilter.empty())
            return;
        entt::registry& reg = m_scene.registry();
        for (auto [e, info] : reg.view<PrimInfo>().each())
        {
            if (¬containsCaseInsensitive(info.name, m_appliedFilter))
                continue;
            for (entt::entity a = e; a ≠ entt::null ∧ m_visibleByFilter.insert(a).second;)
                a = reg.get<Hierarchy>(a).parent;
        }
    }
```

This code is used in section 225.

250. Names are compared without regard to case, which is what one expects of a search field. The standard library has no such function, but *std*::*search* with a predicate that compares lowered characters does the job. (The cast to **unsigned char** keeps *std*::*tolower* defined for bytes above 127.)

```
⟨Search a name ignoring case 250⟩ ≡
    bool containsCaseInsensitive(const std::string& text, const std::string& pattern)
    {
        return std::search(text.begin(), text.end(), pattern.begin(), pattern.end(), [](char a, char b) {
                    return std::tolower(static_cast<unsigned char>(a)) ≡ std::tolower(static_cast<unsigned
                            char>(b));
                }) ≠ text.end();
    }
```

This code is used in section 225.

251. Drawing a node. A node is skipped if a filter is active and the node is not in its set. Otherwise we compute the flags of ImGui's tree node: it opens only with its arrow or a double click (so that a single click on the label selects), it spans the width of the panel, it is a leaf if it has no children, and it is highlighted if it is the selection.

```
⟨Draw one node of the hierarchy 251⟩ ≡
    void App::drawHierarchyNode(entt::entity e)
    {
        if (¬m_appliedFilter.empty() ∧ ¬m_visibleByFilter.contains(e))
            return;
        entt::registry& reg = m_scene.registry();
        const PrimInfo& info = reg.get<PrimInfo>(e);
        const Hierarchy& h = reg.get<Hierarchy>(e);

        ⟨Choose the flags of the tree node 252⟩

        ⟨Draw the tree node and react to clicks 253⟩

        if (open ∧ ¬h.children.empty())
        {
            for (entt::entity child : h.children)
                drawHierarchyNode(child);
            ImGui::TreePop();
        }
    }
```

This code is used in section 225.

252. Leaves get *ImGuiTreeNodeFlags_NoTreePushOnOpen*, so that they need no matching *TreePop*; that is why the recursion above pops only nodes with children.

Two situations open a node from the program rather than by the user's hand. When the selection must be revealed, every ancestor of the selected prim is forced open, once. When a filter is active, every visible node is opened when it first appears, so that all matches can be seen; the condition *ImGuiCond_Appearing* still lets the user fold a branch afterwards.

```
⟨Choose the flags of the tree node 252⟩ ≡
    ImGuiTreeNodeFlags flags = ImGuiTreeNodeFlags_OpenOnArrow | ImGuiTreeNodeFlags_OpenOnDoubleClick |
                               ImGuiTreeNodeFlags_SpanAvailWidth;
    if (h.children.empty())
        flags |= ImGuiTreeNodeFlags_Leaf | ImGuiTreeNodeFlags_NoTreePushOnOpen;
    if (e ≡ m_selected)
        flags |= ImGuiTreeNodeFlags_Selected;
    if (m_revealSelection ∧ m_reveal.contains(e))
        ImGui::SetNextItemOpen(true);
    if (¬m_appliedFilter.empty())
        ImGui::SetNextItemOpen(true, ImGuiCond_Appearing);
```

This code is used in section 251.

253. Prims that produced triangles for the renderer are drawn in light blue, so that the geometry stands out among transforms, scopes and materials. ImGui identifies tree nodes by an ID; prim names are not unique (many prims are called "Mesh"), but the entity is, so we pass its integer value as a pointer ID and the name only as the label.

A click on the label selects the prim, while a click on the arrow merely opens or closes the node; the function *IsItemToggledOpen* tells the two apart. When the selection is being revealed, the function *SetScrollHereY* scrolls the panel to the selected node. The type of the prim follows its name, in grey.

⟨Draw the tree node and react to clicks 253⟩ ≡

```
const bool hasMesh = reg.all_of<MeshGeometry>(e);
if (hasMesh)
    ImGui::PushStyleColor(ImGuiCol_Text, ImVec4(0.6f, 0.85f, 1.0f, 1.0f));
const bool open = ImGui::TreeNodeEx(reinterpret_cast<void*>(static_cast<uintptr_t>(entt::to_
        integral(e))), flags,
                                    "%s", info.name.c_str());
if (hasMesh)
    ImGui::PopStyleColor();
if (ImGui::IsItemClicked() ∧ ¬ImGui::IsItemToggledOpen())
    select(e, false);
if (m_revealSelection ∧ e ≡ m_selected)
    ImGui::SetScrollHereY();
if (¬info.typeName.empty())
{
    ImGui::SameLine();
    ImGui::TextDisabled("%s", info.typeName.c_str());
}
```

This code is used in section 251.

254. Selecting an entity informs the renderer, which tints the selected mesh, and restarts the rendering. A selection made in the tree needs no revealing; one made by clicking on the image does, so in that case we collect the ancestors of the entity and raise *m_revealSelection*. The flag is lowered again at the end of the next drawing of the tree (⟨Draw the hierarchy panel 248⟩), so the forced opening and scrolling happen exactly once, and the user is free to fold the branches again.

⟨Select an entity 254⟩ ≡

```
void App::select(entt::entity e, bool reveal)
{
    m_selected = e;
    entt::registry& reg = m_scene.registry();
    const MeshGeometry* mesh = reg.valid(e) ? reg.try_get<MeshGeometry>(e) : nullptr;
    m_renderer.setSelectedMesh(mesh ? mesh→meshId : kNoMesh);
    m_needsRender = true;
    m_reveal.clear();
    if (reveal ∧ reg.valid(e))
    {
        for (entt::entity a = reg.get<Hierarchy>(e).parent; a ≠ entt::null; a = reg.get<Hierarchy>(a).parent)
            m_reveal.insert(a);
        m_revealSelection = true;
    }
}
```

This code is used in section 225.

255. The inspector. The right panel describes the selected prim; when nothing is selected, it describes the stage. For a prim it has three sections: the prim's identity, the geometry and material that the renderer made of it (if any), and a table of all its properties.

⟨Draw the inspector 255⟩ ≡

```
void App::drawInspector()
{
    ImGui::Begin("Inspector", nullptr, kPanelFlags);
    const UsdPrim prim = m_scene.prim(m_selected);
    if (¬prim)
    {
        ⟨Describe the stage when nothing is selected 256⟩
        ImGui::End();
        return;
    }
```

⟨Describe the identity of the prim 257⟩

⟨Describe the rendered geometry and its material 258⟩

⟨List the properties of the prim 262⟩
ImGui::*End*();
}

This code is used in section 225.

256. The stage summary gives the file, the stage's up axis and units, its time range if it is animated, its default prim, and how many materials and textures the scene found. Any warnings the scene collected while reading materials and textures (an unsupported shader, a missing file, a translucent material drawn opaque) are listed in orange; they explain why an image may differ from what another renderer would show.

⟨Describe the stage when nothing is selected 256⟩ ≡
```
if (m_scene.loaded())
{
    const UsdStageRefPtr& stage = m_scene.stage();
    ImGui::SeparatorText("Stage");
    ImGui::TextWrapped("%s", m_scene.filePath().c_str());
    ImGui::Text("Up axis: %s", m_scene.zUp() ? "Z" : "Y");
    ImGui::Text("Meters per unit: %g", UsdGeomGetStageMetersPerUnit(stage));
    if (stage→HasAuthoredTimeCodeRange())
        ImGui::Text("Time codes: %g - %g", stage→GetStartTimeCode(), stage→GetEndTimeCode());
    if (SdfPrimSpecHandle def = stage→GetRootLayer()→GetPrimAtPath(stage→GetDefaultPrim().GetPath()))
        ImGui::Text("Default prim: %s", def→GetName().c_str());
    const FlatGeometry& g = m_scene.geometry();
    ImGui::Text("Materials: %zu   Textures: %zu", g.materials.size(), g.textures.size());
    if (¬m_scene.warnings().empty())
    {
        ImGui::SeparatorText("Warnings");
        for (const std::string& w : m_scene.warnings())
            ImGui::TextColored(ImVec4(1.0f, 0.75f, 0.3f, 1.0f), "%s", w.c_str());
    }
    ImGui::Spacing();
    ImGui::TextDisabled("Select a prim in the hierarchy or click on the viewport.");
}
```

This code is used in section 255.

257. The identity of a prim is shown as a two-column table of key and value, filled by a small local function *row*. Some rows appear only when they say something: the *kind* of a model, instancing, references, applied API schemas. The specifier (def, over or class) is printed by OpenUSD as, for instance, "SdfSpecifierDef"; we strip the prefix.

⟨Describe the identity of the prim 257⟩ ≡
```
ImGui::SeparatorText("Prim");
ImGui::TextWrapped("%s", prim.GetPath().GetText());
if (ImGui::BeginTable("prim", 2, ImGuiTableFlags_SizingStretchProp))
{
    auto row = [](const char* key, const std::string& value) {
        ImGui::TableNextRow();
        ImGui::TableNextColumn();
        ImGui::TextDisabled("%s", key);
        ImGui::TableNextColumn();
        ImGui::TextWrapped("%s", value.c_str());
    };
    row("Type", prim.GetTypeName().IsEmpty() ? "(typeless)" : prim.GetTypeName().GetString());
    row("Specifier", TfStringReplace(TfStringify(prim.GetSpecifier()), "SdfSpecifier", ""));
    TfToken kind;
    if (UsdModelAPI(prim).GetKind(&kind) ∧ ¬kind.IsEmpty())
        row("Kind", kind.GetString());
    row("Active", prim.IsActive() ? "yes" : "no");
    if (prim.IsInstance() ∨ prim.IsInstanceProxy())
        row("Instancing", prim.IsInstance() ? "instance" : "instance proxy");
    if (prim.HasAuthoredReferences())
```

```
        row("Composition", "has references");
    const std::vector<std::string> schemas = [&] {
        std::vector<std::string> out;
        for (const TfToken& t : prim.GetAppliedSchemas())
            out.push_back(t.GetString());
        return out;
    }();
    if (¬schemas.empty())
        row("API schemas", TfStringJoin(schemas, ", "));
    ImGui::EndTable();
}
```

This code is used in section 255.

258. The second section exists only for prims that the scene turned into triangles, that is, entities with a *MeshGeometry* component. It shows what the *renderer* sees, after the scene has resolved the bound material, which is often more useful than the raw attributes when an image looks wrong: the number of triangles, the material's path, and each of its inputs in turn. A button frames the prim.

```
⟨Describe the rendered geometry and its material 258⟩ ≡
if (const MeshGeometry* mesh = m_scene.registry().try_get<MeshGeometry>(m_selected))
{
    ImGui::SeparatorText("Rendered geometry");
    ImGui::Text("Triangles: %u", mesh→triangleCount);
    const FlatGeometry& g = m_scene.geometry();
    const Material& material = g.materials[mesh→material];
    ImGui::TextWrapped("Material: %s", material.name.c_str());
    ⟨Describe the diffuse color or texture 259⟩
    ⟨Describe the normal map 260⟩
    ⟨Describe roughness, metallic or specular color, and ior 261⟩
    if (mesh→subsetMaterials)
        ImGui::Text("GeomSubsets with their own material: %u", mesh→subsetMaterials);
    if (ImGui::Button("Frame (F)"))
        frameSelection();
}
```

This code is used in section 255.

259. A diffuse texture is described by its resolved path (which may point inside a `.usdz` package), its size, its color space and the primvar that supplies its texture coordinates. If the prim lacks that primvar the texture cannot be applied, and we say so. Without a texture, the diffuse color is shown as a small swatch; the *ColorEdit3* widget is used only for display, with its inputs and picker disabled.

```
⟨Describe the diffuse color or texture 259⟩ ≡
if (material.texture ≥ 0)
{
    const Texture& t = g.textures[material.texture];
    ImGui::TextWrapped("Diffuse texture: %s", t.path.c_str());
    ImGui::Text("  %d x %d, %s, uv set '%s'%s", t.width, t.height, t.srgb ? "sRGB" : "raw",
                material.uvSet.c_str(), mesh→hasUVs ? "" : " (missing!)");
}
else
{
    float color[3] = {material.color[0], material.color[1], material.color[2]};
    ImGui::ColorEdit3("Diffuse color", color, ImGuiColorEditFlags_NoInputs | ImGuiColorEditFlags_NoPicker);
}
```

This code is used in section 258.

260. For a normal map, the scale and bias matter as much as the image: they decide how the stored colors become tangent-space vectors, and an asset that flips the green channel (as many DirectX-style maps do) does it through them.

```
⟨Describe the normal map 260⟩ ≡
if (material.normalTexture ≥ 0)
{
```

```
        const Texture& t = g.textures[material.normalTexture];
        ImGui::TextWrapped("Normal map: %s", t.path.c_str());
        ImGui::Text("  %d x %d, %s, scale (%g, %g, %g), bias (%g, %g, %g)", t.width, t.height,
                    t.srgb ? "sRGB" : "raw", material.normalScale[0], material.normalScale[1],
                    material.normalScale[2], material.normalBias[0], material.normalBias[1],
                    material.normalBias[2]);
    }
```

This code is used in section 258.

261. Roughness and metallic are scalar inputs that may be constants or one channel of a texture; the local function *scalarText* prints either form, naming the channel by indexing the string `"rgba"`. Materials in the specular workflow have a specular color instead of a metallic value, so we show one or the other, followed by the index of refraction.

```
⟨Describe roughness, metallic or specular color, and ior 261⟩ ≡
    auto scalarText = [&](const char* label, const ScalarInput& in) {
        if (in.texture ≥ 0)
            ImGui::TextWrapped("%s: texture channel %c of %s (x%g %+g)", label, "rgba"[in.channel],
                               g.textures[in.texture].path.c_str(), in.scale, in.bias);
        else
            ImGui::Text("%s: %g", label, in.value);
    };
    scalarText("Roughness", material.roughness);
    if (material.specularWorkflow)
    {
        if (material.specularTexture ≥ 0)
            ImGui::TextWrapped("Specular color: %s", g.textures[material.specularTexture].path.c_str());
        else
        {
            float color[3] = {material.specularColor[0], material.specularColor[1], material.specularColor[2]};
            ImGui::ColorEdit3("Specular color", color,
                              ImGuiColorEditFlags_NoInputs | ImGuiColorEditFlags_NoPicker);
        }
    }
    else
        scalarText("Metallic", material.metallic);
    ImGui::Text("IOR: %g", material.ior);
```

This code is used in section 258.

262. The last section lists every property of the prim, attributes first and then relationships, in a table of three columns: name, type and value. The table takes the rest of the panel's height and scrolls by itself, keeping its header row in view (*TableSetupScrollFreeze*).

```
⟨List the properties of the prim 262⟩ ≡
    ImGui::SeparatorText("Properties");
    if (ImGui::BeginTable("props", 3,
                          ImGuiTableFlags_RowBg | ImGuiTableFlags_BordersInnerV | ImGuiTableFlags_Resizable |
                              ImGuiTableFlags_ScrollY,
                          ImVec2(0, ImGui::GetContentRegionAvail().y)))
    {
        ImGui::TableSetupScrollFreeze(0, 1);
        ImGui::TableSetupColumn("Name", ImGuiTableColumnFlags_WidthStretch, 0.35f);
        ImGui::TableSetupColumn("Type", ImGuiTableColumnFlags_WidthStretch, 0.2f);
        ImGui::TableSetupColumn("Value", ImGuiTableColumnFlags_WidthStretch, 0.45f);
        ImGui::TableHeadersRow();
        ⟨List the attributes of the prim 263⟩
        ⟨List the relationships of the prim 265⟩
        ImGui::EndTable();
    }
```

This code is used in section 255.

263. An attribute's value is read at the default time. An attribute that has only time samples has no default value, so for it we show the value at the earliest sample, and note how many samples there are. The names

of attributes whose value was authored in the file are drawn normally; those that merely report a fallback value defined by their schema are greyed out.

⟨List the attributes of the prim 263⟩ ≡
```
for (const UsdAttribute& attr : prim.GetAttributes())
{
    VtValue value;
    attr.Get(&value, UsdTimeCode::Default());
    if (value.IsEmpty() ∧ attr.GetNumTimeSamples() > 0)
        attr.Get(&value, UsdTimeCode::EarliestTime());
    ImGui::TableNextRow();
    ImGui::TableNextColumn();
    if (attr.HasAuthoredValue())
        ImGui::TextUnformatted(attr.GetName().GetText());
    else
        ImGui::TextDisabled("%s", attr.GetName().GetText());
    ImGui::TableNextColumn();
    ImGui::TextDisabled("%s", attr.GetTypeName().GetAsToken().GetText());
    ImGui::TableNextColumn();
    ImGui::TextWrapped("%s", valueString(value).c_str());
    if (attr.GetNumTimeSamples() > 0)
    {
        ImGui::SameLine();
        ImGui::TextDisabled("(%zu samples)", attr.GetNumTimeSamples());
    }
}
```
This code is used in section 262.

264. Any USD value can be turned into text by *TfStringify*, but some values are long: the points of a mesh may run to megabytes. We cut the text at 160 characters, which is enough to recognize a value and keeps the table responsive.

⟨Format an attribute value for the inspector 264⟩ ≡
```
std::string valueString(const VtValue& value, size_t maxLength = 160)
{
    std::string s = value.IsEmpty() ? std::string("<no value>") : TfStringify(value);
    if (s.size() > maxLength)
        s = s.substr(0, maxLength) + " ...";
    return s;
}
```
This code is used in section 225.

265. A relationship has no value but a list of target paths, such as the material bound to a prim.

⟨List the relationships of the prim 265⟩ ≡
```
for (const UsdRelationship& rel : prim.GetRelationships())
{
    SdfPathVector targets;
    rel.GetTargets(&targets);
    std::vector<std::string> names;
    for (const SdfPath& p : targets)
        names.push_back(p.GetString());
    ImGui::TableNextRow();
    ImGui::TableNextColumn();
    ImGui::TextUnformatted(rel.GetName().GetText());
    ImGui::TableNextColumn();
    ImGui::TextDisabled("rel");
    ImGui::TableNextColumn();
    ImGui::TextWrapped("%s", names.empty() ? "<none>" : TfStringJoin(names, ", ").c_str());
}
```
This code is used in section 262.

266. The camera. The orbit camera is described by a target point $\mathbf{t}$, a distance d, a yaw angle θ and a

pitch angle φ. The eye is at $\mathbf{e} = \mathbf{t} + d\,\mathbf{o}$, where the unit vector $\mathbf{o}$ points from the target towards the eye. USD stages declare which axis is "up", Y or Z, and the camera must follow suit, or a Z-up scene would lie on its side.

For a Y-up stage, θ turns around Y starting from the $+Z$ axis, and φ lifts the eye above the XZ plane:

$$\mathbf{o} = (\cos\varphi \sin\theta,\ \sin\varphi,\ \cos\varphi \cos\theta).$$

For a Z-up stage, θ turns around Z starting from the $+X$ axis, and φ lifts the eye above the XY plane:

$$\mathbf{o} = (\cos\varphi \cos\theta,\ \cos\varphi \sin\theta,\ \sin\varphi).$$

In both cases the up vector given to the renderer is simply the world's up axis; the renderer builds an orthonormal frame from it and the viewing direction. This is also the place where all the settings of the View menu reach the renderer, because it is called whenever *m_needsRender* is set.

⟨Send the camera and the options to the renderer 266⟩ ≡

```
void App::updateCamera()
{
    const float cp = std::cos(m_pitch), sp = std::sin(m_pitch), cy = std::cos(m_yaw), sy = std::sin(m_yaw);
    float offset[3], up[3];
    if (m_scene.zUp())
    {
        offset[0] = cp * cy, offset[1] = cp * sy, offset[2] = sp;
        up[0] = 0, up[1] = 0, up[2] = 1;
    }
    else
    {
        offset[0] = cp * sy, offset[1] = sp, offset[2] = cp * cy;
        up[0] = 0, up[1] = 1, up[2] = 0;
    }
    float eye[3];
    for (int k = 0; k < 3; ++k)
        eye[k] = m_target[k] + offset[k] * m_distance;
    m_renderer.setCamera(eye, m_target, up, m_fovY);
    m_renderer.setLightDir(m_light);
    m_renderer.setShadows(m_shadows);
    m_renderer.setMipmaps(m_mipmaps);
    m_renderer.setNormalMaps(m_normalMaps);
    m_renderer.setReflections(m_reflections);
}
```

This code is used in section 225.

267. Framing. To frame an axis-aligned box, we aim at its center and enclose it in a sphere whose radius r is half the box's diagonal. A sphere of radius r seen from distance d subtends a half-angle α with $\sin\alpha = r/d$: the line of sight that grazes the sphere is tangent to it, and the radius to the point of tangency is perpendicular to that line. The sphere just fits the vertical field of view ϕ when $\alpha = \phi/2$, that is, when

$$d = \frac{r}{\sin(\phi/2)}.$$

We add 15% of margin. The viewport is wider than it is tall, so the horizontal direction then fits as well. A tiny lower bound on r keeps an empty or flat scene from putting the eye on the target.

⟨Frame a bounding box 267⟩ ≡

```
void App::frameBounds(const float lo[3], const float hi[3])
{
    float radius = 0.0f;
    for (int k = 0; k < 3; ++k)
    {
        m_target[k] = 0.5f * (lo[k] + hi[k]);
        radius += (hi[k] - lo[k]) * (hi[k] - lo[k]);
    }
    radius = std::max(0.5f * std::sqrt(radius), 1e-3f);
    m_distance = 1.15f * radius / std::sin(0.5f * m_fovY * float(M_PI) / 180.0f);
```

```
    m_needsRender = true;
}
```

This code is used in section 225.

268. The bounds of a selected mesh are not stored anywhere, but they are cheap to compute: the scene recorded the range of triangles each mesh contributed to the flattened geometry, so we visit their vertices, already in world space. Without a selected mesh we frame the whole stage, whose bounds the scene computed while flattening it.

```
⟨Frame the selection 268⟩ ≡
void App::frameSelection()
{
    const FlatGeometry& g = m_scene.geometry();
    const MeshGeometry* mesh =
        m_scene.registry().valid(m_selected) ? m_scene.registry().try_get<MeshGeometry>(m_selected) : nullptr;
    if (¬mesh)
    {
        frameBounds(g.boundsMin, g.boundsMax);
        return;
    }
    float lo[3] = {INFINITY, INFINITY, INFINITY}, hi[3] = {−INFINITY, −INFINITY, −INFINITY};
    for (unsigned t = mesh→firstTriangle; t < mesh→firstTriangle + mesh→triangleCount; ++t)
        for (int c = 0; c < 3; ++c)
        {
            const float* v = &g.vertices[3 * size_t(g.indices[3 * size_t(t) + c])];
            for (int k = 0; k < 3; ++k)
                lo[k] = std::min(lo[k], v[k]), hi[k] = std::max(hi[k], v[k]);
        }
    frameBounds(lo, hi);
}
```

This code is used in section 225.

269. The viewport. The middle panel shows the ray-traced image and turns mouse actions into camera motion and selection. It is also where rendering happens: the viewport knows its own size, which is the size of the image to render.

```
⟨Draw the viewport 269⟩ ≡
void App::drawViewport()
{
    ⟨Open the borderless viewport window 270⟩
    ⟨Cover the viewport with an invisible button 271⟩
    ⟨Orbit with the left mouse button 272⟩
    ⟨Pan with the right or middle mouse button 273⟩
    ⟨Zoom with the mouse wheel 274⟩
    ⟨Compute the resolution of the image 275⟩
    ⟨Pick the prim under a mouse click 276⟩
    ⟨Resize the image with the viewport 277⟩
    ⟨Restart or continue the progressive rendering 278⟩
    ⟨Show the image in the viewport 279⟩
    ImGui::End();
}
```

This code is used in section 225.

270. The viewport window has no title bar, no scroll bar and no padding, so that the image fills it to the edges. Without a working renderer, or while the window is too small to hold an image, it only shows a message (or nothing).

```
⟨Open the borderless viewport window 270⟩ ≡
ImGui::PushStyleVar(ImGuiStyleVar_WindowPadding, ImVec2(0, 0));
ImGui::Begin("Viewport", nullptr, kPanelFlags | ImGuiWindowFlags_NoScrollbar | ImGuiWindowFlags_
        NoTitleBar);
```

```
    ImGui::PopStyleVar();
    const ImVec2 size = ImGui::GetContentRegionAvail();
    const ImVec2 origin = ImGui::GetCursorScreenPos();
    if (¬m_rendererOk ∨ size.x < 4 ∨ size.y < 4)
    {
        ImGui::TextWrapped("%s", m_rendererOk ? "" : "OptiX renderer unavailable.");
        ImGui::End();
        return;
    }
```

This code is used in section 269.

271. An image drawn by ImGui does not react to the mouse, and dragging across a window's empty space would move the window. So we cover the whole viewport with an invisible button that accepts all three mouse buttons. While the button is *active*—a mouse button was pressed on it and is still held—the drag belongs to the viewport, even if the mouse leaves it.

```
⟨Cover the viewport with an invisible button 271⟩ ≡
    ImGui::InvisibleButton("viewport", size, ImGuiButtonFlags_MouseButtonLeft | ImGuiButtonFlags_
            MouseButtonRight |
                                                    ImGuiButtonFlags_MouseButtonMiddle);
    const ImGuiIO& io = ImGui::GetIO();
    const bool hovered = ImGui::IsItemHovered(), active = ImGui::IsItemActive();
```

This code is used in section 269.

272. Dragging with the left button orbits: horizontal motion changes the yaw and vertical motion the pitch, by 0.006 radians per unit, about a third of a degree. The pitch is kept within ± 1.55 radians, just short of $\pm\pi/2 \approx \pm 1.5708$: at exactly a right angle the viewing direction would be parallel to the up vector, and the camera frame built from their cross product would collapse. A drag must exceed two units before it counts, so that a click with a slightly unsteady hand still selects.

```
⟨Orbit with the left mouse button 272⟩ ≡
    // camera controls: left = orbit, right/middle = pan, wheel = zoom
    if (active ∧ ImGui::IsMouseDragging(ImGuiMouseButton_Left, 2.0f))
    {
        m_yaw −= io.MouseDelta.x * 0.006f;
        m_pitch = std::clamp(m_pitch + io.MouseDelta.y * 0.006f, -1.55f, 1.55f);
        m_needsRender = true;
    }
```

This code is used in section 269.

273. Panning moves the target in the plane of the screen, so that the point under the mouse follows the mouse. We need the camera's right and up vectors in world space. With $\mathbf{f} = -\mathbf{o}$ the viewing direction and $\mathbf{u}_w$ the world's up axis, the right vector is $\mathbf{r} \propto \mathbf{f} \times \mathbf{u}_w$ and the camera's up vector is $\mathbf{u} = \mathbf{r} \times \mathbf{f}$. Working out the products with the two formulas for $\mathbf{o}$ gives, for a Y-up stage,

$$\mathbf{r} = (\cos\theta,\ 0,\ -\sin\theta), \qquad \mathbf{u} = (-\sin\varphi\sin\theta,\ \cos\varphi,\ -\sin\varphi\cos\theta),$$

and for a Z-up stage,

$$\mathbf{r} = (-\sin\theta,\ \cos\theta,\ 0), \qquad \mathbf{u} = (-\sin\varphi\cos\theta,\ -\sin\varphi\sin\theta,\ \cos\varphi).$$

How far should the target move for each unit the mouse moves? At distance d, a vertical field of view ϕ spans a height of $2d\tan(\phi/2)$ in the world, and this height fills the H units of the viewport's height; one unit of mouse motion is therefore worth $k = 2d\tan(\phi/2)/H$ in the world at the target's depth. Moving the mouse to the right should drag the scene to the right, which means moving the target to the left, along $-\mathbf{r}$; screen coordinates grow downwards, so moving the mouse down moves the target along $+\mathbf{u}$ and the scene down.

```
⟨Pan with the right or middle mouse button 273⟩ ≡
    if (active ∧ (ImGui::IsMouseDragging(ImGuiMouseButton_Right, 1.0f) ∨
                  ImGui::IsMouseDragging(ImGuiMouseButton_Middle, 1.0f)))
    {
        // move the target in the camera plane
        const float cp = std::cos(m_pitch), sp = std::sin(m_pitch), cy = std::cos(m_yaw), sy = std::sin(m_yaw);
```

```
    float right[3], up[3];
    if (m_scene.zUp())
        right[0] = −sy, right[1] = cy, right[2] = 0, up[0] = −sp * cy, up[1] = −sp * sy, up[2] = cp;
    else
        right[0] = cy, right[1] = 0, right[2] = −sy, up[0] = −sp * sy, up[1] = cp, up[2] = −sp * cy;
    const float k = m_distance * 2.0f * std::tan(0.5f * m_fovY * float(M_PI) / 180.0f) / size.y;
    for (int i = 0; i < 3; ++i)
        m_target[i] += (io.MouseDelta.y * up[i] − io.MouseDelta.x * right[i]) * k;
    m_needsRender = true;
}
```

This code is used in section 269.

274. The wheel zooms by multiplying the distance by 0.88 per notch forward and dividing by it per notch backward. A geometric step feels the same at every scale, from a whole city down to a screw, and the distance can approach the target but never reach or cross it.

```
⟨Zoom with the mouse wheel 274⟩ ≡
    if (hovered ∧ io.MouseWheel ≠ 0.0f)
    {
        m_distance *= std::pow(0.88f, io.MouseWheel);
        m_needsRender = true;
    }
```

This code is used in section 269.

275. ImGui measures in logical units, but the image should have one ray per physical pixel. The framebuffer scale converts one into the other.

```
⟨Compute the resolution of the image 275⟩ ≡
    const float fbScale = io.DisplayFramebufferScale.x;
    const unsigned w = unsigned(size.x * fbScale), h = unsigned(size.y * fbScale);
```

This code is used in section 269.

276. Picking. Clicking on the image selects the prim under the mouse. The renderer already knows which mesh each pixel shows: on the first sample of every image, which goes through the center of each pixel, it records a mesh identifier per pixel and copies that buffer back to the CPU. Picking is therefore a lookup, with no extra ray to trace.

A click is a release of the left button that did not travel far: ImGui records the largest squared distance the mouse moved while the button was down, and we accept less than 16, that is, less than 4 units. Anything longer was an orbit, not a click. We also require that the renderer's buffer has the viewport's current width, so that pixel coordinates mean the same thing in both. The selection is revealed in the tree; a click on the background, which has no mesh, clears it.

```
⟨Pick the prim under a mouse click 276⟩ ≡
    // pick on click (left release without dragging)
    if (hovered ∧ ImGui::IsMouseReleased(ImGuiMouseButton_Left) ∧ io.MouseDragMaxDistanceSqr[0] < 16.0f ∧
        m_renderer.width() ≡ w)
    {
        const unsigned px = unsigned((io.MousePos.x − origin.x) * fbScale);
        const unsigned py = unsigned((io.MousePos.y − origin.y) * fbScale);
        const unsigned meshId = m_renderer.meshIdAt(px, py);
        const FlatGeometry& g = m_scene.geometry();
        select(meshId < g.meshEntity.size() ? g.meshEntity[meshId] : entt::null, true);
    }
```

This code is used in section 269.

277. When the viewport changes size, the renderer reallocates its buffers and we replace the SDL texture by one of the new size. The texture is a *streaming* texture, meant to be rewritten often, in the pixel format *SDL_PIXELFORMAT_RGBA32*, whose bytes are red, green, blue and alpha in memory order—exactly the layout of the renderer's *uchar4* pixels.

```
⟨Resize the image with the viewport 277⟩ ≡
    if (w ≠ m_renderer.width() ∨ h ≠ m_renderer.height())
    {
```

```
    m_renderer.resize(w, h);
    if (m_texture)
        SDL_DestroyTexture(m_texture);
    m_texture = SDL_CreateTexture(m_sdlRenderer, SDL_PIXELFORMAT_RGBA32, SDL_TEXTUREACCESS_
            STREAMING, int(w), int(h));
    m_needsRender = true;
}
```

This code is used in section 269.

278. Progressive rendering is driven from here. Any change—camera, selection, settings, a new file, a new size—has set *m_needsRender*. We then pass the new camera and options to the renderer and reset its accumulation, which throws away the samples of the old image. In every frame, as long as fewer than *kMaxSamples* samples have been taken, the renderer adds one more sample to each pixel, and the averaged image is copied into the texture. While the user drags the camera, each frame thus shows a fresh one-sample image; when the user lets go, the image refines itself in a few seconds and then the GPU rests.

A failure of the renderer, reported as an exception, disables it and shows the message in the error popup, rather than ending the program.

⟨Restart or continue the progressive rendering 278⟩ ≡

```
// any change restarts the progressive accumulation; otherwise keep adding
// samples until kMaxSamples (glossy reflections and antialiasing converge)
if (m_needsRender)
{
    updateCamera();
    m_renderer.resetAccumulation();
    m_needsRender = false;
}
if (m_texture ∧ m_rendererOk ∧ m_renderer.samples() < kMaxSamples)
{
    try
    {
        m_renderer.render();
        SDL_UpdateTexture(m_texture, nullptr, m_renderer.pixels().data(), int(w * 4));
    }
    catch (const std::exception& e)
    {
        m_error = std::string("Rendering failed: ") + e.what();
        m_rendererOk = false;
    }
}
```

This code is used in section 269.

279. Finally the texture is drawn over the invisible button. ImGui's SDL renderer back end interprets a texture identifier as a pointer to an *SDL_Texture*. If the stage has nothing the renderer can draw, a note says so in the corner, so that an empty viewport is not mistaken for a broken one.

⟨Show the image in the viewport 279⟩ ≡

```
if (m_texture)
    ImGui::GetWindowDrawList()→AddImage(ImTextureRef(ImTextureID(reinterpret_cast<intptr_t>(m_
            texture))), origin,
                                        ImVec2(origin.x + size.x, origin.y + size.y));
if (m_scene.loaded() ∧ m_scene.geometry().empty())
    ImGui::GetWindowDrawList()→AddText(ImVec2(origin.x + 12, origin.y + 12), IM_COL32(220, 220, 220, 255),
                                       "This stage has no renderable geometry (Mesh, Cube, Sphere).");
```

This code is used in section 269.

280. The status bar and the popups. The status bar is a one-line window at the bottom. It counts the prims (minus one, for the pseudo-root, which is an entity like the others but not a prim anyone wrote), the meshes, the triangles and the textures, names the OptiX version and the GPU, and shows the progress of the rendering with the time of the last sample. *OPTIX_VERSION* encodes version 9.1.0 as 90100, so we take it apart with divisions.

⟨Draw the status bar 280⟩ ≡

```
void App::drawStatusBar()
{
    ImGui::Begin("Status", nullptr, kPanelFlags | ImGuiWindowFlags_NoTitleBar | ImGuiWindowFlags_
            NoScrollbar);
    const FlatGeometry& g = m_scene.geometry();
    ImGui::Text("%s  |  prims: %zu  meshes: %zu  triangles: %zu  textures: %zu  |  OptiX %d.%d.%d
            on %s  |  "
                "samples %u/%u, %.2f ms",
                m_scene.loaded() ? m_scene.filePath().c_str() : "no file",
                m_scene.loaded() ? m_scene.registry().storage<PrimInfo>().size() − 1 : 0, g.meshEntity.size(),
                g.indices.size() / 3, g.textures.size(), OPTIX_VERSION / 10000, (OPTIX_VERSION / 100) %
                        100, OPTIX_VERSION % 100,
                m_renderer.deviceName().c_str(), m_renderer.samples(), kMaxSamples, m_
                        renderer.lastRenderMs());
    ImGui::End();
}
```

This code is used in section 225.

281. Two modal popups complete the interface. The error popup opens whenever *m_error* holds a message and stays until the user acknowledges it, which clears the message. The About box is opened here, at the top level of the window stack, when the Help menu has raised *m_showAbout*; it summarizes the tools and the mouse and keyboard controls.

⟨Draw the popups 281⟩ ≡

```
void App::drawPopups()
{
    if (¬m_error.empty() ∧ ¬ImGui::IsPopupOpen("Error"))
        ImGui::OpenPopup("Error");
    if (ImGui::BeginPopupModal("Error", nullptr, ImGuiWindowFlags_AlwaysAutoResize))
    {
        ImGui::TextUnformatted(m_error.c_str());
        if (ImGui::Button("OK"))
        {
            m_error.clear();
            ImGui::CloseCurrentPopup();
        }
        ImGui::EndPopup();
    }
    if (m_showAbout)
    {
        ImGui::OpenPopup("About");
        m_showAbout = false;
    }
    if (ImGui::BeginPopupModal("About", nullptr, ImGuiWindowFlags_AlwaysAutoResize))
    {
        ImGui::Text("USD OptiX Viewer");
        ImGui::Separator();
        ImGui::Text("OpenUSD scene loading, EnTT scene graph, OptiX %d ray tracing,", OPTIX_
                VERSION / 10000);
        ImGui::Text("SDL3 windowing and Dear ImGui %s user interface.", IMGUI_VERSION);
        ImGui::Spacing();
        ImGui::TextDisabled("Left drag: orbit   Right/middle drag: pan   Wheel: zoom");
        ImGui::TextDisabled("Click: select   F: frame selection   Esc: clear selection");
        if (ImGui::Button("Close"))
            ImGui::CloseCurrentPopup();
        ImGui::EndPopup();
    }
}
```

This code is used in section 225.

282. Decoding images. Our build of OpenUSD leaves out its imaging libraries, and with them OpenUSD's

image readers, so the scene decodes textures with Sean Barrett's `stb_image.h`, a single-header library in the public domain. Such a header contains only declarations, unless a certain macro, *STB_IMAGE_IMPLEMENTATION*, is defined before the header is included; then it contains the implementation as well. Exactly one source file of the program must do so, and this is it.

⟨`UsdOptixViewer/src/StbImage.cpp` 282⟩ ≡

```
// Image decoding for textures (PNG, JPEG, TGA, BMP, HDR, ...).
#define STB_IMAGE_IMPLEMENTATION
#include "third_party/stb_image.h"
```

This code is written to file `UsdOptixViewer/src/StbImage.cpp`.

Installation for all users

283. The tools were first installed in the home directory of one user, without administrator rights. When everything worked, a second step installed them for every user of the machine. That step needs `sudo`, so it was written as a script for the owner of the machine to run, rather than performed by the assistant that wrote the rest of the program. This part presents the script, which also documents where every tool ended up.

⟨`install_all_users.sh` 283⟩ ≡
```
#!/usr/bin/env bash
⟨Describe what the script does 284⟩
set -euo pipefail

⟨Refuse to run without root privileges 285⟩

⟨Locate the per-user installation 286⟩

⟨Install the CUDA toolkit system-wide 287⟩

mkdir -p "$DEST"

⟨Copy the OptiX headers and the Dear ImGui sources 288⟩

⟨Install SDL3 and EnTT from their build trees 289⟩

⟨Copy OpenUSD and rewrite the paths it records 290⟩

⟨Make the dynamic loader find the libraries 291⟩

⟨Set up the environment of login shells 292⟩

⟨Report the result 293⟩
```
This code is written to file `install_all_users.sh`.

284. The opening comment is the script's own summary of its effect. The option *set −euo pipefail* makes the script stop at the first failing command, at the first use of an undefined variable, and when any command of a pipeline fails.

⟨Describe what the script does 284⟩ ≡
```
# Step 3 of logbook.md: install the visualization tools for all users.
# Reuses what was downloaded/built in ~/.local/opt (step 1).
#
# sudo bash install_all_users.sh
#
# Result:
# /usr/local/cuda-13.2 (+ /usr/local/cuda symlink) CUDA toolkit
# /opt/viztools/{optix-9.1.0,sdl3,entt-4.0.0,imgui-1.93.0,usd-26.11}
# /etc/profile.d/viztools.sh environment for login shells
# /etc/ld.so.conf.d/viztools.conf runtime library paths
```
This code is used in section 283.

285.

⟨Refuse to run without root privileges 285⟩ ≡
```
if [[ $EUID -ne 0 ]]; then
    echo "Run with sudo: sudo bash $0" >&2
    exit 1
fi
```
This code is used in section 283.

286. Under `sudo`, the home directory is that of the invoking user, which `sudo` records in *SUDO_USER*. The per-user tools live in `.local/opt` below it; their downloads and build trees were kept in `.local/opt/src`, because SDL3, EnTT and CUDA are installed again from there.

⟨Locate the per-user installation 286⟩ ≡
```
USER_HOME=$(getent passwd "${SUDO_USER:-pablofigueroa}" | cut -d: -f6)
LOCAL=$USER_HOME/.local/opt
SRC=$LOCAL/src
```

DEST=/opt/viztools

This code is used in section 283.

287. CUDA is installed with NVIDIA's own installer, as the toolkit only (the driver was already there). The test makes the script safe to run twice.

⟨Install the CUDA toolkit system-wide 287⟩ ≡

```
echo "==> CUDA 13.2 toolkit -> /usr/local/cuda-13.2"
if [[ ! -x /usr/local/cuda-13.2/bin/nvcc ]]; then
    TMPDIR=$SRC bash "$SRC/cuda_13.2.1_linux.run" --nox11 -- --silent --toolkit --no-man-page --no-drm
        --override
fi
```

This code is used in section 283.

288. OptiX and Dear ImGui are plain directories of headers and sources, so a copy is all they need.

⟨Copy the OptiX headers and the Dear ImGui sources 288⟩ ≡

```
echo "==> OptiX 9.1.0 headers"
rm -rf "$DEST/optix-9.1.0"
cp -r "$LOCAL/optix-9.1.0" "$DEST/optix-9.1.0"

echo "==> Dear ImGui 1.93.0 sources + backends"
rm -rf "$DEST/imgui-1.93.0"
cp -r "$LOCAL/imgui-1.93.0" "$DEST/imgui-1.93.0"
```

This code is used in section 283.

289. SDL3 and EnTT were built with CMake, which can install a finished build tree to a new prefix. This rewrites their CMake package files with the right paths.

⟨Install SDL3 and EnTT from their build trees 289⟩ ≡

```
echo "==> SDL3 3.4.18"
cmake --install "$SRC/SDL3/build" --prefix "$DEST/sdl3" > /dev/null

echo "==> EnTT 4.0.0"
cmake --install "$SRC/entt/build" --prefix "$DEST/entt-4.0.0" > /dev/null
```

This code is used in section 283.

290. OpenUSD's own build script installed it directly, so it is copied instead. Its package file, and a few others, still name the old directory, so every text file that mentions it is edited in place. The new tree belongs to root and is readable by everyone.

⟨Copy OpenUSD and rewrite the paths it records 290⟩ ≡

```
echo "==> OpenUSD 26.11.1"
rm -rf "$DEST/usd-26.11"
cp -r "$LOCAL/usd-26.11" "$DEST/usd-26.11"

# Point text files (cmake/pkg-config) that mention the per-user location to /opt/viztools
grep -rlI "$LOCAL" "$DEST" | xargs -r sed -i "s#$LOCAL#$DEST#g"

chown -R root:root "$DEST"
chmod -R a+rX "$DEST"
```

This code is used in section 283.

291. Programs linked against CUDA, SDL3 or OpenUSD must find these shared libraries when they start. A file in `/etc/ld.so.conf.d` adds the three directories to the system's search list, and `ldconfig` rebuilds the loader's cache.

⟨Make the dynamic loader find the libraries 291⟩ ≡

```
echo "==> /etc/ld.so.conf.d/viztools.conf"
cat > /etc/ld.so.conf.d/viztools.conf <<EOF
/usr/local/cuda-13.2/lib64
$DEST/sdl3/lib
$DEST/usd-26.11/lib
EOF
ldconfig
```

This code is used in section 283.

292. Login shells read every script in `/etc/profile.d`. Ours puts `nvcc` and the USD command-line tools on the *PATH*, and sets the variables that the CMake project of the build-system part looks for. The quoted `'EOF'` keeps the dollar signs for the shell that will read the file later.

⟨Set up the environment of login shells 292⟩ ≡

```
echo "==> /etc/profile.d/viztools.sh"
cat > /etc/profile.d/viztools.sh <<'EOF'
# Visualization tools installed for all users (see /opt/viztools)
export CUDA_HOME=/usr/local/cuda-13.2
export OptiX_INSTALL_DIR=/opt/viztools/optix-9.1.0
export IMGUI_DIR=/opt/viztools/imgui-1.93.0
export PATH=$CUDA_HOME/bin:/opt/viztools/usd-26.11/bin:$PATH
export CMAKE_PREFIX_PATH=/opt/viztools/sdl3:/opt/viztools/entt-4.0.0:/opt/viztools/
       usd-26.11:$CUDA_HOME${CMAKE_PREFIX_PATH:+:$CMAKE_PREFIX_PATH}
EOF
chmod 644 /etc/profile.d/viztools.sh
```

This code is used in section 283.

293.

⟨Report the result 293⟩ ≡

```
echo "==> Done. Verifying:"
/usr/local/cuda-13.2/bin/nvcc --version | tail -1
ls "$DEST"
```

This code is used in section 283.

Names of the sections

⟨`install_all_users.sh` 283⟩ Root (written to a file).
⟨`UsdOptixViewer/cmake/EmbedFile.cmake` 16⟩ Root (written to a file).
⟨`UsdOptixViewer/CMakeLists.txt` 6⟩ Root (written to a file).
⟨`UsdOptixViewer/src/App.cpp` 225⟩ Root (written to a file).
⟨`UsdOptixViewer/src/App.h` 216⟩ Root (written to a file).
⟨`UsdOptixViewer/src/devicePrograms.cu` 163⟩ Root (written to a file).
⟨`UsdOptixViewer/src/LaunchParams.h` 17⟩ Root (written to a file).
⟨`UsdOptixViewer/src/main.cpp` 215⟩ Root (written to a file).
⟨`UsdOptixViewer/src/OptixRenderer.cpp` 109⟩ Root (written to a file).
⟨`UsdOptixViewer/src/OptixRenderer.h` 99⟩ Root (written to a file).
⟨`UsdOptixViewer/src/Scene.cpp` 38⟩ Root (written to a file).
⟨`UsdOptixViewer/src/Scene.h` 29⟩ Root (written to a file).
⟨`UsdOptixViewer/src/StbImage.cpp` 282⟩ Root (written to a file).
⟨A linear congruential generator 168⟩ Used in section 166.
⟨Accumulate the sample and write the pixel 189⟩ Used in section 186.
⟨Add a glossy reflection 207⟩ Used in section 192.
⟨Add the ambient fill 202⟩ Used in section 192.
⟨Add the diffuse and specular reflection of the light 206⟩ Used in section 203.
⟨Add the directional light 203⟩ Used in section 192.
⟨Allocate a mipmapped array for the texture 150⟩ Used in section 149.
⟨Append the material, texture coordinates and normals of each triangle 94⟩ Used in section 93.
⟨Append the triangles of a prim to the flat geometry 93⟩ Used in section 46.
⟨Assign a material to every face of a prim 90⟩ Used in section 46.
⟨Average one channel of the block 118⟩ Used in section 115.
⟨Build Dear ImGui and the viewer executable 14⟩ Used in section 6.
⟨Build the geometry acceleration structure 148⟩ Used in section 140.
⟨Build the pipeline and set the default parameters 126⟩ Used in section 123.
⟨Build the rays through the neighbouring pixels 176⟩ Used in section 174.
⟨Build the shader binding table 135⟩ Used in section 109.
⟨Build the tangent frame from positions and texture coordinates 180⟩ Used in section 179.
⟨Camera and display options of the application 221⟩ Used in section 216.
⟨Camera, light and rendering switches 27⟩ Used in section 25.
⟨Choose the acceleration structure build options 147⟩ Used in section 140.
⟨Choose the flags of the tree node 252⟩ Used in section 251.
⟨Choose the root of the installed tools and of CUDA 9⟩ Used in section 8.
⟨Close the bookkeeping of a generated primitive 59⟩ Used in section 38.
⟨Collect a warning once 42⟩ Used in section 38.
⟨Colors, textures and inputs of a material 24⟩ Used in section 22.
⟨Compile the device programs to PTX and embed them 13⟩ Used in section 6.
⟨Compile the embedded PTX into a module 130⟩ Used in section 127.
⟨Compute smooth normals for a subdivision surface 57⟩ Used in section 55.
⟨Compute the geometric and shading normals 194⟩ Used in section 192.
⟨Compute the next mip level of a texture 115⟩ Used in section 109.
⟨Compute the regions of the window 241⟩ Used in section 240.
⟨Compute the resolution of the image 275⟩ Used in section 269.
⟨Compute the shading normals of each triangle corner 55⟩ Used in section 51.
⟨Compute the st gradients of the pixel footprint 174⟩ Used in section 172.
⟨Constants of the application 227⟩ Used in section 225.
⟨Conversions from host material fields 143⟩ Used in section 142.
⟨Convert a UsdGeomMesh into a local triangle soup 51⟩ Used in section 38.

⟨Convert a UsdShade material into a Material record 72⟩ Used in section 38.
⟨Copy OpenUSD and rewrite the paths it records 290⟩ Used in section 283.
⟨Copy the OptiX headers and the Dear ImGui sources 288⟩ Used in section 283.
⟨Cover the viewport with an invisible button 271⟩ Used in section 269.
⟨Create the hit group 133⟩ Used in section 127.
⟨Create the miss program groups 132⟩ Used in section 127.
⟨Create the OptiX device context 125⟩ Used in section 123.
⟨Create the OptiX pipeline 127⟩ Used in section 109.
⟨Create the OptiX renderer 231⟩ Used in section 228.
⟨Create the ray generation program group 131⟩ Used in section 127.
⟨Create the texture object 153⟩ Used in section 149.
⟨Create the window and the SDL renderer 229⟩ Used in section 228.
⟨CUDA and OptiX handles of the renderer 106⟩ Used in section 99.
⟨Data members of the scene 36⟩ Used in section 34.
⟨Decide how to decode the normal map 83⟩ Used in section 82.
⟨Decide the color space and wrap modes, and consult the texture cache 88⟩ Used in section 86.
⟨Declare the embedded PTX 111⟩ Used in section 109.
⟨Declare the project and the language standard 7⟩ Used in section 6.
⟨Decode a texture through the asset resolver 89⟩ Used in section 86.
⟨Decode the normal map and leave tangent space 181⟩ Used in section 179.
⟨Derive the roughness and the reflectance at normal incidence 200⟩ Used in section 198.
⟨Describe how the texture is sampled 152⟩ Used in section 149.
⟨Describe roughness, metallic or specular color, and ior 261⟩ Used in section 258.
⟨Describe the diffuse color or texture 259⟩ Used in section 258.
⟨Describe the identity of the prim 257⟩ Used in section 255.
⟨Describe the normal map 260⟩ Used in section 258.
⟨Describe the rendered geometry and its material 258⟩ Used in section 255.
⟨Describe the stage when nothing is selected 256⟩ Used in section 255.
⟨Describe the triangles to OptiX 146⟩ Used in section 140.
⟨Describe what the script does 284⟩ Used in section 283.
⟨Destroy the application 234⟩ Used in section 225.
⟨Destroy the renderer 122⟩ Used in section 109.
⟨Diffuse, cutout and normal-map fields of a material 65⟩ Used in section 64.
⟨Download the image and the picking ids 161⟩ Used in section 158.
⟨Draw one node of the hierarchy 251⟩ Used in section 225.
⟨Draw the hierarchy panel 248⟩ Used in section 225.
⟨Draw the inspector 255⟩ Used in section 225.
⟨Draw the menu bar 244⟩ Used in section 225.
⟨Draw the popups 281⟩ Used in section 225.
⟨Draw the status bar 280⟩ Used in section 225.
⟨Draw the tree node and react to clicks 253⟩ Used in section 251.
⟨Draw the viewport 269⟩ Used in section 225.
⟨Error checking of CUDA and OptiX calls 112⟩ Used in section 109.
⟨Evaluate the material at the hit 198⟩ Used in section 192.
⟨Express the view direction in a frame around the shading normal 208⟩ Used in section 207.
⟨Fan-triangulate the faces of a mesh 53⟩ Used in section 51.
⟨Fetch the hit triangle and its material 193⟩ Used in section 192.
⟨File requests from other threads 224⟩ Used in section 216.
⟨Fill in one device material 144⟩ Used in section 142.
⟨Find SDL3, EnTT, OpenUSD and CUDA 12⟩ Used in section 8.
⟨Find the authored normals of a mesh 56⟩ Used in section 55.
⟨Find the Dear ImGui sources 11⟩ Used in section 8.

⟨Find the hit point and the offset for secondary rays 204⟩ Used in section 203.
⟨Find the OptiX headers 10⟩ Used in section 8.
⟨Find the prim of an entity 43⟩ Used in section 38.
⟨Find the primvar that holds the diffuse texture coordinates 79⟩ Used in section 77.
⟨Find the primvar that holds the normal-map coordinates 84⟩ Used in section 82.
⟨Find the shader that drives an input 69⟩ Used in section 38.
⟨Flatten the stage into world-space triangles 46⟩ Used in section 38.
⟨Forget the current stage 41⟩ Used in section 38.
⟨Format an attribute value for the inspector 264⟩ Used in section 225.
⟨Frame a bounding box 267⟩ Used in section 225.
⟨Frame the selection 268⟩ Used in section 225.
⟨Free a device buffer 121⟩ Used in section 109.
⟨Free the textures 154⟩ Used in section 109.
⟨Gather the 2x2 block of source texels 117⟩ Used in section 115.
⟨Gather the triangle and the hit point 175⟩ Used in section 174.
⟨Generate the triangles of a cube 60⟩ Used in section 38.
⟨Generate the triangles of a sphere 61⟩ Used in section 38.
⟨Handle an empty image or scene 159⟩ Used in section 158.
⟨Handle an SDL event 239⟩ Used in section 225.
⟨Handle the keyboard shortcuts 243⟩ Used in section 240.
⟨Headers and declarations of the renderer interface 100⟩ Used in section 99.
⟨Headers needed by the scene declarations 30⟩ Used in section 29.
⟨Headers needed by the scene implementation 39⟩ Used in section 38.
⟨Headers of the application 226⟩ Used in section 225.
⟨Headers of the renderer implementation 110⟩ Used in section 109.
⟨Host-side results of the renderer 108⟩ Used in section 99.
⟨Identify a shader and its wrap modes 70⟩ Used in section 38.
⟨Initialize CUDA and OptiX 123⟩ Used in section 109.
⟨Initialize the application 228⟩ Used in section 225.
⟨Install SDL3 and EnTT from their build trees 289⟩ Used in section 283.
⟨Install the CUDA toolkit system-wide 287⟩ Used in section 283.
⟨Install the viewer 15⟩ Used in section 6.
⟨Interpolate the shading normal from the corner normals 195⟩ Used in section 194.
⟨Interpolate the texture coordinates at the hit 173⟩ Used in section 172.
⟨Launch one sample per pixel 160⟩ Used in section 158.
⟨Lay out the panels and handle shortcuts 240⟩ Used in section 225.
⟨Link the pipeline and set its stack size 134⟩ Used in section 127.
⟨List the attributes of the prim 263⟩ Used in section 262.
⟨List the properties of the prim 262⟩ Used in section 255.
⟨List the relationships of the prim 265⟩ Used in section 262.
⟨Load a texture once 86⟩ Used in section 38.
⟨Load a USD file into the scene and the renderer 236⟩ Used in section 225.
⟨Local triangle soup of one prim 50⟩ Used in section 38.
⟨Locate the per-user installation 286⟩ Used in section 283.
⟨Locate the visualization tools 8⟩ Used in section 6.
⟨Look up the base color 199⟩ Used in section 198.
⟨Look up the mesh under a pixel 162⟩ Used in section 109.
⟨Make the dynamic loader find the libraries 291⟩ Used in section 283.
⟨Material data on the device 22⟩ Used in section 17.
⟨Material for a prim without a bound material 71⟩ Used in section 38.
⟨Mirror the prim hierarchy into the registry 44⟩ Used in section 38.
⟨Module compile options 128⟩ Used in section 127.

⟨Normal mapping in the tangent frame 179⟩ Used in section 163.
⟨Open a stage and build the scene 40⟩ Used in section 38.
⟨Open the borderless viewport window 270⟩ Used in section 269.
⟨Open the requested file and select the requested prim 233⟩ Used in section 228.
⟨Opening of the device programs file 164⟩ Used in section 163.
⟨Opening of the shared parameters header 18⟩ Used in section 17.
⟨Orbit with the left mouse button 272⟩ Used in section 269.
⟨Output buffers of a launch 26⟩ Used in section 25.
⟨Pack the hit records 138⟩ Used in section 135.
⟨Pack the materials for the device 142⟩ Used in section 140.
⟨Pack the miss records 137⟩ Used in section 135.
⟨Pack the ray generation record 136⟩ Used in section 135.
⟨Pan with the right or middle mouse button 273⟩ Used in section 269.
⟨Parse the command line 232⟩ Used in section 228.
⟨Payload helpers and the sky 169⟩ Used in section 163.
⟨Perturb the shading normal with the normal map 196⟩ Used in section 194.
⟨Pick the prim under a mouse click 276⟩ Used in section 269.
⟨Pick the shading normal of each triangle corner 58⟩ Used in section 55.
⟨Pipeline compile options 129⟩ Used in section 127.
⟨Place and draw the four panels 242⟩ Used in section 240.
⟨Point the shader binding table at the records 139⟩ Used in section 135.
⟨Prepare the flattening of the stage 47⟩ Used in section 46.
⟨Private helpers of the scene 37⟩ Used in section 34.
⟨Private member functions of the application 218⟩ Used in section 216.
⟨Private steps of the renderer 105⟩ Used in section 99.
⟨Public interface of the application 217⟩ Used in section 216.
⟨Public interface of the scene 35⟩ Used in section 34.
⟨Random numbers on the device 166⟩ Used in section 163.
⟨Read a float input that may come from a texture 85⟩ Used in section 38.
⟨Read an alpha cutout of a surface 80⟩ Used in section 72.
⟨Read the diffuse color of a surface 76⟩ Used in section 72.
⟨Read the diffuse texture of a surface 77⟩ Used in section 72.
⟨Read the normal map of a surface 82⟩ Used in section 72.
⟨Read the points and the face topology of a mesh 52⟩ Used in section 51.
⟨Read the scale, bias and fallback of the diffuse texture 78⟩ Used in section 77.
⟨Read the specular parameters of a surface 81⟩ Used in section 72.
⟨Read the texture coordinates of a prim 91⟩ Used in section 46.
⟨Read the value of a shader input 68⟩ Used in section 38.
⟨Reading scalar material inputs 178⟩ Used in section 163.
⟨Recognize shadow rays 170⟩ Used in section 169.
⟨Recompute the hierarchy filter 249⟩ Used in section 225.
⟨Record the bounds of the scene 97⟩ Used in section 46.
⟨Record the mesh in the registry 95⟩ Used in section 93.
⟨Reflect the view direction about the sampled normal 210⟩ Used in section 207.
⟨Refuse to run without root privileges 285⟩ Used in section 283.
⟨Render one progressive sample 158⟩ Used in section 109.
⟨Renderer construction and initialization 101⟩ Used in section 99.
⟨Renderer frame production and results 103⟩ Used in section 99.
⟨Renderer scene and view settings 102⟩ Used in section 99.
⟨Report the result 293⟩ Used in section 283.
⟨Request a file to be opened 235⟩ Used in section 225.
⟨Resize a device buffer 119⟩ Used in section 109.

⟨Resize the image with the viewport 277⟩ Used in section 269.
⟨Resize the output image 155⟩ Used in section 109.
⟨Resolve the file of a texture 87⟩ Used in section 86.
⟨Restart or continue the progressive rendering 278⟩ Used in section 269.
⟨Return the cached Material of a UsdShade material 73⟩ Used in section 72.
⟨Return the color and the seed to the caller 213⟩ Used in section 192.
⟨Rewind the triangles of a left-handed mesh 54⟩ Used in section 51.
⟨Sample a visible normal of the GGX distribution 209⟩ Used in section 207.
⟨Scalar material inputs 21⟩ Used in section 17.
⟨Scene and output buffers of the renderer 107⟩ Used in section 99.
⟨Scene and renderer of the application 220⟩ Used in section 216.
⟨Schlick's Fresnel approximation 185⟩ Used in section 182.
⟨Search a name ignoring case 250⟩ Used in section 225.
⟨Seed the random generator for this pixel and frame 187⟩ Used in section 186.
⟨Select an entity 254⟩ Used in section 225.
⟨Select the GPU and its primary context 124⟩ Used in section 123.
⟨Selection and hierarchy filter state 222⟩ Used in section 216.
⟨Send the camera and the options to the renderer 266⟩ Used in section 225.
⟨Set the camera 156⟩ Used in section 109.
⟨Set the light direction 157⟩ Used in section 109.
⟨Set the scene geometry 140⟩ Used in section 109.
⟨Set up Dear ImGui 230⟩ Used in section 228.
⟨Set up the environment of login shells 292⟩ Used in section 283.
⟨Shoot the camera ray through a jittered point of the pixel 188⟩ Used in section 186.
⟨Show the file dialog 237⟩ Used in section 225.
⟨Show the image in the viewport 279⟩ Used in section 269.
⟨Skip prims that are not visible renderable geometry 48⟩ Used in section 46.
⟨Small vector algebra on the device 165⟩ Used in section 163.
⟨Small vector helpers of the host 114⟩ Used in section 109.
⟨Specular fields of a material 66⟩ Used in section 64.
⟨Split the energy between specular and diffuse reflection 201⟩ Used in section 198.
⟨Start a Material with the UsdPreviewSurface defaults 74⟩ Used in section 72.
⟨Store a new Material in the cache 75⟩ Used in section 72.
⟨Test whether a material reads any texture 67⟩ Used in section 64.
⟨Text input and dialog state 223⟩ Used in section 216.
⟨Texture remapping coefficients of a material 23⟩ Used in section 22.
⟨Texture sampling with ray differentials 172⟩ Used in section 163.
⟨Texture wrap modes and decoded textures 62⟩ Used in section 29.
⟨The any-hit program for alpha cutouts 191⟩ Used in section 163.
⟨The background mesh id and the ray types 19⟩ Used in section 17.
⟨The camera frame 20⟩ Used in section 17.
⟨The closest-hit program 192⟩ Used in section 163.
⟨The device buffer helper 104⟩ Used in section 99.
⟨The File menu 245⟩ Used in section 244.
⟨The flattened world-space geometry 45⟩ Used in section 29.
⟨The GGX microfacet model 182⟩ Used in section 163.
⟨The GGX normal distribution 183⟩ Used in section 182.
⟨The Help menu 247⟩ Used in section 244.
⟨The Hierarchy component 32⟩ Used in section 29.
⟨The launch parameters 25⟩ Used in section 17.
⟨The main loop of the application 238⟩ Used in section 225.
⟨The Material record 64⟩ Used in section 29.

⟨The MeshGeometry component 33⟩ Used in section 29.
⟨The miss programs 190⟩ Used in section 163.
⟨The PrimInfo component 31⟩ Used in section 29.
⟨The ray generation program 186⟩ Used in section 163.
⟨The ScalarInput material parameter 63⟩ Used in section 29.
⟨The scene as seen by the device 28⟩ Used in section 25.
⟨The Scene class 34⟩ Used in section 29.
⟨The shader binding table record type 113⟩ Used in section 109.
⟨The sky 171⟩ Used in section 169.
⟨The Smith masking function 184⟩ Used in section 182.
⟨The sRGB to linear lookup table 116⟩ Used in section 115.
⟨The TEA hash 167⟩ Used in section 166.
⟨The View menu 246⟩ Used in section 244.
⟨Trace a shadow ray towards the light 205⟩ Used in section 203.
⟨Trace the reflection ray and clamp fireflies 212⟩ Used in section 211.
⟨Transform the points of a prim to world space 92⟩ Used in section 46.
⟨Triangulate a prim in its local space 49⟩ Used in section 46.
⟨Turn a neighbouring ray into an st offset 177⟩ Used in section 174.
⟨Turn both normals towards the viewer 197⟩ Used in section 194.
⟨Upload every mip level 151⟩ Used in section 149.
⟨Upload host data to a device buffer 120⟩ Used in section 109.
⟨Upload the flattened scene arrays 141⟩ Used in section 140.
⟨Upload the textures 149⟩ Used in section 109.
⟨Upload the textures and publish the device pointers 145⟩ Used in section 140.
⟨Warn about textured prims without texture coordinates 96⟩ Used in section 46.
⟨Weight the reflected light and fetch it 211⟩ Used in section 207.
⟨Window and SDL objects of the application 219⟩ Used in section 216.
⟨Zoom with the mouse wheel 274⟩ Used in section 269.